

If scripts include the `timeframe` parameter in the indicator declaration, the user can choose the timeframe in which the script runs. The script can set a default timeframe.

By default, the following script plots the current and previous day's opening prices, similar to the previous examples. It is much simpler, but behaves quite differently. For historical bars, the script returns values when the day closes, effectively one day "late". For realtime and elapsed realtime bars, the script returns live values, if the option "Wait for timeframe closes" is not selected in the script settings.

```
//@version=6
indicator("Previous and current day open using `timeframe`", "", true, timeframe = "1D", timeframe_gaps = true)

plot(open[1], "Yesterday's Open", color.red, 2, plot.style_line)
plot(open, "Today's Open", color.green, 2, plot.style_line)
```

Note that:

- Only simple scripts that do not use drawings can use the `timeframe` parameter.
- Scripts that use the `timeframe` parameter can plot values quite differently depending on which settings are chosen. For an explanation, see this Help Center article.

How can I count the occurrences of a condition in the last x bars?

One obvious method is to use a for loop to retrospectively review each of the last x bars and check for the condition. However, this method is inefficient, because it examines all bars in range *again* on every bar, even though it already examined all but the last bar.

In general, using unnecessary, large, or nested for loops can result in slower processing and longer chart loading times.

The simplest and most efficient method is to use the built-in `math.sum()` function, and pass it a conditional series to count. This function maintains a running total of the count as each bar is processed, and can take a simple or series length.

The following example script uses both of these calculation methods. It also uses a series length that adjusts for the first part of the chart, where the number of bars available is less than the length. This way, the functions do not return na values.



Figure 475: image

```

//@version=6
indicator("Number of occurrences demo", overlay = false)

int lengthInput = input.int(100, "Length", minval = 1)

// Condition to count.
bool isUpBar = close > open

// Count using a loop (inefficient).
countWithLoop(bool condition, int length) =>
    int count = 0
    for i = 0 to length - 1
        if condition[i]
            count += 1
    count

// Count using Pine's built-in function. Can be "simple" or "series" length.
countWithSum(bool condition, int length) =>
    float result = math.sum(condition ? 1 : 0, length)

float v1 = countWithSum(isUpBar, math.min(lengthInput, bar_index + 1))
int v2 = countWithLoop(isUpBar, math.min(lengthInput, bar_index + 1))
plot(v1, "Efficient count", color.red, 4)
plot(v2, "Inefficient count", color.black, 1)

```

How can I implement an on/off switch?

An on/off switch is a persistent state that can be turned on once, and persists across bars until it is turned off. Scripts can use the `var` keyword to initialize a variable only once, and maintain its most recent value across subsequent bars unless it is reassigned. Such persistent states can be boolean values, or integers, or any other type.

The following example script show how to implement this. Each instance of the on and off triggers displays with an arrow and the word “On” or “Off”. A green background highlights the bars where the switch is in the “On” state.

```

//@version=6
indicator("On/Off condition example", overlay = true)

bool upBar = close > open

// On/off conditions.
bool triggerOn = upBar and upBar[1] and upBar[2]
bool triggerOff = not upBar and not upBar[1]

// Switch state is saved across bars.
var bool onOffSwitch = false

// Turn the switch on or off, otherwise persist its state.
onOffSwitch := triggerOn ? true : triggerOff ? false : onOffSwitch

bgcolor(onOffSwitch ? color.new(color.green, 90) : na)
plotchar(triggerOn, "triggerOn", " ", location.belowbar, color.lime, size = size.tiny, text = "On")
plotchar(triggerOff, "triggerOff", " ", location.abovebar, color.red, size = size.tiny, text = "Off")

```

How can I alternate conditions?

Scripts can alternate from one state to another strictly, even when the triggers to change state do not occur in strict order. This can be useful to mark only the first trigger and not any subsequent triggers, or to prevent multiple alerts.

The following example script plots all pivots, defined by Williams fractals. These pivots can occur in any order. The script stores the type of the most recent pivot, and confirms the next pivot *only* if it is of the opposite type, such that confirmed pivots appear strictly high-low-high or low-high-low, etc. Confirmed pivots are plotted in a larger size and different color. The chart background color is colored according to the type of the most recent confirmed pivot.



Figure 476: image



Figure 477: image

```

//@version=6
indicator("Alternating states", "", true)

lookback = input.int(2, title="Lookback & Lookahead")

// Define an enum of allowed pivot types.
enum PivotType
    high
    low
    undefined

const color red80 = color.new(color.red, 80)
const color green80 = color.new(color.green, 80)
const color yellow80 = color.new(color.yellow, 80)

// Define a variable of type PivotType to track the pivot direction.
var PivotType lastPivot = PivotType.undefined

// Define pivots.
float pivotLowPrice = ta.pivotlow(lookback, lookback)
float pivotHighPrice = ta.pivohigh(lookback, lookback)
bool isPivotLow = not na(pivotLowPrice)
bool isPivotHigh = not na(pivotHighPrice)

// Plot triangles for pivot points.
plotshape(isPivotLow ? pivotLowPrice : na, "Low", shape.triangleup, location.belowbar, color.yellow,
    offset = -lookback, size = size.tiny)
plotshape(isPivotHigh ? pivotHighPrice : na, "High", shape.triangledown, location.abovebar, color.yellow,
    offset = -lookback, size = size.tiny)

// Confirm highs and lows strictly in order. `PivotType.undefined` handles the case where no pivot has yet occurred.
bool confirmedLow = isPivotLow and (lastPivot == PivotType.high or lastPivot == PivotType.undefined)
bool confirmedHigh = isPivotHigh and (lastPivot == PivotType.low or lastPivot == PivotType.undefined)

// Plot larger triangles for confirmed pivots.
plotshape(confirmedLow ? pivotLowPrice : na, "Low Confirmed", shape.triangleup, location.belowbar, color.green,
    offset = -lookback, size = size.normal)
plotshape(confirmedHigh ? pivotHighPrice : na, "High Confirmed", shape.triangledown, location.abovebar, color.red,
    offset = -lookback, size = size.normal)

// Update last pivot direction.
lastPivot := confirmedLow ? PivotType.low : confirmedHigh ? PivotType.high : lastPivot

// Color the background of the chart based on the direction of the most recent confirmed pivot.
bgcolor(lastPivot == PivotType.low ? green80 : lastPivot == PivotType.high ? red80 :
    lastPivot == PivotType.undefined ? yellow80 : na)

```

Note that:

- The script uses an enum variable with three possible values to store the type of the last pivot and to decide whether to confirm subsequent pivots.
- A single boolean value cannot reliably do this, because boolean values can only be **true** or **false** and not **na**. Using a boolean value can cause unexpected behavior, for example, at the beginning of the chart history where no trigger condition has occurred.
- A pair of boolean variables can replicate this behavior, with careful handling. See the FAQ entry “How can I accumulate a value for two exclusive states?” for an example of using two boolean values in this way.
- A string variable can also do the same thing. The advantage of an enum over a string is that all possible allowed values are known, thus avoiding the case where a condition tests for a value that is misspelled, outdated or otherwise not relevant. Such a test silently fails in every possible case, and the corresponding logic never runs. Such tests can therefore cause bugs that are difficult to find.

Can I merge two or more indicators into one?

It is possible to combine indicators, paying attention to the following points:

- Ensure that the scales that the indicators use are compatible, or re-scale them to be compatible. For example, combining a moving average indicator, designed to overlay the bar chart, with a volume bar indicator that's meant for a separate indicator pane is unlikely to display as expected.
- Check that variable names do not overlap.
- Convert each script to the most recent version of Pine Script™, or at least the same version, before combining them.
- Ensure that there is only one version declaration and script declaration in the resulting script.

How can I rescale an indicator from one scale to another?

Rescaling an indicator from one scale to another means trying to ensure that the values display within a similar range to other values, from the same indicator or from the chart.

For example, consider a script that displays volume typically measuring in the millions of units, and also RSI, which ranges from zero to one hundred. If the script displays these values in the same pane, the volume is visible but the RSI will be so small as to be unreadable.

Where values are dissimilar like this, they must be *rescaled* or *normalized*:

- If the minimum and maximum possible values are known, or *bounded*, the values can be *rescaled*, that is, adjusted to a new range bounded by different maximum and minimum values. Each value differs in absolute terms, but retains the same *relative* proportion to other rescaled values.
- If the values are *unbounded*, meaning that either the maximum or minimum values, or both, are not known, they must instead be *normalized*. Normalizing means scaling the values relative to historical maximum and minimum values. Because the maximum and minimum historical values can change over time as the script runs on more historical and realtime bars, the new scale is *dynamic* and therefore the new values are *not* exactly proportional to each other.

The example script below uses a `rescale()` function to rescale RSI values, and a `normalize()` function to normalize Commodity Channel Index (CCI) and volume values. Although normalizing is an imperfect solution, it is more complete than using `ta.lowest()` and `ta.highest()`, because it uses the minimum and maximum values for the complete set of elapsed bars instead of a subset of fixed length.

```
//@version=6
indicator("Rescaling and normalizing values", "", overlay = false)

// @function      Rescales a signal with a known scale (bounded) to a new scale.
// @param src      (series float) The series to rescale.
// @param oldMin    (simple float) The minimum value of the original signal's scale.
// @param oldMax    (simple float) The maximum value of the original signal's scale.
// @param newMin    (simple float) The minimum value of the new scale.
// @param newMax    (simple float) The maximum value of the new scale.
// @returns        (float) The rescaled value of the signal.
rescale(series float src, simple float oldMin, simple float oldMax, simple float newMin, simple float newMax)
    float result = newMin + (newMax - newMin) * (src - oldMin) / math.max(oldMax - oldMin, 10e-10)

// @function      Rescales a signal with an unknown scale (unbounded) using its historical low and high values.
// @param src      (series float) The series to rescale.
// @param min       (simple float) The minimum value of the rescaled series.
// @param max       (simple float) The maximum value of the rescaled series.
// @returns        (float) The rescaled value of the signal.
normalize(series float src, simple float min, simple float max) =>
    var float historicMin = 10e10
    var float historicMax = -10e10
    historicMin := math.min(nz(src, historicMin), historicMin)
    historicMax := math.max(nz(src, historicMax), historicMax)
    float result = min + (max - min) * (src - historicMin) / math.max(historicMax - historicMin, 10e-10)

// ----- Plot normalized CCI
cci = ta.cci(close, 20)
plot(normalize(cci, 100, 300), "Normalized CCI", #2962FF)
// Arbitrary and inexact equivalent of 100 and -100 levels rescaled to the 100/300 scale.
```

```

band00 = hline(150, "Lower Band", color.new(#C0C0C0, 90), hline.style_solid)
band01 = hline(250, "Upper Band", color.new(#C0C0C0, 90), hline.style_solid)
fill(band01, band00, color.new(#21328F, 80), "Background")

// ----- Plot normalized volume in the same region as the rescaled RSI
color volColor = close > open ? #26a69a : #ef5350
plot(normalize(volume, -100, 100), "Normalized volume", volColor, style = plot.style_columns, histbase = -100)
hline(100, "", color.new(color.gray, 50), hline.style_dashed)
hline(-100, "", color.new(color.gray, 50), hline.style_solid)

// ----- Plot rescaled RSI
plot(rescale(ta.rsi(close, 14), 0, 100, -100, 100), "Rescaled RSI", #8E1599)
hline(0, "RSI 50 level", color.new(color.gray, 70), hline.style_solid)
// Precise equivalent of 70 and 30 levels rescaled to the -100/100 scale.
band10 = hline(-40, "Lower Band", color.new(#9915FF, 80), hline.style_solid)
band11 = hline(40, "Upper Band", color.new(#9915FF, 80), hline.style_solid)
fill(band11, band10, color.new(#9915FF, 90), "Background")

// ----- Plot original values in Data Window
plot(na, "", display = display.data_window)
plot(cci, "Original CCI", display = display.data_window)
plot(volume, "Original volume", display = display.data_window)
plot(ta.rsi(close, 14), "Original RSI", display = display.data_window)

```



Figure 478: image

How can I calculate my script's run time?

Programmers can measure the time that a script takes to run and see detailed information about which parts of the code take longest in the Pine Profiler. See the section of the User Manual on Profiling and optimization for more information.

How can I save a value when an event occurs?

To save a value when an event occurs, use a *persistent variable*. Scripts declare persistent variables by using the `var` keyword. Such variables are initialized only once, at `bar_index` zero, instead of on each bar, and maintain the same value after that unless changed.

In the following example script, the `var` keyword allows the `priceAtCross` variable to maintain its value between bars until a crossover event occurs, when the script updates the variable with the current close price. The `:=` reassignment operator ensures that the global variable `priceAtCross` is modified. Using the `=` assignment operator instead would create a new local variable that is inaccessible outside the `if` block. The new local variable would have the same name as the global variable, which is called *shadowing*. The compiler warns about shadow variables.

```
//@version=6
indicator("Save a value when an event occurs", "", true)
float hiHi = ta.highest(high, 5)[1]
var float priceAtCross = na
if ta.crossover(close, hiHi) // When a crossover occurs, assign the current close price to `priceAtCross`.
    priceAtCross := close
plot(hiHi)
plot(priceAtCross, "Price At Cross", color.orange, 3, plot.style_circles)
```

How can I count touches of a specific level?

The most efficient way to count touches of a specific level is by tracking the series on each bar. A robust approach requires maintaining separate tallies for up and down bar touches and taking into account any gaps across the level. Using loops instead would be inefficient and impractical in this case.

The following example script records a value of 1 in a series whenever a touch occurs, and uses the `math.sum()` function to count these instances within the last `touchesLengthInput` bars. This script displays the median and touches on the chart using the `force_overlay` parameter of the `plot*()` functions, and displays the count in a separate pane.



Figure 479: image

```
//@version=6
```

```

indicator("Median Touches", "", overlay = false)

int medianLengthInput = input.int(100, "Median calculation: Number of previous closes")
int touchesLengthInput = input.int(50, "Number of previous bars to check for price touches")
float median = ta.percentile_nearest_rank(close, medianLengthInput, 50)
// Don't count neutral touches when price doesn't move.
bool barUp = close > open
bool barDn = close < open
// Bar touches median.
bool medianTouch = high > median and low < median
bool gapOverMedian = high[1] < median and low > median
bool gapUnderMedian = low[1] > median and high < median
// Record touches.
int medianTouchUp = medianTouch and barUp or gapOverMedian ? 1 : 0
int medianTouchDn = medianTouch and barDn or gapUnderMedian ? 1 : 0
// Count touches over the last n bars.
float touchesUp = math.sum(medianTouchUp, touchesLengthInput)
float touchesDn = math.sum(medianTouchDn, touchesLengthInput)
// ----- Plots
// Markers
plotchar(medianTouchUp, "medianTouchUp", " ", location.belowbar, color.lime, force_overlay = true)
plotchar(medianTouchDn, "medianTouchDn", " ", location.abovebar, color.red, force_overlay = true)
// Median
plot(median, "Median", color.orange, force_overlay = true)
// Base areas.
plot(touchesUp, "Touches Up", color.green, style = plot.style_columns)
plot(-touchesDn, "Touches Dn", color.maroon, style = plot.style_columns)
// Exceeding area.
float minTouches = math.min(touchesUp, touchesDn)
bool minTouchesIsUp = touchesUp < touchesDn
basePlus = plot(minTouches, "Base Plus", display = display.none)
hiPlus = plot(not minTouchesIsUp ? touchesUp : na, "High Plus", display = display.none)
baseMinus = plot(-minTouches, "Base Plus", display = display.none)
loMinus = plot(minTouchesIsUp ? -touchesDn : na, "Low Minus", display = display.none)
fill(basePlus, hiPlus, color.lime)
fill(baseMinus, loMinus, color.red)

```

How can I know if something is happening for the first time since the beginning of the day?

One way is to use the `ta.barssince()` function to check if the number of bars since the last occurrence of a condition, plus one, is greater than the number of bars since the beginning of the new day.

Another method is to use a *persistent state* to decide whether an *event* can happen. When the timeframe changes to a new day, the state is reset to allow the event. If the condition occurs while the state allows it, an event triggers. When the event triggers, the state is set so as not to allow the event.

The following example script shows both methods.

```

//@version=6
indicator("First time today example", "", true)

bool isUpCandle = close > open

// ----- Method 1.
int barsSincePreviousUpCandle = ta.barssince(isUpCandle[1])
int barsSinceStartOfDay = ta.barssince(timeframe.change("1D")) - 1
bool previousUpCandleWasNotToday = barsSincePreviousUpCandle > barsSinceStartOfDay
bool isFirstToday1 = isUpCandle and previousUpCandleWasNotToday
plotchar(isFirstToday1, "isFirstToday1", "•", location.top, color = color.silver, size = size.normal)

plot(barsSinceStartOfDay, "barsSinceStartOfDay", display=display.data_window)

```




Figure 480: image

```
// ----- Method 2.
var bool hadUpCandleToday = false // This is a persistent state.
bool isFirstToday2 = false // This is a one-off event.
if timeframe.change("1D") // When the day begins..
    hadUpCandleToday := false // we have not yet had an up candle today, so reset the state.
if isUpCandle and not hadUpCandleToday // If this is the first up candle today..
    hadUpCandleToday := true // set the persistent state
    isFirstToday2 := true // and update the event.
plotchar(isFirstToday2, "isFirstToday2", "•", location.top, color = color.yellow, size = size.small)
```

How can I optimize Pine Script™ code?

Optimizing Pine Script™ code can make scripts run faster and use less memory. For large or complex scripts, optimization can avoid scripts reaching the computational limits.

The Pine Profiler analyzes all significant code in a script and displays how long each line or block takes to run. Before optimizing code, run the Pine Profiler to identify which parts of the code to optimize first. The Pine Profiler section of the User Guide contains an extensive discussion of how to optimize code. In addition, consider the following tips:

- Use strategy scripts only to model trades. Otherwise, use indicator scripts, which are faster.
- Become familiar with the Pine execution model and time series to structure code effectively.
- Declare variables with the `var` keyword when initialization involves time-consuming operations like complex functions, arrays, objects, or string manipulations.
- Keep operations on strings to a necessary minimum, because they can be more resource-intensive than operations on other types.
- Using built-in functions is usually faster than writing custom functions that do the same thing. Sometimes, alternative logic can be more efficient than using standard functions. For example, use a persistent variable when an event occurs, to avoid using `ta.valuewhen()`, as described in the FAQ entry [How can I save a value when an event occurs?](#). Or save the `bar_index` when a condition occurs to avoid using `ta.barssince()`, as described in the FAQ entry [How to remember the last time a condition occurred?](#).

How can I access a stock's financial information?

In Pine, the `request.financial()` function can directly request financial data.

On the chart, users can open financial indicators in the “Financials” section of the “Indicators, Metrics & Strategies” window.

How can I find the maximum value in a set of events?

Finding the maximum value of a variable that has a meaningful value *on every bar*, such as the high or low in price, is simple, using the `ta.highest()` function.

However, if the values do not occur on every bar, we must instead store each value when it occurs and then find the maximum. The most flexible way to do this is by using an array.

The following example script stores pivot highs in a fixed-length array. The array is managed as a queue: the script adds new pivots to the end, and removes the oldest element from the array. To identify the highest value among the stored pivots, we use the `array.max()` function and plot this maximum value on the chart. Additionally, we place markers on the chart to indicate when the pivots are detected, and the bars where the pivots occurred. By definition, these points are not the same, because a pivot is only confirmed after a certain number of bars have elapsed.

```
//@version=6
indicator("Max pivot demo", "", true)
// Create inputs to specify the pivot legs and the number of last pivots to keep to compare.
int pivotLengthInput = input.int(5, "Pivot length", minval = 1)
int numPivotsInput   = input.int(3, "Number of pivots to check")
// Initialize an array with a size based on the number of recent pivots to evaluate.
var array<float> pivotsArray = array.new<float>(numPivotsInput)
// Find the pivot value and set up a condition to verify if a value has been found.
float ph = ta.pivohigh(pivotLengthInput, pivotLengthInput)
bool newPH = not na(ph)
// When a new pivot is found, add it to the array and discard the oldest value.
if newPH
    pivotsArray.push(ph)
    pivotsArray.shift()
// Display the max value from the array on the chart, along with markers indicating the positions and detection.
plot(pivotsArray.max())
plotchar(newPH, "newPH", "•", location.abovebar, offset = - pivotLengthInput)
plotchar(newPH, "newPH", " ", location.top)
```

How can I display plot values in the chart's scale?

To display the names and values of plots from an indicator in the chart's scale, right-click on the chart to open the chart “Settings” menu. In the “Scales and lines” tab, select “Name” and “Value” from the “Indicators and financials” drop-down menu.

How can I reset a sum on a condition?

To sum a series of values, initialize a persistent variable by using the `var` keyword to track the sum. Then use a logical test to reset the values when a condition occurs.

In the following example script, we initialize a persistent variable called `cumulativeVolume` to track the sum of the volume. Then we reset it to zero on a Moving Average Convergence/Divergence (MACD) cross up or down.

We plot the cumulative volume on the chart, as well as arrows to show the MACD crosses.

```
//@version=6
indicator("Reset sum on condition example", overlay = false)
const color TEAL = color.new(color.teal, 50)
const color RED  = color.new(color.red, 50)
[macdLine, signalLine, _] = ta.macd(close, 12, 26, 9)
bool crossUp = ta.crossover(macdLine, signalLine)
bool crossDn = ta.crossunder(macdLine, signalLine)
bool doReset = crossUp or crossDn
var float cumulativeVolume = na
```

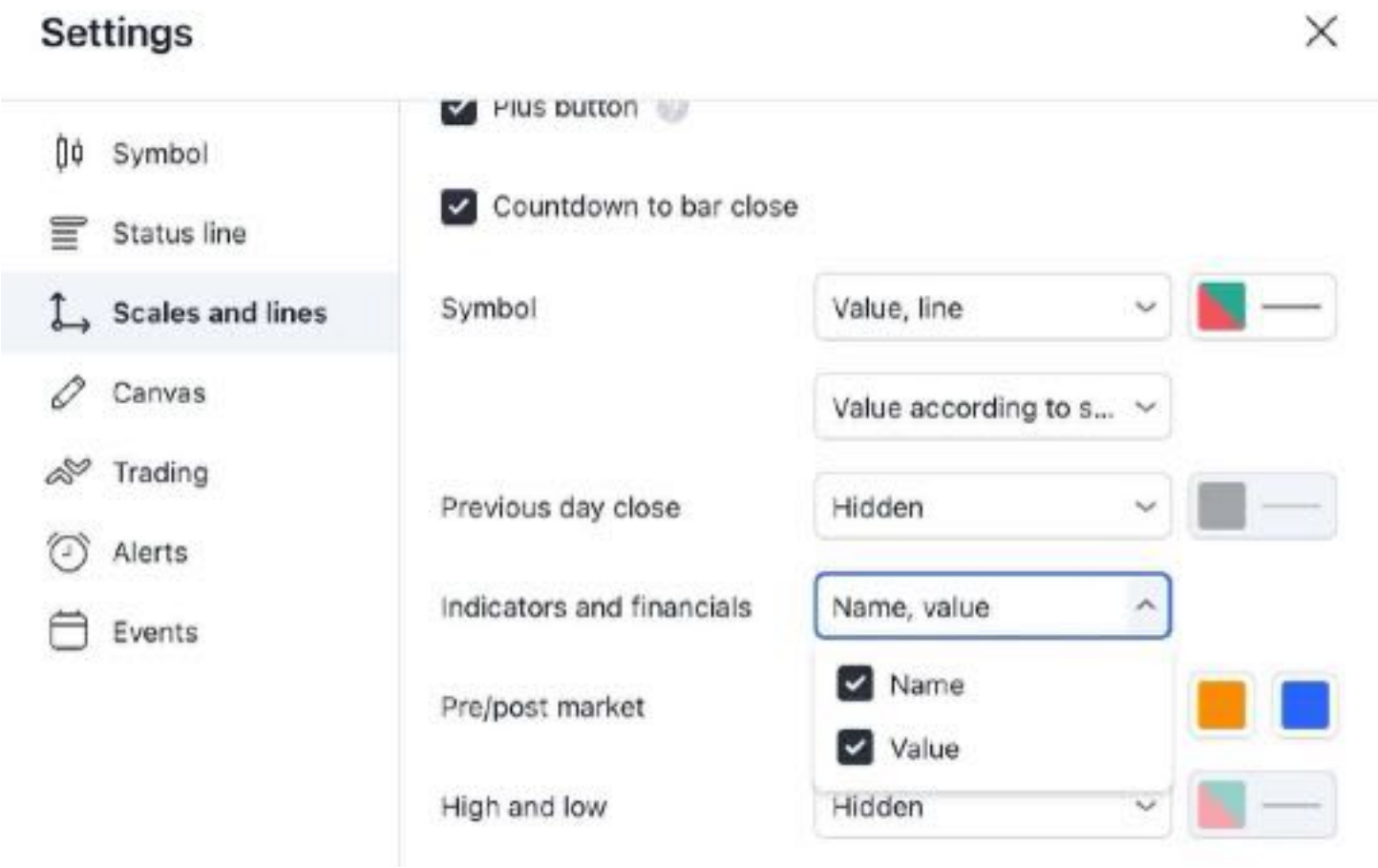


Figure 481: image



Figure 482: image

```

cumulativeVolume += volume // On every bar, we sum the volume.
cumulativeVolume := doReset ? 0. : cumulativeVolume // But when we get a cross, we reset it to zero.
plot(cumulativeVolume, "Cumulative volume", close >= open ? TEAL : RED, 1, plot.style_columns)
plotshape(crossUp, "crossDn", shape.arrowup, location.top, color.lime)
plotshape(crossDn, "crossUp", shape.arrowdown, location.top, color.fuchsia)

```

Note that:

- In the `ta.macd()` function call, we only require two of the three values returned in the tuple. To avoid unnecessary variable declarations, we assign the third tuple value to an underscore. Here, the underscore acts like a dummy variable.

How can I accumulate a value for two exclusive states?

Consider a simple indicator defined by two exclusive states: *buy* and *sell*. The indicator cannot be in both *buy* and *sell* states simultaneously. In the *buy* state, the script accumulates the volume of shares being traded. In the *sell* state, the accumulation of volume begins again from zero.

There are different ways to code this kind of logic. See the FAQ entry “How can I alternate conditions” for an example of using an enum to manage two exclusive states. The following example script uses two boolean variables to do the same thing.

Additionally, this script demonstrates the concept of *events* and *states*. An event is a condition that occurs on one or more arbitrary bars. A state is a condition that persists over time. Typically, programmers use events to turn states on and off. In turn, states can allow or prevent other processing.

The script plots arrows for events, which are based on rising or falling values of the close price. These events determine which of the two exclusive states is active; the script colors the background according to the current state. The script accumulates bullish and bearish volume only in the corresponding bullish or bearish state, displaying it in a Weis Wave fashion.

```

//@version=6
indicator("Cumulative volume", "")

bool upEvent = ta.rising(close, 2)

```



Figure 483: image

```

bool dnEvent = ta.falling(close, 2)

var bool upState = false, var bool dnState = false
// When the right event occurs, turn the state on; when a counter-event occurs, turn it off; otherwise, persist
upState := upEvent ? true : dnEvent ? false : upState
dnState := upEvent ? false : dnEvent ? true : dnState

var float volUp = na, var float volDn = na

if upState // For every bar that we are in the up state,
    volUp += volume // sum the up volume.
if dnState
    volDn += volume

if upEvent // If we change state to up,
    volDn := 0 // reset the down volume.
if dnEvent
    volUp := 0

plot(+volUp, "Up Volume", color.green, 4, plot.style_columns)
plot(-volDn, "Dn Volume", color.maroon, 4, plot.style_columns)
plotchar(upEvent, "Up Event", " ", location.bottom, color.green, size = size.tiny)
plotchar(dnEvent, "Dn Event", " ", location.top, color.maroon, size = size.tiny)
bgcolor(upState ? color.new(color.green, 90) : dnState ? color.new(color.red, 90) : na)

```

Note that:

- Equivalent logic using ternary conditions is smaller and potentially more efficient, but not as easy to read, extend, or debug. This more verbose logic illustrates the concepts of events and states, which can apply to many types of scripting problems. This logic is an extension of the on-off switch in the FAQ entry “How can I implement an on/off switch?”.

- When using states, it is important to make the conditions for resetting states explicit, to avoid unforeseen problems.
- Displaying all events and states during script development, either on the chart or in the Data Window, helps debugging.

How can I organize my script's inputs in the Settings/Inputs tab?

A script's plots and inputs constitute its user interface. The following example script uses the following techniques to organize inputs for greater clarity:

- **Grouping inputs:** Create a section header for a group of inputs by using the `group` parameter in the `input()` functions. Use constants for group names to simplify any potential name changes.
- **Visual boundaries:** Use ASCII characters to create separators, establishing visual boundaries for distinct group sections. For continuous separator lines, reference group headers 1 and 2 in our script below, which use ASCII characters 205 or 196. Conversely, the dash (ASCII 45) and Em dash (ASCII 151), shown in group headers 3 and 4, do not join continuously, resulting in a less visually appealing distinction. Note that Unicode characters might display differently across different machines and browsers, potentially altering their appearance or spacing for various users.
- **Indentation of sub-sections:** For a hierarchical representation, use Unicode whitespace characters to indent input sub-sections. Group 3 in our script uses the Em space () 8195 (0x2003) to give a tab-like spacing.
- **Vertical alignment of inlined inputs:** In our script, Group 1 shows how vertical alignment is difficult when inline inputs have varied title lengths. To counteract this misalignment, Group 2 uses the Unicode EN space (): 8194 (0x2002) for padding, since regular spaces are stripped from the label. For precise alignment, use different quantities and types of Unicode spaces. See here for a list of Unicode spaces of different widths. Note that, much like the separator characters, the rendering of these spaces might differ across browsers and machines.
- **Placing inputs on one line:** Add multiple related inputs into a single line using the `inline` parameter. Group 4 in our script adds the title argument to just the first input and skips it for the others.

```
//@version=6
indicator("Inputs", overlay = true)

// Defining options strings improves script readability.
// It also enables the creation of boolean variables by comparing these constants with user input strings in a
string EQ1 = "On"
string EQ2 = "Off"

// The `GRP*` strings used for group headers demonstrate using ASCII characters to create a visual boundary,
// making it easier for users to differentiate between different sections in the menu.

// Group 1 demonstrates inline inputs that do not align vertically in the menu.
string GRP1 = "          Settings          " // ASCII 205
float  ao1SrcInput  = input.source(close, "A0 source",      inline = "11", group = GRP1)
int    ao1LenInput  = input.int(14,         "Length",       inline = "11", group = GRP1)
float  long1SrcInput = input.source(close, "Signal source", inline = "12", group = GRP1)
int    long1LenInput = input.int(3,         "Length",       inline = "12", group = GRP1)

// In Group 2, the title of `ao2SrcInput` is padded with three Unicode EN spaces (U+2002) to compensate for the
string GRP2 = "          Settings          " // ASCII 196
float  ao2SrcInput  = input.source(close, "A0 source  ", inline = "21", group = GRP2)
int    ao2LenInput  = input.int(14,         "Length",       inline = "21", group = GRP2)
float  long2SrcInput = input.source(close, "Signal source", inline = "22", group = GRP2)
int    long2LenInput = input.int(3,         "Length",       inline = "22", group = GRP2)

// This configuration uses Unicode white space characters to indent input sub-sections. We use Em space ( ): 8194
string GRP3 = "----- Settings -----" // ASCII 151 (Em dash)
float  level1Input  = input.float(65.,     "First level",      group = GRP3)
float  level2Input  = input.float(65.,     "  Second Level",      group = GRP3)
bool   level3Input  = input.string(EQ1,    "  Checkbox equivalent", group = GRP3, options = [EQ1, EQ2])
float  level4Input  = input.float(65.,     "Widest Legend        ", group = GRP3)

// These options demonstrate the use of the `inline` parameter to create structured blocks of inputs that are
string GRP4 = "----- Settings -----" // ASCII 45 (dash)
bool   showMa1Input = input(true,         "MA #1", inline = "1", group = GRP4)
string ma1TypeInput  = input.string("SMA", "", inline = "1", group = GRP4, options = ["SMA", "EMA", "SMM"])
```

```

float  ma1SourceInput = input(close,      "",      inline = "1", group = GRP4)
int    ma1LengthInput = input.int(20,    "",      inline = "1", group = GRP4, minval = 1)
color  ma1ColorInput  = input(#f6c309,   "",      inline = "1", group = GRP4)

bool   showMa2Input   = input(true,      "MA №2", inline = "2", group = GRP4)
string ma2TypeInput   = input.string("SMA", "",      inline = "2", group = GRP4, options = ["SMA", "EMA", "SMM
float  ma2SourceInput = input(close,     "",      inline = "2", group = GRP4)
int    ma2LengthInput = input.int(50,    "",      inline = "2", group = GRP4, minval = 1)
color  ma2ColorInput  = input(#fb9800,   "",      inline = "2", group = GRP4)

bool   showMa3Input   = input(true,      "MA №3", inline = "3", group = GRP4)
string ma3TypeInput   = input.string("SMA", "",      inline = "3", group = GRP4, options = ["SMA", "EMA", "SMM
float  ma3SourceInput = input(close,     "",      inline = "3", group = GRP4)
int    ma3LengthInput = input.int(100,   "",      inline = "3", group = GRP4, minval = 1)
color  ma3ColorInput  = input(#fb6500,   "",      inline = "3", group = GRP4)

// @function      Calculates various types of moving averages for the `source` based on the specified `m
// @param series  (series float) Series of values to process.
// @param length  (simple int) Number of bars (length).
// @param maType  (simple string) The type of moving average to calculate.
//               Options are "SMA", "EMA", "SMMA (RMA)", "WMA", and "VWMA".
// @returns      (float) The moving average of the `source` for `length` bars back.
ma(series float source, simple int length, simple string maType) =>
    switch maType
        "SMA"      => ta.sma(source, length)
        "EMA"      => ta.ema(source, length)
        "SMMA (RMA)" => ta.rma(source, length)
        "WMA"      => ta.wma(source, length)
        "VWMA"     => ta.vwma(source, length)
    =>
        na

// Calculate the moving averages with the user-defined settings.
float ma1 = ma(ma1SourceInput, ma1LengthInput, ma1TypeInput)
float ma2 = ma(ma2SourceInput, ma2LengthInput, ma2TypeInput)
float ma3 = ma(ma3SourceInput, ma3LengthInput, ma3TypeInput)

// Plot the moving averages, if each checkbox is enabled.
plot(showMa1Input ? ma1 : na, "MA №1", ma1ColorInput)
plot(showMa2Input ? ma2 : na, "MA №2", ma2ColorInput)
plot(showMa3Input ? ma3 : na, "MA №3", ma3ColorInput)

```

Tips:

- Order the inputs to prioritize user convenience rather than to reflect the order used in the script's calculations.
- Never use two checkboxes for mutually exclusive selections. Use dropdown menus instead.
- Remember that dropdown menus can accommodate long strings.
- Provide adequate minimum and maximum values for numeric values, selecting the proper float or int type.
- Customize step values based on the specific needs of each input.
- Because checkboxes cannot be indented, use the input() function's **options** parameter to create dropdown selections so that the sections appear more organized compared to using checkboxes.
- Observe how the **level3Input** is calculated as a boolean variable by comparing the input with the EQ1 "ON" constant. This method provides a visually appealing indented on-off switch in the menu without adding complexity to the code.
- For a consistent visual appearance, vertically center the separator titles across all inputs. Due to the proportional spacing of the font, achieving this might require some trial and error.
- To ensure that separators align just slightly to the left of the furthest edge of dropdowns, begin with the longest input title, because it sets the width of the window.
- To avoid adjusting separators if the longest input title is shorter than initially anticipated, extend its length using Unicode white space. Refer to the code example for input **level4Input** for a demonstration.

[Previous

Error messages

The if statement is too long

This error occurs when the indented code inside an if statement is too large for the compiler. Because of how the compiler works, you won't receive a message telling you exactly how many lines of code you are over the limit. The only solution now is to break up your if statement into smaller parts (functions or smaller if statements). The example below shows a reasonably lengthy if statement; theoretically, this would throw **line 4: if statement is too long**:

```
//@version=6
indicator("My script")

var e = 0
if barstate.islast
    a = 1
    b = 2
    c = 3
    d = 4
    e := a + b + c + d
```

```
plot(e)
```

To fix this code, you could move these lines into their own function:

```
//@version=6
indicator("My script")

var e = 0
doSomeWork() =>
    a = 1
    b = 2
    c = 3
    d = 4

    result = a + b + c + d

if barstate.islast
    e := doSomeWork()

plot(e)
```

Script requesting too many securities

The maximum number of securities in script is limited to 40. If you declare a variable as a `request.security` function call and then use that variable as input for other variables and calculations, it will not result in multiple `request.security` calls. But if you will declare a function that calls `request.security` — every call to this function will count as a `request.security` call.

It is not easy to say how many securities will be called looking at the source code. Following example have exactly 3 calls to `request.security` after compilation:

```
//@version=6
indicator("Securities count")
a = request.security(syminfo.tickerid, '42', close) // (1) first unique security call
b = request.security(syminfo.tickerid, '42', close) // same call as above, will not produce new security call

plot(a)
plot(a + 2)
plot(b)
```

```

sym(p) => // no security call on this line
    request.security(syminfo.tickerid, p, close)
plot(sym('D')) // (2) one indirect call to security
plot(sym('W')) // (3) another indirect call to security

c = request.security(syminfo.tickerid, timeframe.period, open) // result of this line is never used, and will

```

Script could not be translated from: null

```
study($)
```

Usually this error occurs in version 1 Pine scripts, and means that code is incorrect. Pine Script™ of version 2 (and higher) is better at explaining errors of this kind. So you can try to switch to version 2 by adding a special attribute in the first line. You'll get line 2: no viable alternative at character '\$':

```

// @version=2
study($)
```

line 2: no viable alternative at character '\$'

This error message gives a hint on what is wrong. \$ stands in place of string with script title. For example:

```

// @version=2
study("title")
```

Mismatched input <...> expecting <??>

Same as no viable alternative, but it is known what should be at that place. Example:

```

//@version=6
indicator("My Script")
    plot(1)
```

line 3: mismatched input 'plot' expecting 'end of line without line continuation'

To fix this you should start line with plot on a new line without an indent:

```

//@version=6
indicator("My Script")
plot(1)
```

Loop is too long (> 500 ms)

We limit the computation time of loop on every historical bar and realtime tick to protect our servers from infinite or very long loops. This limit also fail-fast indicators that will take too long to compute. For example, if you'll have 5000 bars, and indicator takes 500 milliseconds to compute on each of bars, it would have result in more than 16 minutes of loading:

```

//@version=6
indicator("Loop is too long", max_bars_back = 101)
s = 0
for i = 1 to 1e3 // to make it longer
    for j = 0 to 100
        if timestamp(2017, 02, 23, 00, 00) <= time[j] and time[j] < timestamp(2017, 02, 23, 23, 59)
            s := s + 1
plot(s)
```

It might be possible to optimize algorithm to overcome this error. In this case, algorithm may be optimized like this:

```

//@version=6
indicator("Loop is too long", max_bars_back = 101)
bar_back_at(t) =>
    i = 0
    step = 51
    for j = 1 to 100
        if i < 0
```

```

        i := 0
        break
    if step == 0
        break
    if time[i] >= t
        i := i + step
        i
    else
        i := i - step
        i
    step := step / 2
    step
i

s = 0
for i = 1 to 1e3 // to make it longer
    s := s - bar_back_at(timestamp(2017, 02, 23, 23, 59)) +
        bar_back_at(timestamp(2017, 02, 23, 00, 00))
s
plot(s)

```

Script has too many local variables

This error appears if the script is too large to be compiled. A statement **var=expression** creates a local variable for **var**. Apart from this, it is important to note, that auxiliary variables can be implicitly created during the process of a script compilation. The limit applies to variables created both explicitly and implicitly. The limitation of 1000 variables is applied to each function individually. In fact, the code placed in a *global* scope of a script also implicitly wrapped up into the main function and the limit of 1000 variables becomes applicable to it. There are few refactorings you can try to avoid this issue:

```

var1 = expr1
var2 = expr2
var3 = var1 + var2

```

can be converted into:

```

var3 = expr1 + expr2

```

Pine Script™ cannot determine the referencing length of a series. Try using max_bars_back in the indicator or strategy function

The error appears in cases where Pine Script™ wrongly autodetects the required maximum length of series used in a script. This happens when a script's flow of execution does not allow Pine Script™ to inspect the use of series in branches of conditional statements (**if**, **iff** or **?**), and Pine Script™ cannot automatically detect how far back the series is referenced. Here is an example of a script causing this problem:

```

//@version=6
indicator("Requires max_bars_back")
test = 0.0
if bar_index > 1000
    test := ta.roc(close, 20)
plot(test)

```

In order to help Pine Script™ with detection, you should add the **max_bars_back** parameter to the script's **indicator** or **strategy** function:

```

//@version=6
indicator("Requires max_bars_back", max_bars_back = 20)
test = 0.0
if bar_index > 1000
    test := ta.roc(close, 20)
plot(test)

```

You may also resolve the issue by taking the problematic expression out of the conditional branch, in which case the **max_bars_back** parameter is not required:

```
//@version=6
indicator("My Script")
test = 0.0
roc20 = ta.roc(close, 20)
if bar_index > 1000
    test := roc20
plot(test)
```

In cases where the problem is caused by a **variable** rather than a built-in **function** (**vwma** in our example), you may use the **max_bars_back** function to explicitly define the referencing length for that variable only. This has the advantage of requiring less runtime resources, but entails that you identify the problematic variable, e.g., variable **s** in the following example:

```
//@version=6
indicator("My Script")
f(off) =>
    t = 0.0
    s = close
    if bar_index > 242
        t := s[off]
    t
plot(f(301))
```

This situation can be resolved using the **max_bars_backfunction** to define the referencing length of variable **s** only, rather than for all the script's variables:

```
//@version=6
indicator("My Script")
f(off) =>
    t = 0.0
    s = close
    max_bars_back(s, 301)
    if bar_index > 242
        t := s[off]
    t
plot(f(301))
```

When using drawings that refer to previous bars through **bar_index[n]** and **xloc = xloc.bar_index**, the time series received from this bar will be used to position the drawings on the time axis. Therefore, if it is impossible to determine the correct size of the buffer, this error may occur. To avoid this, you need to use **max_bars_back(time, n)**. This behavior is described in more detail in the section about drawings.

Memory limits exceeded. The study allocates X times more than allowed

The most common cause for this error is returning objects and collections from **request.*()** functions. Other possible causes include unnecessary drawing updates, excess historical buffer capacity, or inefficient use of **max_bars_back**.

Returning collections from **request.*()** functions

A common source of the “*Memory limits exceeded*” error is returning objects or collections from another chart symbol or timeframe using **request.*()** functions.

When requesting data from other contexts, the data for *each bar* is copied and stored in memory to allow the script to reference it later in the main context. This can use a lot of memory, depending on the data. Requesting large collections can easily lead to excessive memory consumption.

Let's look at an example script where we request data to calculate the balance of power (BOP) for the symbol at a higher timeframe. Here, the *request expression* is a custom function that populates a persistent array with our calculated BOP values, returning the *full array* to the main context on *each* bar. We intend to use these stored array values to calculate and plot the average BOP in the main context. However, returning every array instance consumes a lot of memory, and so this script can throw a memory error on charts with a sufficiently long history:

```
//@version=6
indicator("BOP array in higher timeframe context", "Memory limit demo")
```

```

//@variable User-input length for calculating average of BOP values.
int avgLength = input.int(5, "Average BOP Length", minval = 1)

//Returns a copy of the `dataArray` on every bar, which uses a lot of memory.
dataFunction() =>
    //@variable Persistent array containing the "balance of power" (BOP) values for all bars from the higher t
    var array<float> dataArray = array.new_float(0)

    //@variable The "balance of power" percentage calculated for the current bar.
    float bop = (close - open) / (high - low) * 100
    dataArray.push(bop)

    //Return the full collection.
    dataArray

// Request the full BOP array from the 1D timeframe.
array<float> reqData = request.security(syminfo.tickerid, "1D", dataFunction())

// Plot zero line.
hline(0, "Zero line", color.gray, hline.style_dotted)

// Latest BOP value and average BOP are calculated in the main context if `reqData` is not `na`.
//@variable The latest BOP value from the `reqData` array.
float latestValue = na
//@variable The average of the last `avgLength` BOP values.
float avgBOP = na

if not na(reqData)
    // Retrieve BOP value for the current main context bar.
    latestValue := reqData.last()

    // Calculate the average BOP for the most-recent values from the higher timeframe array.
    //@variable Size of the `reqData` array returned from the higher timeframe.
    int dataSize = reqData.size()
    //@variable A subset of the latest values from the `reqData` array. Its size is determined by the `avgLeng
    array<float> lastValues = dataSize >= avgLength ? reqData.slice(dataSize - avgLength, dataSize): reqData
    avgBOP := lastValues.avg()

// Plot the BOP value and average line.
color plotColor = latestValue >= 0 ? color.aqua : color.orange
plot(latestValue, "BOP", plotColor, style = plot.style_columns)
plot(avgBOP, "Avg", color.purple, linewidth = 3)

```

How do I fix this?

Optimize requests and limit the data returned to the main context to ensure that only the *minimum necessary* data is stored in memory.

If possible, try to return *calculated results* directly rather than returning the collections themselves, or only return collections *conditionally*, when they are necessary in the main context.

Let's consider a few common scenarios where scripts need specific data in the main context.

Return last state only If a script needs only the *last state* of a requested collection in the main context: use an `ifbarstate.islast` condition to return a copy of the *last bar's collection* only.

Here, we modified our script to display only the *latest* average BOP (a single value), rather than plotting an average line. The updated request function now returns the calculated BOP values directly for each bar, and returns the higher timeframe's array only on the last bar:

```

//@version=6
indicator("BOP array on last bar", "Memory limit demo")

```



Figure 484: image

```

//@variable User-input length for calculating average of BOP values.
int avgLength = input.int(5, "Average BOP Length", minval = 1)

// Returns the calculated `bop` each bar, and a copy of the `dataArray` on the last bar or `na` otherwise.
dataFunction() =>
    //@variable Persistent array containing the "balance of power" (BOP) values for all higher timeframe bars.
    var array<float> dataArray = array.new_float(0)

    //@variable The "balance of power" percentage calculated for the current higher timeframe bar.
    float bop = (close - open) / (high - low) * 100
    dataArray.push(bop)

    // Return the collection on the last bar only.
    if barstate.islast
        [bop, dataArray]
    else
        [bop, na]

// Request calculated BOP value, and BOPs array if on last bar, from the higher timeframe.
[reqValue, reqData] = request.security(syminfo.tickerid, "1D", dataFunction())

// Plot zero line.
hline(0, "Zero line", color.gray, hline.style_dotted)

// Plot the BOP value for each main context bar.
color plotColor = reqValue >= 0 ? color.aqua : color.orange
plot(reqValue, "BOP", plotColor, style = plot.style_columns)

// Calculate the average BOP for most-recent values from the higher timeframe array, and display result in a table.
if not na(reqData)
    //@variable Size of the `reqData` array returned from the higher timeframe.
    int dataSize = reqData.size()
    //@variable A subset of the latest values from the `reqData` array. Its size is determined by the `avgLength`
    array<float> lastValues = dataSize >= avgLength ? reqData.slice(dataSize - avgLength, dataSize): reqData
    //@variable The average of the last `avgLength` BOP values.
    float avgBOP = lastValues.avg()

    // Display latest average value in a single-cell table.
    var table displayTable = table.new(position.bottom_right, 1, 1, color.purple)
    displayTable.cell(0, 0, "Avg of last " + str.tostring(avgLength) + " BOPs: " + str.tostring(avgBOP, "##.##"),
        text_color = color.white)

```

Return calculated results If a script needs the *result* of a calculation on a collection, but does not need the collection itself in the main context, use a user-defined function as the request expression. The function can calculate on the collection in the *requested* context and return only the result to the main context.

For example, we can calculate the average BOP directly within our request function. Therefore, only the calculated values are stored in memory, and the request expression returns a tuple (current BOP and average BOP) to plot the results in the main context:

```

//@version=6
indicator("Return BOP results only", "Memory limit demo")

//@variable User-input length for calculating average of BOP values.
int avgLength = input.int(5, "Average BOP Length", minval = 1)

// Returns the calculated `bop` and `avgBOP` values directly.
dataFunction() =>
    //@variable Persistent array containing the "balance of power" (BOP) values for all higher timeframe bars.
    var array<float> dataArray = array.new_float(0)

```




Figure 485: image

```

//@variable The "balance of power" percentage calculated for the current higher timeframe bar.
float bop = (close - open) / (high - low) * 100
dataArray.push(bop)

// Calculate the average BOP for the `avgLength` most-recent values.
//@variable Size of the `dataArray`.
int dataSize = dataArray.size()
//@variable A subset of the latest values from the `dataArray`. Its size is determined by the `avgLength`
array<float> lastValues = dataSize >= avgLength ? dataArray.slice(dataSize - avgLength, dataSize): dataArray
//@variable The average of the last `avgLength` BOP values.
float avgBOP = lastValues.avg()

//Return the calculated results.
[bop, avgBOP]

// Request BOP and average BOP values from the higher timeframe.
[reqValue, reqAverage] = request.security(syminfo.tickerid, "1D", dataFunction())

// Plot zero line.
hline(0, "Zero line", color.gray, hline.style_dotted)

// Plot the BOP value and average line.
color plotColor = reqValue >= 0 ? color.aqua : color.orange
plot(reqValue, "BOP", plotColor, style = plot.style_columns)
plot(reqAverage, "Avg", color.purple, linewidth = 3)

Return the collection on some bars If a script needs the collection itself in the main context, but *not for**every bar*,
use conditional expressions to return only the necessary collections to the main context, returning na otherwise. The logic
in the main context can then address the na gaps in the series and perform its desired actions on the reduced collections.

For example, if we want to calculate the average BOP across each month instead of using a user-input length, we can return
the array from the requested context only when there is a change to a new month, returning na otherwise. We then maintain
the previous month's values in the main context to keep a valid array for all intra-month bars:

//@version=6
indicator("Monthly BOP array", "Memory limit demo")

// Returns the calculated `bop`, and a copy of the `dataArray` on a month's first trading day only, or `na` ot.
dataFunction() =>
    //@variable Persistent array containing the "balance of power" (BOP) values for all higher timeframe bars.
    var array<float> dataArray = array.new_float(0)

    // When a new month starts, return monthly data array to calculate average BOP for completed month.
    //@variable Array is `na` except on first trading day of each month, when it contains completed month's BOP
    array<float> returnArray = na
    //@variable Is `true` on the first bar of each month, `false` otherwise.
    bool isNewMonth = timeframe.change("1M")
    if isNewMonth
        returnArray := dataArray
    //Clear persistent array to start storing new month's data.
    if isNewMonth[1]
        dataArray.clear()

    //@variable The "balance of power" percentage calculated for the current higher timeframe bar.
    float bop = (close - open) / (high - low) * 100
    dataArray.push(bop)

    //Return the calculated result and the `returnArray`.
    [bop, returnArray]

```



Figure 486: image

```
// Request BOP data from the higher timeframe. (Returns calculated BOP and array of BOP values if new month starts)
[reqValue, reqData] = request.security(syminfo.tickerid, "1D", dataFunction())

// Calculate the average BOP for the most-recent completed month.
//@variable Persistent array that holds the BOP values for the most-recent completed month.
var array<float> completedMonthBOPs = array.new_float(0)
// If new month starts (i.e., `reqData` is not returned as `na`), then `completedMonthBOPs` is updated with new values.
// Otherwise, it persists the last valid values for the rest of the month to adjust for `na` gaps.
completedMonthBOPs := na(reqData) ? completedMonthBOPs : reqData
//@variable The average BOP for the most-recent completed month.
float avgBOP = completedMonthBOPs.avg()

// Plot the BOP value and average line.
color plotColor = reqValue >= 0 ? color.aqua : color.orange
plot(reqValue, "BOP", plotColor, style = plot.style_columns)
plot(avgBOP, "Avg", color.purple, linewidth = 3)
```

Other possible error sources and their fixes

There are a few other ways to optimize scripts to consume less memory.

Minimize `request.*()` calls The `request.*()` function calls can be computationally expensive, because they retrieve data from other contexts, which can often require significant resource usage. Excessive or inefficient requests can easily cause scripts to reach the memory limit.

This memory consumption is especially substantial for scripts requesting data from *lower timeframes*, where the request function returns an array of multiple lower timeframe bars for *each* main context bar. For example, requesting “1” data on a “1D” chart returns hundreds of “1” bars for each “1D” bar that executes the request. In the process, the script must allocate memory to store all the requested data arrays so that it can access them later in the main context, which quickly increases the memory consumption.

Programmers can reduce the number of requested expressions by:

- Removing unnecessary `request.*()` function calls.
- Changing the requested timeframe to a higher resolution.
- Condensing multiple requests to the *same* context into a single `request.*()` call.
- Adjusting the `request.*()` function’s `calc_bars_count` parameter to restrict the historical data points in the requested context.

See this section in the User Manual for more information on optimizing `request.*()` calls.

Refrain from using `max_bars_back` unless necessary The `max_bars_back` parameter of an indicator or strategy sets the size of the *history buffer* for all series variables in a script. The history buffer determines the number of historical references *stored in memory* for the script’s built-in and user-defined variables.

By default, the Pine Script™ runtime automatically allocates an appropriate buffer for each variable. Therefore, the `max_bars_back` parameter and function are only necessary when Pine cannot determine the referencing length of a series.

If you encounter this referencing length error, ensure that you set the `max_bars_back` value appropriately to your script’s needs. Setting a value that’s too large can lead to excessive memory consumption, as it stores unnecessary historical data that the script ultimately doesn’t use. Read up on how to optimize using `max_bars_back` in our Help Center.

Minimize historical buffer calculations The Pine Script™ runtime automatically creates historical buffers for all variables and function calls in a script. It determines the size of a buffer based on the *historical references* needed in the code (the references made using the `[]` history-referencing operator).

As the script runs across the dataset, referencing distant points in bar history can cause the script to restart its execution on previous bars to adjust its historical buffer size (see this User Manual article to learn more). Larger buffers in turn lead to an increase in memory consumption and can result in a runtime error. Ensure that scripts are referencing *necessary* historical values only, and avoid referencing very distant points in history when possible.

You can use the `indicator()` function’s `calc_bars_count` parameter or the `max_bars_back()` function to *manually restrict* the historical data capacity on a script-wide or variable-specific scale. However, be aware that these methods can also cause memory consumption issues of their own if used improperly.

Reduce drawing updates for tables Tables only display their *last state* on a chart. Any updates to a table on historical bars are redundant, because they are not visible. To use the least memory, draw the table *once*, and fill it on the last bar.

Use the `var` keyword to declare table objects once. Enclose all other setter function calls in a conditional `ifbarstate.islast` block for better performance. For more about tables, see this User Manual article.

Do not update drawings on historical bars Similar to tables, any updates to drawing objects such as lines and labels that are made on historical bars are never seen by the user. The user only sees updates on *realtime* bars.

Eliminate updates to historical drawings during historical bars wherever possible. For more information, see this User Manual section.

Minimize total drawings stored for a chart Drawing objects such as lines and labels can consume a lot of memory, especially if a script *recreates* drawings unnecessarily.

For example, if a script draws a line from point `x1` to `x2`, then needs to update the line's endpoint (`x2`), it's more computationally expensive to delete the existing line and redraw a new line from `x1` to `x3`. Instead, using the *setter* function `line.set_x2()` to update the existing line's endpoint is more efficient.

Look for ways to optimize drawing objects in a script:

- Reduce the number of redrawn objects by initializing drawing object *identifiers* and using their setter functions to modify properties.
- Remove unnecessary chart drawings using the `delete()` functions (e.g., `line.delete()` and `label.delete()`).
- Reduce an indicator's maximum drawings limit using the `max_lines_count` or `max_labels_count` parameters.

Filter dates in strategies The total number of trades or orders in a strategy can impact the memory consumption of a script. For large datasets, reduce the number of unnecessary historical orders stored in memory by limiting the *starting point* of your strategy.

You can filter the strategy's date by adding a conditional expression that compares the bar time to a specified timestamp to only place entry/exit orders beyond a certain date.

See an example of date filtering in strategies here.

[Previous

FAQ](#faq)[Next

Release notes](#release-notes) User Manual/Release notes

Release notes

This page contains release notes describing notable changes to the Pine Script™ experience.

2025

March 2025

The `for` loop structure has updated boundary-checking behavior. Previously, any `for` statement established the loop counter's end boundary (`to_num`) *before* starting the first iteration, and the final possible counter value *could not change* during the loop's execution. Changing the result of an expression used as a `for` loop's `to_num` argument inside the local scope *did not* affect the loop's iteration range.

Now, a `for` loop evaluates the `to_num` boundary *dynamically*, before *every iteration*. With this update, the loop statement can modify its stopping condition after any change to the `to_num` argument's result across iterations.

To learn more about this new behavior, refer to the `for` loops section of the Loops page and the Dynamic `for` loop boundaries section of the v6 migration guide.

February 2025

We've removed the scope count limit. Previously, any script's total number of scopes, including the global scope and all local scopes from user-defined functions and methods, loops, conditional structures, user-defined types, and enums, was limited to 550. Now, scripts can contain an indefinite number of local scopes from these structures.

We've introduced two new built-in variables, **bid** and **ask**, providing access to real-time market prices:

- **bid** - represents the highest price an active buyer is willing to pay for the instrument at its current value.
- **ask** - represents the lowest price an active seller will accept for the instrument at its current value.

These variables are only available on the "1T" timeframe. On other timeframes, their values are **na**.

2024

December 2024

The `strategy.exit()` function has updated calculation behaviors. Previously, calls to this command with arguments for the absolute and relative parameters defining a price level for the same exit order always prioritized the *absolute* parameter and *ignored* the relative one. For example, a call with specified **limit** and **profit** values always ignored the **profit** value. Now, the command evaluates *both* related parameters and uses the level that the market price is expected to *activate first*. See this section of the v6 migration guide for more information.

November 2024

Introducing Pine Script™ v6 Pine Script™ has graduated to v6! Starting today, future Pine updates will apply exclusively to this version. Therefore, we recommend converting existing v5 scripts to access new features as we roll them out. See our migration guide to understand the changes to existing Pine behaviors and learn how to convert scripts to v6.

Several new features and behaviors come with this version's release:

- Scripts can now call `request.*()` functions with “*series string*” arguments for the parameters that define the requested context, meaning a single `request.*()` call can change its requested data feed on *any* historical bar. Additionally, it is now possible to call `request.*()` functions inside the local scopes of loops, conditional structures, and exported library functions. See the Dynamic requests section of the Other timeframes and data page to learn more.
- Values of the “bool” type are now strictly **true** or **false**. They are never **na** in v6. Additionally, the **or** and **and** operators now feature *short-circuit* (“*lazy*”) evaluation. If the first expression of an **or** operation is **true**, or the first expression of an **and** operation is **false**, the script does **not** evaluate the second expression because it is not necessary to determine the result. These improvements help boost the runtime efficiency of scripts that rely on “bool” values and conditional expressions.
- The **size** property of labels and the **text_size** property of boxes and tables now support “int” values in addition to the **size.*** constants. These “int” values represent sizes in *typographic points*, offering a more granular and wide range of text size possibilities.
- The new **text_formatting** parameter of the `label.new()`, `box.new()`, and `table.cell()` functions determines whether the object's displayed text is **bold**, *italicized*, or **both**. It accepts one of these three new **text.*** constants: `text.format_bold`, `text.format_italic`, `text.format_none`. To modify a drawing object's **text_formatting** property, use the corresponding `*set_text_formatting()` functions.
- Strategies no longer stop calculating and raise an error when they reach the 9000 trade limit while not using Deep Backtesting mode. Instead, they *trim* the oldest orders to make space for new ones. The trimmed orders are *not* visible in the Strategy Tester, but that does not change the strategy's simulation. To retrieve the trade index of the earliest *non-trimmed* order, use the `strategy.closedtrades.first_index` variable.
- The `array.get()`, `array.set()`, `array.insert()`, and `array.remove()` functions now support *negativeindex* arguments to reference elements starting from the *end* of an array. For instance, the call `array.get(myArray, -2)` retrieves the second to last element in `myArray`, which is equivalent to `array.get(myArray, array.size(myArray) - 2)`.
- The new `syminfo.mincontract` variable holds a value representing the smallest number of contracts/shares/lots/units required to trade the current symbol, as set by the exchange.
- Two new variables, `syminfo.main_tickerid` and `timeframe.main_period`, reference the ticker ID and timeframe from the script's *main context*, even if the script uses them in the **expression** argument of a `request.*()` call. Here, “main context” refers to the current chart's symbol and timeframe, unless the script is an `indicator()` that includes **symbol** or **timeframe** arguments in its declaration statement.

October 2024

We've added an optional `behind_chart` parameter to the `indicator()` and `strategy()` functions. This parameter specifies where plots and drawings appear relative to the main chart display when the `overlay` parameter is `true`. If `behind_chart` is `true`, the script's visuals appear behind the chart display. If `false`, they appear in front of the chart display. The default is `true`.

August 2024

The `ticker.new()` and `ticker.modify()` functions feature two new parameters: `settlement_as_close` and `backadjustment`. Users can specify whether these parameters are on, off, or set to inherit the symbol's default settings. These settings only affect the data from futures symbols with these options available on their charts. They have no effect on other symbols.

- The `backadjustment` parameter specifies whether past contract data on continuous futures symbols is back-adjusted. Its possible values are: `backadjustment.on`, `backadjustment.off`, or `backadjustment.inherit`.
- The `settlement_as_close` parameter specifies whether a futures symbol's close value represents the actual closing price or the settlement price on "1D" and higher timeframes. Its possible values are: `settlement_as_close.on`, `settlement_as_close.off`, or `settlement_as_close.inherit`.

The Sharpe and Sortino ratios in the Strategy Tester module have updated calculations. Previously, the ratios used strategy returns over monthly periods if the trading range was three or more months and daily periods if the range was three or more days but less than three months. Both ratios now always use monthly periods for consistency.

June 2024

We've added a new parameter to the `box.new()`, `label.new()`, `line.new()`, `polyline.new()`, and `table.new()` functions:

- `force_overlay` - If true, the drawing will display on the main chart pane, even when the script occupies a separate pane. Optional. The default is false.

Pine Script™ Enums Enums, also known as *enumerations*, *enumerated types*, or enum types, are unique data types with all possible values declared by the programmer. They can help programmers maintain more strict control over the values allowed by variables, conditional expressions, and collections, and they enable convenient dropdown input creation with the new `input.enum()` function. See our User Manual's Enums page to learn more about these new types and how to use them.

May 2024

We've added an optional `calc_bars_count` parameter to the `indicator()`, `strategy()`, `request.security()`, `request.security_lower_tf()`, and `request.seed()` functions that allows users to limit the number of recent historical bars a script or data request can execute across. When a script's `indicator()` or `strategy()` declaration statement includes a `calc_bars_count` argument, its "Settings/Inputs" tab will include a "Calculated bars" input in the "Calculation" section. The default value in all these functions is 0, which signifies that the script or request executes across all the available data.

The `strategy.*` namespace features several new built-in variables:

- `strategy.avg_trade` - Returns the average amount of money gained or lost per trade. Calculated as the sum of all profits and losses divided by the number of closed trades.
- `strategy.avg_trade_percent` - Returns the average percentage gain or loss per trade. Calculated as the sum of all profit and loss percentages divided by the number of closed trades.
- `strategy.avg_winning_trade` - Returns the average amount of money gained per winning trade. Calculated as the sum of profits divided by the number of winning trades.
- `strategy.avg_winning_trade_percent` - Returns the average percentage gain per winning trade. Calculated as the sum of profit percentages divided by the number of winning trades.
- `strategy.avg_losing_trade` - Returns the average amount of money lost per losing trade. Calculated as the sum of losses divided by the number of losing trades.
- `strategy.avg_losing_trade_percent` - Returns the average percentage loss per losing trade. Calculated as the sum of loss percentages divided by the number of losing trades.

Pine Profiler Our new Pine Profiler is a powerful utility that analyzes the executions of all significant code in a script and displays helpful performance information next to the code lines *inside* the Pine Editor. The Profiler's information provides insight into a script's runtime, the distribution of runtime across significant code regions, and the number of times each code region executes. With these insights, programmers can effectively pinpoint performance *bottlenecks* and ensure they focus on optimizing their code where it truly matters when they need to improve execution times.

See the new Profiling and optimization page to learn more about the Profiler, how it works, and how to use it to analyze a script's performance and identify optimization opportunities.

Pine Editor improvements When opening the detached Pine Editor from a tab with a chart, it now links directly to that tab, as indicated by the “Linked” status and green icon in the bottom-right corner. While linked, the “Add to chart”, “Update on chart”, and “Apply to entire layout” buttons affect the charts on the main tab.

The detached Pine Editor now includes the Pine console.

April 2024

We've added a new parameter to the `plot()`, `plotchar()`, `plotcandle()`, `plotbar()`, `plotarrow()`, `plotshape()`, and `bgcolor()` functions:

- **force_overlay** - If true, the output will display on the main chart pane, even when the script occupies a separate pane.

March 2024

The `syminfo.*` namespace features a new built-in variable:

- **syminfo.expiration_date** - On non-continuous futures symbols, returns a UNIX timestamp representing the start of the last day of the current contract.

The `time()` and `time_close()` functions have a new parameter:

- **bars_back** - If specified, the function will calculate the timestamp from the bar N bars back relative to the current bar on its timeframe. It can also calculate the expected time of a future bar up to 500 bars away if the argument is a negative value. Optional. The default is 0.

February 2024

We've added two new functions for working with strings:

- **str.repeat()** - Constructs a new string containing the source string repeated a specified number of times with a separator injected between each repeated instance.
- **str.trim()** - Constructs a new string with all consecutive whitespaces and other control characters removed from the left and right of the source string.

The `request.financial()` function now accepts “D” as a **period** argument, allowing scripts to request available daily financial data.

For example:

```
//@version=5
indicator("Daily financial data demo")

//@variable The daily Premium/Discount to Net Asset Value for "AMEX:SPY"
float f1 = request.financial("AMEX:SPY", "NAV", "D")
plot(f1)
```

The `strategy.*` namespace features a new variable for monitoring available capital in a strategy's simulation:

- **strategy.opentrades.capital_held** - Returns the capital amount currently held by open trades.

January 2024

The `syminfo.*` namespace features new built-in variables:

Syminfo:

- **syminfo.employees** - The number of employees the company has.
- **syminfo.shareholders** - The number of shareholders the company has.
- **syminfo.shares_outstanding_float** - The total number of shares outstanding a company has available, excluding any of its restricted shares.
- **syminfo.shares_outstanding_total** - The total number of shares outstanding a company has available, including restricted shares held by insiders, major shareholders, and employees.

Target price:

- `syminfo.target_price_average` - The average of the last yearly price targets for the symbol predicted by analysts.
- `syminfo.target_price_date` - The starting date of the last price target prediction for the current symbol.
- `syminfo.target_price_estimates` - The latest total number of price target predictions for the current symbol.
- `syminfo.target_price_high` - The last highest yearly price target for the symbol predicted by analysts.
- `syminfo.target_price_low` - The last lowest yearly price target for the symbol predicted by analysts.
- `syminfo.target_price_median` - The median of the last yearly price targets for the symbol predicted by analysts.

Recommendations:

- `syminfo.recommendations_buy` - The number of analysts who gave the current symbol a “Buy” rating.
- `syminfo.recommendations_buy_strong` - The number of analysts who gave the current symbol a “Strong Buy” rating.
- `syminfo.recommendations_date` - The starting date of the last set of recommendations for the current symbol.
- `syminfo.recommendations_hold` - The number of analysts who gave the current symbol a “Hold” rating.
- `syminfo.recommendations_total` - The total number of recommendations for the current symbol.
- `ssyminfo.recommendations_sell` - The number of analysts who gave the current symbol a “Sell” rating.
- `syminfo.recommendations_sell_strong` - The number of analysts who gave the current symbol a “Strong Sell” rating.

2023

December 2023

We’ve added `format` and `precision` parameters to all `plot*()` functions, allowing indicators and strategies to selectively apply formatting and decimal precision settings to plotted results in the chart pane’s y-axis, the script’s status line, and the Data Window. The arguments passed to these parameters supersede the values in the `indicator()` and `strategy()` functions. Both are optional. The defaults for these parameters are the same as the values specified in the script’s declaration statement.

For example:

```
//@version=5
indicator("My script", format = format.percent, precision = 4)

plot(close, format = format.price)           // Price format with 4-digit precision.
plot(100 * bar_index / close, precision = 2) // Percent format with 2-digit precision.
```

November 2023

We’ve added the following variables and functions to the `strategy.*` namespace:

- `strategy.grossloss_percent` - The total gross loss value of all completed losing trades, expressed as a percentage of the initial capital.
- `strategy.grossprofit_percent` - The total gross profit value of all completed winning trades, expressed as a percentage of the initial capital.
- `strategy.max_runup_percent` - The maximum rise from a trough in the equity curve, expressed as a percentage of the trough value.
- `strategy.max_drawdown_percent` - The maximum drop from a peak in the equity curve, expressed as a percentage of the peak value.
- `strategy.netprofit_percent` - The total value of all completed trades, expressed as a percentage of the initial capital.
- `strategy.openprofit_percent` - The current unrealized profit or loss for all open positions, expressed as a percentage of realized equity.
- `strategy.closedtrades.max_drawdown_percent()` - Returns the maximum drawdown of the closed trade, i.e., the maximum possible loss during the trade, expressed as a percentage.
- `strategy.closedtrades.max_runup_percent()` - Returns the maximum run-up of the closed trade, i.e., the maximum possible profit during the trade, expressed as a percentage.
- `strategy.closedtrades.profit_percent()` - Returns the profit/loss value of the closed trade, expressed as a percentage. Losses are expressed as negative values.
- `strategy.opentrades.max_drawdown_percent()` - Returns the maximum drawdown of the open trade, i.e., the maximum possible loss during the trade, expressed as a percentage.
- `strategy.opentrades.max_runup_percent()` - Returns the maximum run-up of the open trade, i.e., the maximum possible profit during the trade, expressed as a percentage.
- `strategy.opentrades.profit_percent()` - Returns the profit/loss of the open trade, expressed as a percentage. Losses are expressed as negative values.

October 2023

Pine Script™ Polylines Polylines are drawings that sequentially connect the coordinates from an array of up to 10,000 chart points using straight or *curved* line segments, allowing scripts to draw custom formations that are difficult or impossible to achieve using line or box objects. To learn more about this new drawing type, see the Polylines section of our User Manual’s page on Lines and boxes.

September 2023

New functions were added:

- `strategy.default_entry_qty()` - Calculates the default quantity, in units, of an entry order from `strategy.entry()` or `strategy.order()` if it were to fill at the specified `fill_price` value.
- `chart.point.new()` - Creates a new `chart.point` object with the specified `time`, `index`, and `price`.
- `request.seed()` - Requests data from a user-maintained GitHub repository and returns it as a series. An in-depth tutorial on how to add new data can be found [here](#).
- `ticker.inherit()` - Constructs a ticker ID for the specified `symbol` with additional parameters inherited from the ticker ID passed into the function call, allowing the script to request a symbol’s data using the same modifiers that the `from_tickerid` has, including extended session, dividend adjustment, currency conversion, non-standard chart types, back-adjustment, settlement-as-close, etc.
- `timeframe.from_seconds()` - Converts a specified number of `seconds` into a valid timeframe string based on our timeframe specification format.

The `dividends.*` namespace now includes variables for retrieving future dividend information:

- `dividends.future_amount` - Returns the payment amount of the upcoming dividend in the currency of the current instrument, or `na` if this data isn’t available.
- `dividends.future_ex_date` - Returns the Ex-dividend date (Ex-date) of the current instrument’s next dividend payment, or `na` if this data isn’t available.
- `dividends.future_pay_date` - Returns the Payment date (Pay date) of the current instrument’s next dividend payment, or `na` if this data isn’t available.

The `request.security_lower_tf()` function has a new parameter:

- `ignore_invalid_timeframe` - Determines how the function behaves when the chart’s timeframe is smaller than the `timeframe` value in the function call. If `false`, the function will raise a runtime error and halt the script’s execution. If `true`, the function will return `na` without raising an error.

Users can now explicitly declare variables with the `const`, `simple`, and `series` type qualifiers, allowing more precise control over the types of variables in their scripts. For example:

```
//@version=5
indicator("My script")

//@variable A constant `string` used as the `title` in the `plot()` function.
const string plotTitle = "My plot"
//@variable An `int` variable whose value is consistent after the first chart bar.
simple int a = 10
//@variable An `int` variable whose value can change on every bar.
series int b = bar_index

plot(b % a, title = plotTitle)
```

August 2023

Added the following alert placeholders:

- `{{syminfo.currency}}` - Returns the currency code of the current symbol (“EUR”, “USD”, etc.).
- `{{syminfo.basecurrency}}` - Returns the base currency code of the current symbol if the symbol refers to a currency pair. Otherwise, it returns `na`. For example, it returns “EUR” when the symbol is “EURUSD”.

Pine Script™ Maps Maps are collections that hold elements in the form of *key-value pairs*. They associate unique keys of a *fundamental type* with values of a *built-in* or user-defined type. Unlike arrays, these collections are *unordered* and do not utilize an internal lookup index. Instead, scripts access the values of maps by referencing the *keys* from the key-value pairs put into them. For more information on these new collections, see our User Manual’s page on Maps.

July 2023

Fixed an issue that caused strategies to occasionally calculate the sizes of limit orders incorrectly due to improper tick rounding of the `limit` price.

Added a new built-in variable to the `strategy.*` namespace:

- `strategy.margin_liquidation_price` - When a strategy uses margin, returns the price value after which a margin call will occur.

June 2023

New `syminfo.*` built-in variables were added:

- `syminfo.sector` - Returns the sector of the symbol.
- `syminfo.industry` - Returns the industry of the symbol.
- `syminfo.country` - Returns the two-letter code of the country where the symbol is traded.

A new display parameter for all `input.*()` functions was added. It provides you with more control over the display of input values next to a script's name. Four arguments can be used: `display.status_line`, `display.data_window`, `display.all`, and `display.none`. Combinations of these arguments using plus or minus signs are allowed, and regardless of the argument used, input values will always continue to appear in the **Inputs** tab of the script's settings.

May 2023

New parameter added to the `strategy.entry()`, `strategy.order()`, `strategy.close()`, `strategy.close_all()`, and `strategy.exit()` functions:

- `disable_alert` - Disables order fill alerts for any orders placed by the function.

Our “Indicator on indicator” feature, which allows a script to pass another indicator's plot as a source value via the `input.source()` function, now supports multiple external inputs. Scripts can use a multitude of external inputs originating from up to 10 different indicators.

We've added the following array functions:

- `array.every()` - Returns `true` if all elements of the `id` array are `true`, `false` otherwise.
- `array.some()` - Returns `true` if at least one element of the `id` array is `true`, `false` otherwise. These functions also work with arrays of int and float types, in which case zero values are considered `false`, and all others `true`.

April 2023

Fixed an issue with trailing stops in `strategy.exit()` being filled on high/low prices rather than on intrabar prices.

Fixed behavior of `array.mode()`, `matrix.mode()` and `ta.mode()`. Now these functions will return the smallest value when the data has no most frequent value.

March 2023

It is now possible to use seconds-based timeframe strings for the `timeframe` parameter in `request.security()` and `request.security_lower_tf()`.

A new function was added:

- `request.currency_rate()` - provides a daily rate to convert a value expressed in the `from` currency to another in the `to` currency.

February 2023

Pine Script™ Methods Pine Script™ methods are specialized functions associated with specific instances of built-in or user-defined types. They offer a more convenient syntax than standard functions, as users can access methods in the same way as object fields using the handy dot notation syntax. Pine Script™ includes built-in methods for array, matrix, line, linefill, label, box, and table types and facilitates user-defined methods with the new `method` keyword. For more details on this new feature, see our User Manual's page on methods.

January 2023

New array functions were added:

- `array.first()` - Returns the array's first element.
- `array.last()` - Returns the array's last element.

2022

December 2022

Pine Objects Pine objects are instantiations of the new user-defined composite types (UDTs) declared using the `type` keyword. Experienced programmers can think of UDTs as method-less classes. They allow users to create custom types that organize different values under one logical entity. A detailed rundown of the new functionality can be found in our User Manual's page on objects.

A new function was added:

- `ticker.standard()` - Creates a ticker to request data from a standard chart that is unaffected by modifiers like extended session, dividend adjustment, currency conversion, and the calculations of non-standard chart types: Heikin Ashi, Renko, etc.

New `strategy.*` functions were added:

- `strategy.opentrades.entry_comment()` - The function returns the comment message of the open trade's entry.
- `strategy.closedtrades.entry_comment()` - The function returns the comment message of the closed trade's entry.
- `strategy.closedtrades.exit_comment()` - The function returns the comment message of the closed trade's exit.

November 2022

Fixed behaviour of `math.round_to_mintick()` function. For 'na' values it returns 'na'.

October 2022

Pine Script™ now has a new, more powerful and better-integrated editor. Read our blog to find out everything to know about all the new features and upgrades.

New overload for the `fill()` function was added. Now it can create vertical gradients. More info about it in the blog post.

A new function was added:

- `str.format_time()` - Converts a timestamp to a formatted string using the specified format and time zone.

September 2022

The `text_font_family` parameter now allows the selection of a monospace font in `label.new()`, `box.new()` and `table.cell()` function calls, which makes it easier to align text vertically. Its arguments can be:

- `font.family_default` - Specifies the default font.
- `font.family_monospace` - Specifies a monospace font.

The accompanying setter functions are:

- `label.set_text_font_family()` - The function sets the font family of the text inside the label.
- `box.set_text_font_family()` - The function sets the font family of the text inside the box.
- `table.cell_set_text_font_family()` - The function sets the font family of the text inside the cell.

August 2022

A new label style `label.style_text_outline` was added.

A new parameter for the `ta.pivot_point_levels()` function was added:

- **developing** - If `false`, the values are those calculated the last time the anchor condition was true. They remain constant until the anchor condition becomes true again. If `true`, the pivots are developing, i.e., they constantly recalculate on the data developing between the point of the last anchor (or bar zero if the anchor condition was never true) and the current bar. Cannot be `true` when `type` is set to "Woodie".

A new parameter for the `box.new()` function was added:

- **text_wrap** - It defines whether the text is presented in a single line, extending past the width of the box if necessary, or wrapped so every line is no wider than the box itself.

This parameter supports two arguments:

- **text.wrap_none** - Disabled wrapping mode for **box.new** and **box.set_text_wrap** functions.
- **text.wrap_auto** - Automatic wrapping mode for **box.new** and **box.set_text_wrap** functions.

New built-in functions were added:

- **ta.min()** - Returns the all-time low value of **source** from the beginning of the chart up to the current bar.
- **ta.max()** - Returns the all-time high value of **source** from the beginning of the chart up to the current bar.

A new annotation **//@strategy_alert_message** was added. If the annotation is added to the strategy, the text written after it will be automatically set as the default alert message in the [Create Alert] window.

```
//@version=5
// @strategy_alert_message My Default Alert Message
strategy("My Strategy")
plot(close)
```

July 2022

It is now possible to fine-tune where a script's plot values are displayed through the introduction of new arguments for the **display** parameter of the **plot()**, **plotchar()**, **plotshape()**, **plotarrow()**, **plotcandle()**, and **plotbar()** functions.

Four new arguments were added, complementing the previously available **display.all** and **display.none**:

- **display.data_window** displays the plot values in the Data Window, one of the items available from the chart's right sidebar.
- **display.pane** displays the plot in the pane where the script resides, as defined in with the **overlay** parameter of the script's **indicator()**, **strategy()**, or **library()** declaration statement.
- **display.price_scale** controls the display of the plot's label and price in the price scale, if the chart's settings allow them.
- **display.status_line** displays the plot values in the script's status line, next to the script's name on the chart, if the chart's settings allow them.

The **display** parameter supports the addition and subtraction of its arguments:

- **display.all** - **display.status_line** will display the plot's information everywhere except in the script's status line.
- **display.price_scale** + **display.status_line** will display the plot in the price scale and status line only.

June 2022

The behavior of the argument used with the **qty_percent** parameter of **strategy.exit()** has changed. Previously, the percentages used on successive exit orders of the same position were calculated from the remaining position at any given time. Instead, the percentages now always apply to the initial position size. When executing the following strategy, for example:

```
//@version=5
strategy("strategy.exit() example", overlay = true)
strategy.entry("Long", strategy.long, qty = 100)
strategy.exit("Exit Long1", "Long", trail_points = 50, trail_offset = 0, qty_percent = 20)
strategy.exit("Exit Long2", "Long", trail_points = 100, trail_offset = 0, qty_percent = 20)
```

20% of the initial position will be closed on each **strategy.exit()** call. Before, the first call would exit 20% of the initial position, and the second would exit 20% of the remaining 80% of the position, so only 16% of the initial position.

Two new parameters for the built-in **ta.vwap()** function were added:

- **anchor** - Specifies the condition that triggers the reset of VWAP calculations. When **true**, calculations reset; when **false**, calculations proceed using the values accumulated since the previous reset.
- **stdev_mult** - If specified, the **ta.vwap()** calculates the standard deviation bands based on the main VWAP series and returns a [**vwap**, **upper_band**, **lower_band**] tuple.

New overloaded versions of the **strategy.close()** and **strategy.close_all()** functions with the **immediately** parameter. When **immediately** is set to **true**, the closing order will be executed on the tick where it has been placed, ignoring the strategy parameters that restrict the order execution to the open of the next bar.

New built-in functions were added:

- `timeframe.change()` - Returns **true** on the first bar of a new **timeframe**, **false** otherwise.
- `ta.pivot_point_levels()` - Returns a float array with numerical values representing 11 pivot point levels: [P, R1, S1, R2, S2, R3, S3, R4, S4, R5, S5]. Levels absent from the specified **type** return na values.

New built-in variables were added:

- `session.isfirstbar` - returns **true** if the current bar is the first bar of the day's session, **false** otherwise.
- `session.islastbar` - returns **true** if the current bar is the last bar of the day's session, **false** otherwise.
- `session.isfirstbar_regular` - returns **true** on the first regular session bar of the day, **false** otherwise.
- `session.islastbar_regular` - returns **true** on the last regular session bar of the day, **false** otherwise.
- `chart.left_visible_bar_time` - returns the **time** of the leftmost bar currently visible on the chart.
- `chart.right_visible_bar_time` - returns the **time** of the rightmost bar currently visible on the chart.

May 2022

Matrix support has been added to the `request.security()` function.

The historical states of arrays and matrices can now be referenced with the `[]` operator. In the example below, we reference the historic state of a matrix 10 bars ago:

```
//@version=5
indicator("matrix.new<float> example")
m = matrix.new<float>(1, 1, close)
float x = na
if bar_index > 10
    x := matrix.get(m[10], 0, 0)
plot(x)
plot(close)
```

The `ta.change()` function now can take values of `int` and `bool` types as its **source** parameter and return the difference in the respective type.

New built-in variables were added:

- `chart.bg_color` - Returns the color of the chart's background from the "Chart settings/Appearance/Background" field.
- `chart.fg_color` - Returns a color providing optimal contrast with `chart.bg_color`.
- `chart.is_standard` - Returns **true** if the chart type is bars, candles, hollow candles, line, area or baseline, **false** otherwise.
- `currency.USDT` - A constant for the Tether currency code.

New functions were added:

- `syminfo.prefix()` - returns the exchange prefix of the **symbol** passed to it, e.g. "NASDAQ" for "NASDAQ:AAPL".
- `syminfo.ticker()` - returns the ticker of the **symbol** passed to it without the exchange prefix, e.g. "AAPL" for "NASDAQ:AAPL".
- `request.security_lower_tf()` - requests data from a lower timeframe than the chart's.

Added `use_bar_magnifier` parameter for the `strategy()` function. When **true**, the Broker Emulator uses lower timeframe data during history backtesting to achieve more realistic results.

Fixed behaviour of `strategy.exit()` function when stop loss triggered at prices outside the bars price range.

Added new `comment` and `alert` message parameters for the `strategy.exit()` function:

- `comment_profit` - additional notes on the order if the exit was triggered by crossing **profit** or **limit** specifically.
- `comment_loss` - additional notes on the order if the exit was triggered by crossing **stop** or **loss** specifically.
- `comment_trailing` - additional notes on the order if the exit was triggered by crossing **trail_offset** specifically.
- `alert_profit` - text that will replace the '`{{strategy.order.alert_message}}`' placeholder if the exit was triggered by crossing **profit** or **limit** specifically.
- `alert_loss` - text that will replace the '`{{strategy.order.alert_message}}`' placeholder if the exit was triggered by crossing **stop** or **loss** specifically.
- `alert_trailing` - text that will replace the '`{{strategy.order.alert_message}}`' placeholder if the exit was triggered by crossing **trail_offset** specifically.

April 2022

Added the `display` parameter to the following functions: `barcolor`, `bgcolor`, `fill`, `hline`.

A new function was added:

- `request.economic()` - Economic data includes information such as the state of a country's economy or of a particular industry.

New built-in variables were added:

- `strategy.max_runup` - Returns the maximum equity run-up value for the whole trading interval.
- `syminfo.volumetype` - Returns the volume type of the current symbol.
- `chart.is_heikinashi` - Returns true if the chart type is Heikin Ashi, false otherwise.
- `chart.is_kagi` - Returns true if the chart type is Kagi, false otherwise.
- `chart.is_linebreak` - Returns true if the chart type is Line break, false otherwise.
- `chart.is_pnf` - Returns true if the chart type is Point & figure, false otherwise.
- `chart.is_range` - Returns true if the chart type is Range, false otherwise.
- `chart.is_renko` - Returns true if the chart type is Renko, false otherwise.

New matrix functions were added:

- `matrix.new()` - Creates a new matrix object. A matrix is a two-dimensional data structure containing rows and columns. All elements in the matrix must be of the type specified in the type template ("").
- `matrix.row()` - Creates a one-dimensional array from the elements of a matrix row.
- `matrix.col()` - Creates a one-dimensional array from the elements of a matrix column.
- `matrix.get()` - Returns the element with the specified index of the matrix.
- `matrix.set()` - Assigns **value** to the element at the **column** and **row** index of the matrix.
- `matrix.rows()` - Returns the number of rows in the matrix.
- `matrix.columns()` - Returns the number of columns in the matrix.
- `matrix.elements_count()` - Returns the total number of matrix elements.
- `matrix.add_row()` - Adds a row to the matrix. The row can consist of **na** values, or an array can be used to provide values.
- `matrix.add_col()` - Adds a column to the matrix. The column can consist of **na** values, or an array can be used to provide values.
- `matrix.remove_row()` - Removes the row of the matrix and returns an array containing the removed row's values.
- `matrix.remove_col()` - Removes the column of the matrix and returns an array containing the removed column's values.
- `matrix.swap_rows()` - Swaps the rows in the matrix.
- `matrix.swap_columns()` - Swaps the columns in the matrix.
- `matrix.fill()` - Fills a rectangular area of the matrix defined by the indices **from_column** to **to_column**.
- `matrix.copy()` - Creates a new matrix which is a copy of the original.
- `matrix.submatrix()` - Extracts a submatrix within the specified indices.
- `matrix.reverse()` - Reverses the order of rows and columns in the matrix. The first row and first column become the last, and the last become the first.
- `matrix.reshape()` - Rebuilds the matrix to **rows** x **cols** dimensions.
- `matrix.concat()` - Append one matrix to another.
- `matrix.sum()` - Returns a new matrix resulting from the sum of two matrices, or of a matrix and a scalar (a numerical value).
- `matrix.diff()` - Returns a new matrix resulting from the subtraction between matrices, or of matrix and a scalar (a numerical value).
- `matrix.mult()` - Returns a new matrix resulting from the product between the matrices, or between a matrix and a scalar (a numerical value), or between a matrix and a vector (an array of values).
- `matrix.sort()` - Rearranges the rows in the **id** matrix following the sorted order of the values in the **column**.
- `matrix.avg()` - Calculates the average of all elements in the matrix.
- `matrix.max()` - Returns the largest value from the matrix elements.
- `matrix.min()` - Returns the smallest value from the matrix elements.
- `matrix.median()` - Calculates the median ("the middle" value) of matrix elements.
- `matrix.mode()` - Calculates the mode of the matrix, which is the most frequently occurring value from the matrix elements. When there are multiple values occurring equally frequently, the function returns the smallest of those values.
- `matrix.pow()` - Calculates the product of the matrix by itself **power** times.
- `matrix.det()` - Returns the determinant of a square matrix.
- `matrix.transpose()` - Creates a new, transposed version of the matrix by interchanging the row and column index of each element.
- `matrix.pinv()` - Returns the pseudoinverse of a matrix.
- `matrix.inv()` - Returns the inverse of a square matrix.

- `matrix.rank()` - Calculates the rank of the matrix.
- `matrix.trace()` - Calculates the trace of a matrix (the sum of the main diagonal's elements).
- `matrix.eigenvalues()` - Returns an array containing the eigenvalues of a square matrix.
- `matrix.eigenvectors()` - Returns a matrix of eigenvectors, in which each column is an eigenvector of the matrix.
- `matrix.kron()` - Returns the Kronecker product for the two matrices.
- `matrix.is_zero()` - Determines if all elements of the matrix are zero.
- `matrix.is_identity()` - Determines if a matrix is an identity matrix (elements with ones on the main diagonal and zeros elsewhere).
- `matrix.is_binary()` - Determines if the matrix is binary (when all elements of the matrix are 0 or 1).
- `matrix.is_symmetric()` - Determines if a square matrix is symmetric (elements are symmetric with respect to the main diagonal).
- `matrix.is_antisymmetric()` - Determines if a matrix is antisymmetric (its transpose equals its negative).
- `matrix.is_diagonal()` - Determines if the matrix is diagonal (all elements outside the main diagonal are zero).
- `matrix.is_antidiagonal()` - Determines if the matrix is anti-diagonal (all elements outside the secondary diagonal are zero).
- `matrix.is_triangular()` - Determines if the matrix is triangular (if all elements above or below the main diagonal are zero).
- `matrix.is_stochastic()` - Determines if the matrix is stochastic.
- `matrix.is_square()` - Determines if the matrix is square (it has the same number of rows and columns).

Added a new parameter for the `strategy()` function:

- **risk_free_rate** - The risk-free rate of return is the annual percentage change in the value of an investment with minimal or zero risk, used to calculate the Sharpe and Sortino ratios.

March 2022

New array functions were added:

- `array.sort_indices()` - returns an array of indices which, when used to index the original array, will access its elements in their sorted order.
- `array.percentrank()` - returns the percentile rank of a value in the array.
- `array.percentile_nearest_rank()` - returns the value for which the specified percentage of array values (percentile) are less than or equal to it, using the nearest-rank method.
- `array.percentile_linear_interpolation()` - returns the value for which the specified percentage of array values (percentile) are less than or equal to it, using linear interpolation.
- `array.abs()` - returns an array containing the absolute value of each element in the original array.
- `array.binary_search()` - returns the index of the value, or -1 if the value is not found.
- `array.binary_search_leftmost()` - returns the index of the value if it is found or the index of the next smallest element to the left of where the value would lie if it was in the array.
- `array.binary_search_rightmost()` - returns the index of the value if it is found or the index of the element to the right of where the value would lie if it was in the array.

Added a new optional `nth` parameter for the `array.min()` and `array.max()` functions.

Added `index` in `for..in` operator. It tracks the current iteration's index.

Table merging and cell tooltips

- It is now possible to merge several cells in a table. A merged cell doesn't have to be a header: you can merge cells in any direction, as long as the resulting cell doesn't affect any already merged cells and doesn't go outside of the table's bounds. Cells can be merged with the new `table.merge_cells()` function.
- Tables now support tooltips, floating labels that appear when you hover over a table's cell. To add a tooltip, pass a string to the `tooltip` argument of the `table.cell()` function or use the new `table.cell_set_tooltip()` function.

February 2022

Added templates and the ability to create arrays via templates. Instead of using one of the `array.new_*`() functions, a template function `array.new()` can be used. In the example below, we use this functionality to create an array filled with float values:

```
//@version=5
indicator("array.new<float> example")
length = 5
```



```

var a = array.new<float>(length, close)
if array.size(a) == length
    array.remove(a, 0)
    array.push(a, close)
plot(array.sum(a) / length, "SMA")

```

New functions were added:

- `timeframe.in_seconds(timeframe)` - converts the timeframe passed to the **timeframe** argument into seconds.
- `input.text_area()` - adds multiline text input area to the Script settings.
- `strategy.closedtrades.entry_id()` - returns the id of the closed trade's entry.
- `strategy.closedtrades.exit_id()` - returns the id of the closed trade's exit.
- `strategy.opentrades.entry_id()` - returns the id of the open trade's entry.

January 2022

Added new functions to clone drawings:

- `line.copy()`
- `label.copy()`
- `box.copy()`

2021

December 2021

Linefills The space between lines drawn in Pine Script™ can now be filled! We've added a new **linefill** drawing type, along with a number of functions dedicated to manipulating it. Linefills are created by passing two lines and a color to the `linefill.new()` function, and their behavior is based on the lines they're tied to: they extend in the same direction as the lines, move when their lines move, and are deleted when one of the two lines is deleted.

New linefill-related functions:

- `array.new_linefill()`
- `linefill()`
- `linefill.delete()`
- `linefill.get_line1()`
- `linefill.get_line2()`
- `linefill.new()`
- `linefill.set_color()`
- `linefill.all()`

New functions for string manipulation Added a number of new functions that provide more ways to process strings, and introduce regular expressions to Pine Script™:

- `str.contains(source, str)` - Determines if the **source** string contains the **str** substring.
- `str.pos(source, str)` - Returns the position of the **str** string in the **source** string.
- `str.substring(source, begin_pos, end_pos)` - Extracts a substring from the **source** string.
- `str.replace(source, target, replacement, occurrence)` - Contrary to the existing `str.replace_all()` function, `str.replace()` allows the selective replacement of a matched substring with a replacement string.
- `str.lower(source)` and `str.upper(source)` - Convert all letters of the **source** string to lower or upper case:
- `str.startswith(source, str)` and `str.endswith(source, str)` - Determine if the **source** string starts or ends with the **str** substring.
- `str.match(source, regex)` - Extracts the substring matching the specified regular expression.

Textboxes Box drawings now supports text. The `box.new()` function has five new parameters for text manipulation: **text**, **text_size**, **text_color**, **text_valign**, and **text_halign**. Additionally, five new functions to set the text properties of existing boxes were added:

- `box.set_text()`
- `box.set_text_color()`
- `box.set_text_size()`
- `box.set_text_valign()`
- `box.set_text_halign()`

New built-in variables Added new built-in variables that return the `bar_index` and `time` values of the last bar in the dataset. Their values are known at the beginning of the script's calculation:

- `last_bar_index` - Bar index of the last chart bar.
- `last_bar_time` - UNIX time of the last chart bar.

New built-in `source` variable:

- `hlcc4` - A shortcut for $(high + low + close + close)/4$. It averages the high and low values with the double-weighted close.

November 2021

for...in Added a new `for...in` operator to iterate over all elements of an array:

```
//@version=5
indicator("My Script")
int[] a1 = array.from(1, 3, 6, 3, 8, 0, -9, 5)

highest(array) =>
    var int highestNum = na
    for item in array
        if na(highestNum) or item > highestNum
            highestNum := item
    highestNum

plot(highest(a1))
```

Function overloads Added function overloads. Several functions in a script can now share the same name, as long one of the following conditions is true:

- Each overload has a different number of parameters:

```
//@version=5
indicator("Function overload")
```

```
// Two parameters
mult(x1, x2) =>
    x1 * x2
```

```
// Three parameters
mult(x1, x2, x3) =>
    x1 * x2 * x3
```

```
plot(mult(7, 4))
plot(mult(7, 4, 2))
```

- When overloads have the same number of parameters, all parameters in each overload must be explicitly typified, and their type combinations must be unique:

```
//@version=5
indicator("Function overload")
```

```
// Accepts both 'int' and 'float' values - any 'int' can be automatically cast to 'float'
mult(float x1, float x2) =>
    x1 * x2
```

```
// Returns a 'bool' value instead of a number
mult(bool x1, bool x2) =>
    x1 and x2 ? true : false
```

```
mult(string x1, string x2) =>
    str.tonumber(x1) * str.tonumber(x2)
```

```
// Has three parameters, so explicit types are not required
mult(x1, x2, x3) =>
    x1 * x2 * x3

plot(mult(7, 4))
plot(mult(7.5, 4.2))
plot(mult(true, false) ? 1 : 0)
plot(mult("5", "6"))
plot(mult(7, 4, 2))
```

Currency conversion Added a new [currency] argument to most `request.*()` functions. If specified, price values returned by the function will be converted from the source currency to the target currency. The following functions are affected:

- `request.dividends()`
- `request.earnings()`
- `request.financial()`
- `request.security()`

October 2021

Pine Script™ v5 is here! This is a list of the **new** features added to the language, and a few of the **changes** made. See the Pine Script™ v5 Migration guide for a complete list of the **changes** in v5.

New features Libraries are a new type of publication. They allow you to create custom functions for reuse in other scripts. See this manual's page on Libraries.

Pine Script™ now supports switch structures! They provide a more convenient and readable alternative to long ternary operators and if statements.

while loops are here! They allow you to create a loop that will only stop when its controlling condition is false, or a **break** command is used in the loop.

New built-in array variables are maintained by the Pine Script™ runtime to hold the IDs of all the active objects of the same type drawn by your script. They are `label.all`, `line.all`, `box.all` and `table.all`.

The `runtime.error()` function makes it possible to halt the execution of a script and display a runtime error with a custom message. You can use any condition in your script to trigger the call.

Parameter definitions in user-defined functions can now include a default value: a function defined as `f(x = 1) => x` will return 1 when called as `f()`, i.e., without providing an argument for its `x` parameter.

New variables and functions provide better script visibility on strategy information:

- `strategy.closedtrades.entry_price()` and `strategy.opentrades.entry_price()`
- `strategy.closedtrades.entry_bar_index()` and `strategy.opentrades.entry_bar_index()`
- `strategy.closedtrades.entry_time()` and `strategy.opentrades.entry_time()`
- `strategy.closedtrades.size()` and `strategy.opentrades.size()`
- `strategy.closedtrades.profit()` and `strategy.opentrades.profit()`
- `strategy.closedtrades.commission()` and `strategy.opentrades.commission()`
- `strategy.closedtrades.max_runup()` and `strategy.opentrades.max_runup()`
- `strategy.closedtrades.max_drawdown()` and `strategy.opentrades.max_drawdown()`
- `strategy.closedtrades.exit_price()`
- `strategy.closedtrades.exit_bar_index()`
- `strategy.closedtrades.exit_time()`
- `strategy.convert_to_account()`
- `strategy.convert_to_symbol()`
- `strategy.account_currency`

A new `earnings.standardized` constant for the `request.earnings()` function allows requesting standardized earnings data.

A v4 to v5 converter is now included in the Pine Script™ Editor. See the Pine Script™ v5 Migration guide for more information on converting your scripts to v5.

The Reference Manual now includes the systematic mention of the form and type (e.g., “simple int”) required for each function parameter.

The User Manual was reorganized and new content was added.

Changes Many built-in variables, functions and function arguments were renamed or moved to new namespaces in v5. The venerable `study()`, for example, is now `indicator()`, and `security()` is now `request.security()`. New namespaces now group related functions and variables together. This consolidation implements a more rational nomenclature and provides an orderly space to accommodate the many additions planned for Pine Script™.

See the Pine Script™ v5 Migration guide for a complete list of the **changes** made in v5.

September 2021

New parameter has been added for the `dividends()`, `earnings()`, `financial()`, `quandl()`, `security()`, and `splits()` functions:

- `ignore_invalid_symbol` - determines the behavior of the function if the specified symbol is not found: if `false`, the script will halt and return a runtime error; if `true`, the function will return `na` and execution will continue.

July 2021

`tostring` now accepts “bool” and “string” types.

New argument for `time` and `time_close` functions was added:

- `timezone` - timezone of the `session` argument, can only be used when a session is specified. Can be written out in GMT notation (e.g. “GMT-5”) or as an IANA time zone database name (e.g. “America/New_York”).

It is now possible to place a drawing object in the future with `xloc = xloc.bar_index`.

New argument for `study` and `strategy` functions was added:

- `explicit_plot_zorder` - specifies the order in which the indicator’s plots, fills, and hlines are rendered. If true, the plots will be drawn based on the order in which they appear in the indicator’s code, each newer plot being drawn above the previous ones.

June 2021

New variable was added:

- `barstate.islastconfirmedhistory` - returns `true` if script is executing on the dataset’s last bar when market is closed, or script is executing on the bar immediately preceding the real-time bar, if market is open. Returns `false` otherwise.

New function was added:

- `round_to_mintick(x)` - returns the value rounded to the symbol’s mintick, i.e. the nearest value that can be divided by `syminfo.mintick`, without the remainder, with ties rounding up.

Expanded `tostring()` functionality. The function now accepts three new formatting arguments:

- `format.mintick` to format to tick precision.
- `format.volume` to abbreviate large values.
- `format.percent` to format percentages.

May 2021

Improved backtesting functionality by adding the Leverage mechanism.

Added support for table drawings and functions for working with them. Tables are unique objects that are not anchored to specific bars; they float in a script’s space, independently of the chart bars being viewed or the zoom factor used. For more information, see the Tables User Manual page.

New functions were added:

- `color.rgb(red, green, blue, transp)` - creates a new color with transparency using the RGB color model.
- `color.from_gradient(value, bottom_value, top_value, bottom_color, top_color)` - returns color calculated from the linear gradient between `bottom_color` to `top_color`.
- `color.r(color)`, `color.g(color)`, `color.b(color)`, `color.t(color)` - retrieves the value of one of the color components.

- `array.from()` - takes a variable number of arguments with one of the types: `int`, `float`, `bool`, `string`, `label`, `line`, `color`, `box`, `table` and returns an array of the corresponding type.

A new `box` drawing has been added to Pine Script™, making it possible to draw rectangles on charts using the Pine Script™ syntax. For more details see the Pine Script™ reference and the Lines and boxes User Manual page.

The `color.new` function can now accept series and input arguments, in which case, the colors will be calculated at runtime. For more information about this, see our Colors User Manual page.

April 2021

New math constants were added:

- `math.pi` - is a named constant for Archimedes' constant. It is equal to 3.1415926535897932.
- `math.phi` - is a named constant for the golden ratio. It is equal to 1.6180339887498948.
- `math.rphi` - is a named constant for the golden ratio conjugate. It is equal to 0.6180339887498948.
- `math.e` - is a named constant for Euler's number. It is equal to 2.7182818284590452.

New math functions were added:

- `round(x, precision)` - returns the value of `x` rounded to the nearest integer, with ties rounding up. If the `precision` parameter is used, returns a float value rounded to that number of decimal places.
- `median(source, length)` - returns the median of the series.
- `mode(source, length)` - returns the mode of the series. If there are several values with the same frequency, it returns the smallest value.
- `range(source, length)` - returns the difference between the `min` and `max` values in a series.
- `todegrees(radians)` - returns an approximately equivalent angle in degrees from an angle measured in radians.
- `toradians(degrees)` - returns an approximately equivalent angle in radians from an angle measured in degrees.
- `random(min, max, seed)` - returns a pseudo-random value. The function will generate a different sequence of values for each script execution. Using the same value for the optional `seed` argument will produce a repeatable sequence.

New functions were added:

- `session.ismarket` - returns `true` if the current bar is a part of the regular trading hours (i.e. market hours), `false` otherwise.
- `session.ispremarket` - returns `true` if the current bar is a part of the pre-market, `false` otherwise.
- `session.ispostmarket` - returns `true` if the current bar is a part of the post-market, `false` otherwise.
- `str.format` - converts the values to strings based on the specified formats. Accepts certain `number` modifiers: `integer`, `currency`, `percent`.

March 2021

New assignment operators were added:

- `+=` - addition assignment
- `-=` - subtraction assignment
- `*=` - multiplication assignment
- `/=` - division assignment
- `%=` - modulus assignment

New parameters for inputs customization were added:

- `inline` - combines all the input calls with the same inline value in one line.
- `group` - creates a header above all inputs that use the same group string value. The string is also used as the header text.
- `tooltip` - adds a tooltip icon to the `Inputs` menu. The tooltip string is shown when hovering over the tooltip icon.

New argument for `fill` function was added:

- `fillgaps` - controls whether fills continue on gaps when one of the `plot` calls returns an `na` value.

A new keyword was added:

- `varip` - is similar to the `var` keyword, but variables declared with `varip` retain their values between the updates of a real-time bar.

New functions were added:

- `tonumber()` - converts a string value into a float.

- `time_close()` - returns the UNIX timestamp of the close of the current bar, based on the resolution and session that is passed to the function.
- `dividends()` - requests dividends data for the specified symbol.
- `earnings()` - requests earnings data for the specified symbol.
- `splits()` - requests splits data for the specified symbol.

New arguments for the `study()` function were added:

- `resolution_gaps` - fills the gaps between values fetched from higher timeframes when using `resolution`.
- `format.percent` - formats the script output values as a percentage.

February 2021

New variable was added:

- `time_tradingday` - the beginning time of the trading day the current bar belongs to.

January 2021

The following functions now accept a series length parameter:

- `bb()`
- `bbw()`
- `cci()`
- `cmo()`
- `cog()`
- `correlation()`
- `dev()`
- `falling()`
- `mfi()`
- `percentile_linear_interpolation()`
- `percentile_nearest_rank()`
- `percentrank()`
- `rising()`
- `roc()`
- `stdev()`
- `stoch()`
- `variance()`
- `wpr()`

A new type of alerts was added - script alerts. More information can be found in our [Help Center](#).

2020

December 2020

New array types were added:

- `array.new_line()`
- `array.new_label()`
- `array.new_string()`

New functions were added:

- `str.length()` - returns number of chars in source string.
- `array.join()` - concatenates all of the elements in the array into a string and separates these elements with the specified separator.
- `str.split()` - splits a string at a given substring separator.

November 2020

- New `max_labels_count` and `max_lines_count` parameters were added to the `study` and `strategy` functions. Now you can manage the number of lines and labels by setting values for these parameters from 1 to 500.

New function was added:

- `array.range()` - return the difference between the min and max values in the array.

October 2020

The behavior of `rising()` and `falling()` functions have changed. For example, `rising(close,3)` is now calculated as following:

`close[0] > close[1]` and `close[1] > close[2]` and `close[2] > close[3]`

September 2020

Added support for `input.color` to the `input()` function. Now you can provide script users with color selection through the script's "Settings/Inputs" tab with the same color widget used throughout the TradingView user interface. Learn more about this feature in our blog

```
//@version=4
study("My Script", overlay = true)
color c_labelColor = input(color.green, "Main Color", input.color)
var l = label.new(bar_index, close, yloc = yloc.abovebar, text = "Colored label")
label.set_x(l, bar_index)
label.set_color(l, c_labelColor)
```

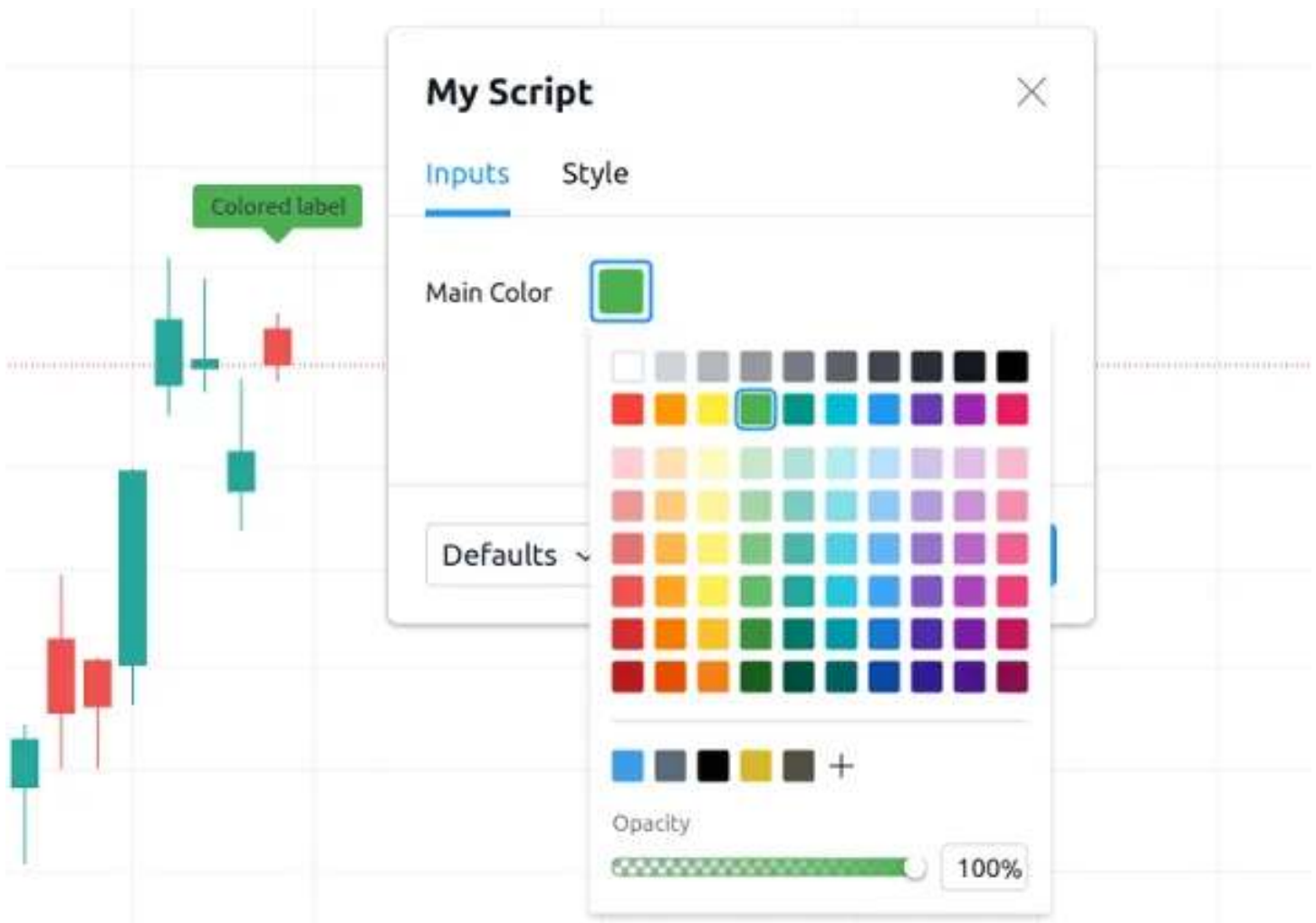


Figure 487: image

Added support for arrays and functions for working with them. You can now use the powerful new array feature to build custom datasets. See our User Manual page on arrays and our blog

```
//@version=4
study("My Script")
```

```

a = array.new_float(0)
for i = 0 to 5
    array.push(a, close[i] - open[i])
plot(array.get(a, 4))

```

The following functions now accept a series length parameter. Learn more about this feature in our [blog](#):

- `alma()`
- `change()`
- `highest()`
- `highestbars()`
- `linreg()`
- `lowest()`
- `lowestbars()`
- `mom()`
- `sma()`
- `sum()`
- `vwma()`
- `wma()`

```

//@version=4
study("My Script", overlay = true)
length = input(10, "Length", input.integer, minval = 1, maxval = 100)
avgBar = avg(highestbars(length), lowestbars(length))
float dynLen = nz(abs(avgBar) + 1, length)
dynSma = sma(close, int(dynLen))
plot(dynSma)

```

August 2020

- Optimized script compilation time. Scripts now compile 1.5 to 2 times faster.

July 2020

- Minor bug fixes and improvements.

June 2020

- New **resolution** parameter was added to the **study** function. Now you can add MTF functionality to scripts and decide the timeframe you want the indicator to run on.

Please note that you need to reapply the indicator in order for the [resolution] parameter to appear.

- The **tooltip** argument was added to the **label.new** function along with the **label.set_tooltip** function:

```

//@version=4
study("My Script", overlay=true)
var l=label.new(bar_index, close, yloc=yloc.abovebar, text="Label")
label.set_x(l, bar_index)
label.set_tooltip(l, "Label Tooltip")

```

- Added an ability to create alerts on strategies.
- A new function `line.get_price()` can be used to determine the price level at which the line is located on a certain bar.
- New label styles allow you to position the label pointer in any direction.
- Find and Replace was added to Pine Editor. To use this, press CTRL+F (find) or CTRL+H (find and replace).
- **timezone** argument was added for time functions. Now you can specify timezone for **second**, **minute**, **hour**, **year**, **month**, **dayofmonth**, **dayofweek** functions:

```

//@version=4
study("My Script")
plot(hour(1591012800000, "GMT+1"))

```

- **syminfo.basecurrency** variable was added. Returns the base currency code of the current symbol. For EURUSD symbol returns EUR.

MA

✕

Inputs

Style

Resolution

Same as symbol

▼

Length

9

Source

close

▼

Offset

0

Defaults

▼

Cancel

Ok

Figure 488: image



Figure 489: image

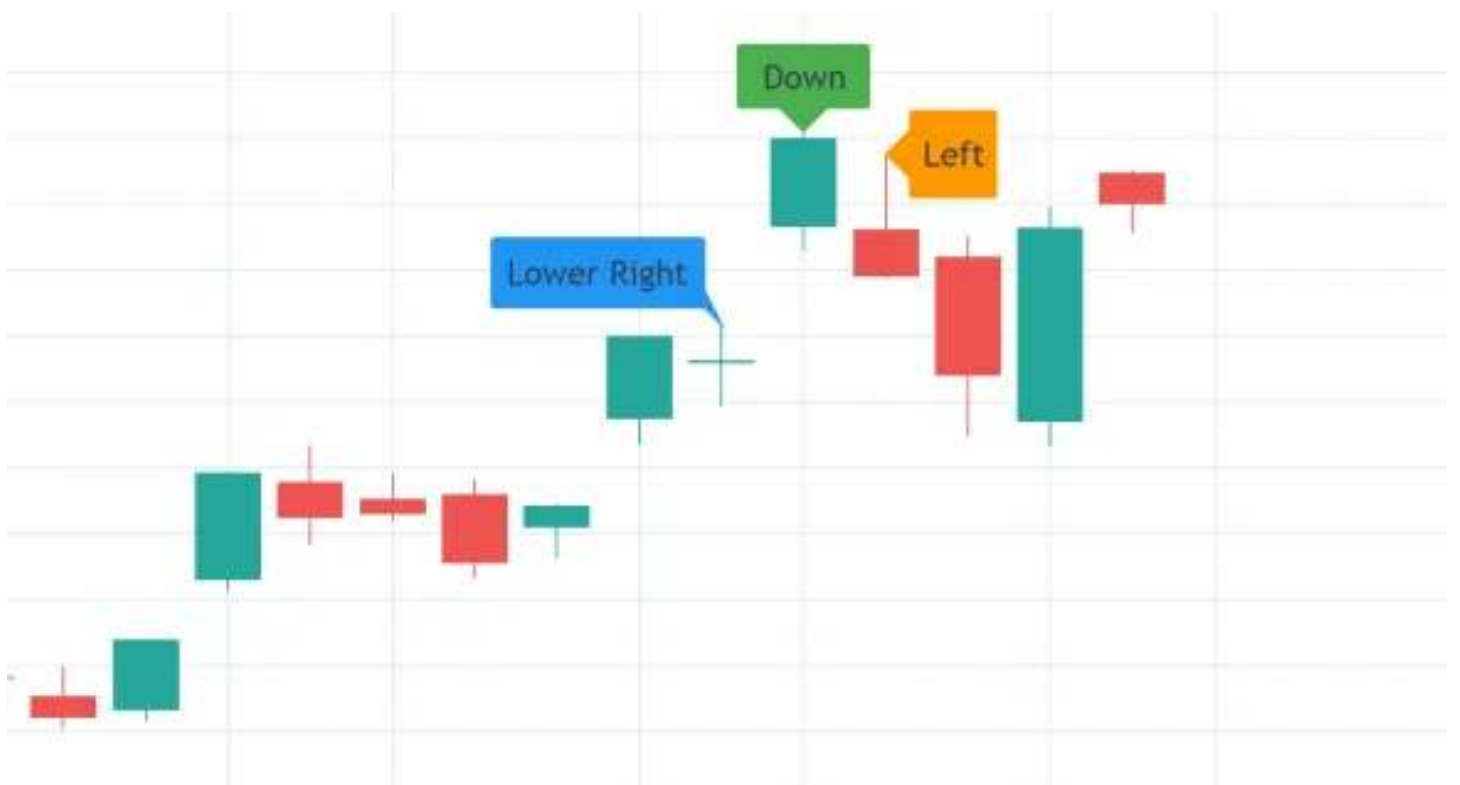


Figure 490: image



Figure 491: image

May 2020

- `else if` statement was added
- The behavior of `security()` function has changed: the `expression` parameter can be series or tuple.

April 2020

New function was added:

- `quandl()` - request quandl data for a symbol

March 2020

New function was added:

- `financial()` - request financial data for a symbol

New functions for common indicators were added:

- `cmo()` - Chande Momentum Oscillator
- `mfi()` - Money Flow Index
- `bb()` - Bollinger Bands
- `bbw()` - Bollinger Bands Width
- `kc()` - Keltner Channels
- `kcw()` - Keltner Channels Width
- `dmi()` - DMI/ADX
- `wpr()` - Williams % R
- `hma()` - Hull Moving Average
- `supertrend()` - SuperTrend

Added a detailed description of all the fields in the Strategy Tester Report.

February 2020

- New Pine Script™ indicator VWAP Anchored was added. Now you can specify the time period: Session, Month, Week, Year.
- Fixed a problem with calculating `percentrank` function. Now it can return a zero value, which did not happen before due to an incorrect calculation.
- The default `transparency` parameter for the `plot()`, `plotshape()`, and `plotchar()` functions is now 0%.
- For the functions `plot()`, `plotshape()`, `plotchar()`, `plotbar()`, `plotcandle()`, `plotarrow()`, you can set the `display` parameter, which controls the display of the plot. The following values can be assigned to it:
 - `display.none` - the plot is not displayed
 - `display.all` - the plot is displayed (Default)
- The `textalign` argument was added to the `label.new` function along with the `label.set_textalign` function. Using those, you can control the alignment of the label's text:

```
//@version=4
study("My Script", overlay = true)
var l = label.new(bar_index, high, text="Right\n aligned\n text", textalign=text.align_right)
label.set_xy(l, bar_index, high)

.. image:: images/ReleaseNotes-Label_text_align.png
```

January 2020

New built-in variables were added:

- `iii` - Intraday Intensity Index
- `wvad` - Williams Variable Accumulation/Distribution
- `wad` - Williams Accumulation/Distribution
- `obv` - On Balance Volume
- `pvt` - Price-Volume Trend
- `nvi` - Negative Volume Index
- `pvi` - Positive Volume Index

New parameters were added for `strategy.close()`:

- `qty` - the number of contracts/shares/lots/units to exit a trade with
- `qty_percent` - defines the percentage of entered contracts/shares/lots/units to exit a trade with
- `comment` - additional notes on the order

New parameter was added for `strategy.close_all`:

- `comment` - additional notes on the order

2019

December 2019

- Warning messages were added.

For example, if you don't specify exit parameters for `strategy.exit` - `profit`, `limit`, `loss`, `stop` or one of the following pairs: `trail_offset` and `trail_price` / `trail_points` - you will see a warning message in the console in the Pine Script™ editor.

- Increased the maximum number of arguments in `max`, `min`, `avg` functions. Now you can use up to ten arguments in these functions.

October 2019

- `plotchar()` function now supports most of the Unicode symbols:

```
//@version=4
study("My Script", overlay=true)
plotchar(open > close, char=" ")
```

.. image:: images/ReleaseNotes-Bears_in_plotchar.png

- New `bordercolor` argument of the `plotcandle()` function allows you to change the color of candles' borders:

```
//@version=4
study("My Script")
plotcandle(open, high, low, close, title='Title', color = open < close ? color.green : color.red, wickcolor=co
```

- New variables added:
 - `syminfo.description` - returns a description of the current symbol
 - `syminfo.currency` - returns the currency code of the current symbol (EUR, USD, etc.)
 - `syminfo.type` - returns the type of the current symbol (stock, futures, index, etc.)

September 2019

New parameters to the `strategy` function were added:

- `process_orders_on_close` allows the broker emulator to try to execute orders after calculating the strategy at the bar's close
- `close_entries_rule` allows to define the sequence used for closing positions

Some fixes were made:

- `fill()` function now works correctly with `na` as the `color` parameter value
- `sign()` function now calculates correctly for literals and constants

`str.replace_all(source, target, replacement)` function was added. It replaces each occurrence of a `target` string in the `source` string with a `replacement` string

July-August 2019

New variables added:

- `timeframe.isseconds` returns true when current resolution is in seconds
- `timeframe.isminutes` returns true when current resolution is in minutes
- `time_close` returns the current bar's close time

The behavior of some functions, variables and operators has changed:

- The `time` variable returns the correct open time of the bar for more special cases than before
- An optional `seconds` parameter of the `timestamp()` function allows you to set the time to within seconds
- `security()` function:
 - Added the possibility of requesting resolutions in seconds:
 - 1, 5, 15, 30 seconds (chart resolution should be less than or equal to the requested resolution)
 - Reduced the maximum value that can be requested in some of the other resolutions:
 - from 1 to 1440 minutes
 - from 1 to 365 days
 - from 1 to 52 weeks
 - from 1 to 12 months
- Changes to the evaluation of ternary operator branches:

In Pine Script™ v3, during the execution of a ternary operator, both its branches are calculated, so when this script is added to the chart, a long position is opened, even if the `long()` function is not called:

```
//@version=3
strategy(title = "My Strategy")
long() =>
    strategy.entry("long", true, 1, when = open > high[1])
    1
c = 0
c := true ? 1 : long()
plot(c)
```

Pine Script v4 contains built-in functions with side effects (```line.new``` and ```label.new```). If calls to t

June 2019

- Support for drawing objects. Added *label* and *line* drawings
- `var` keyword for one time variable initialization
- Type system improvements:
 - *series string* data type
 - functions for explicit type casting
 - syntax for explicit variable type declaration
 - new *input* type forms
- Renaming of built-ins and a version 3 to 4 converter utility
- `max_bars_back` function to control series variables internal history buffer sizes
- Pine Script™ documentation versioning

2018

October 2018

- To increase the number of indicators available to the whole community, Invite-Only scripts can now be published by Premium users only.

April 2018

- Improved the Strategy Tester by reworking the Maximum Drawdown calculation formula.

2017

August 2017

- With the new argument `show_last` in the plot-type functions, you can restrict the number of bars that the plot is displayed on.

June 2017

- A major script publishing improvement: it is now possible to update your script without publishing a new one via the Update button in the publishing dialog.

May 2017

- Expanded the type system by adding a new type of constants that can be calculated during compilation.

April 2017

- Expanded the keyword argument functionality: it is now possible to use keyword arguments in all built-in functions.
- A new `barstate.isconfirmed` variable has been added to the list of variables that return bar status. It lets you create indicators that are calculated based on the closed bars only.
- The `options` argument for the `input()` function creates an input with a set of options defined by the script's author.

March 2017

- Pine Script™ v3 is here! Some important changes:
 - Changes to the default behavior of the `security()` function: it can no longer access the future data by default. This can be changes with the `lookahead` parameter.
 - An implicit conversion of boolean values to numeric values was replaced with an implicit conversion of numeric values (integer and float) to boolean values.
 - Self-referenced and forward-referenced variables were removed. Any PineScript code that used those language constructions can be equivalently rewritten using mutable variables.

February 2017

- Several improvements to the strategy tester and the strategy report:
 - New Buy & Hold equity graph — a new graph that lets you compare performance of your strategy versus a “buy and hold”, i.e if you just bought a security and held onto it without trading.
 - Added percentage values to the absolute currency values.
 - Added Buy & Hold Return to display the final value of Buy & Hold Equity based on last price.
 - Added Sharpe Ratio — it shows the relative effectiveness of the investment portfolio (security), a measure that indicates the average return minus the risk-free return divided by the standard deviation of return on an investment.
 - Slippage lets you simulate a situation when orders are filled at a worse price than expected. It can be set through the Properties dialog or through the slippage argument in the `strategy()` function.
 - Commission allows you to add commission for placed orders in percent of order value, fixed price or per contract. The amount of commission paid is shown in the Commission Paid field. The commission size and its type can be set through the Properties dialog or through the `commission_type` and `commission_value` arguments in the `strategy()` function.

2016

December 2016

- Added invite-only scripts. The invite-only indicators are visible in the Community Scripts, but nobody can use them without explicit permission from the author, and only the author can see the source code.

October 2016

- Introduced indicator revisions. Each time an indicator is saved, it gets a new revision, and it is possible to easily switch to any past revision from the Pine Editor.

September 2016

- It is now possible to publish indicators with protected source code. These indicators are available in the public Script Library, and any user can use them, but only the author can see the source code.

July 2016

- Improved the behavior of the `fill()` function: one call can now support several different colors.

March 2016

- Color type variables now have an additional parameter to set default transparency. The transparency can be set with the `color.new()` function, or by adding an alpha-channel value to a hex color code.

February 2016

- Added `for` loops and keywords `break` and `continue`.
- Pine Script™ now supports mutable variables! Use the `:=` operator to assign a new value to a variable that has already been defined.
- Multiple improvements and bug fixes for strategies.

January 2016

- A new `alertcondition()` function allows for creating custom alert conditions in Pine Script™-based indicators.

2015

October 2015

- Pine has graduated to v2! The new version of Pine Script™ added support for `if` statements, making it easier to write more readable and concise code.

September 2015

- Added backtesting functionality to Pine Script™. It is now possible to create trading strategies, i.e. scripts that can send, modify and cancel orders to buy or sell. Strategies allow you to perform backtesting (emulation of strategy trading on historical data) and forward testing (emulation of strategy trading on real-time data) according to your algorithms. Detailed information about the strategy's calculations and the order fills can be seen in the newly added Strategy Tester tab.

July 2015

- A new `editable` parameter allows hiding the plot from the Style menu in the indicator settings so that it is not possible to edit its style. The parameter has been added to all the following functions: all plot-type functions, `barcolor()`, `bgcolor()`, `hline()`, and `fill()`.

June 2015

- Added two new functions to display custom barsets using PineScript: `plotbar()` and `plotcandle()`.

April 2015

- Added two new shapes to the `plotshape()` function: `shape.labelup` and `shape.labeldown`.
- PineScript Editor has been improved and moved to a new panel at the bottom of the page.
- Added a new `step` argument for the `input()` function, allowing to specify the step size for the indicator's inputs.

March 2015

- Added support for inputs with the `source` type to the `input()` function, allowing to select the data source for the indicator's calculations from its settings.

February 2015

- Added a new `text` argument to `plotshape()` and `plotchar()` functions.
- Added four new shapes to the `plotshape()` function: `shape.arrowup`, `shape.arrowdown`, `shape.square`, `shape.diamond`.

2014

August 2014

- Improved the script sharing capabilities, changed the layout of the Indicators menu and separated published scripts from ideas.

July 2014

- Added three new plotting functions, `plotshape()`, `plotchar()`, and `plotarrow()` for situations when you need to highlight specific bars on a chart without drawing a line.
- Integrated QUANDL data into Pine Script™. The data can be accessed by passing the QUANDL ticker to the `security` function.

June 2014

- Added Pine Script™ sharing, enabling programmers and traders to share their scripts with the rest of the TradingView community.

April 2014

- Added line wrapping.

February 2014

- Added support for inputs, allowing users to edit the indicator inputs through the properties window, without needing to edit the Pine script.
- Added self-referencing variables.
- Added support for multiline functions.
- Implemented the type-casting mechanism, automatically casting constant and simple float and int values to series when it is required.
- Added several new functions and improved the existing ones:
 - `barssince()` and `valuewhen()` allow you to check conditions on historical data easier.
 - The new `barcolor()` function lets you specify a color for a bar based on filling of a certain condition.
 - Similar to the `barcolor()` function, the `bgbcolor()` function changes the color of the background.
 - Reworked the `security()` function, further expanding its functionality.
 - Improved the `fill()` function, enabling it to be used more than once in one script.
 - Added the `round()` function to round and convert float values to integers.

2013

- The first version of Pine Script™ is introduced to all TradingView users, initially as an open beta, on December 13th.

[\[Previous](#)

[Error messages\]\(#error-messages\)\[Next](#)

Overview

The migration guides catalogue all changes introduced between Pine Script™ versions and explain how to rewrite code to convert the indicator successfully. Each guide details sequential changes; for example, the To Pine Script™ version 6 page assumes that the script is already written in Pine v5.

Pine converter

Scripts written in every Pine Script version starting from v3 can be converted to the next version automatically using the converter available in the “Manage Scripts” menu:

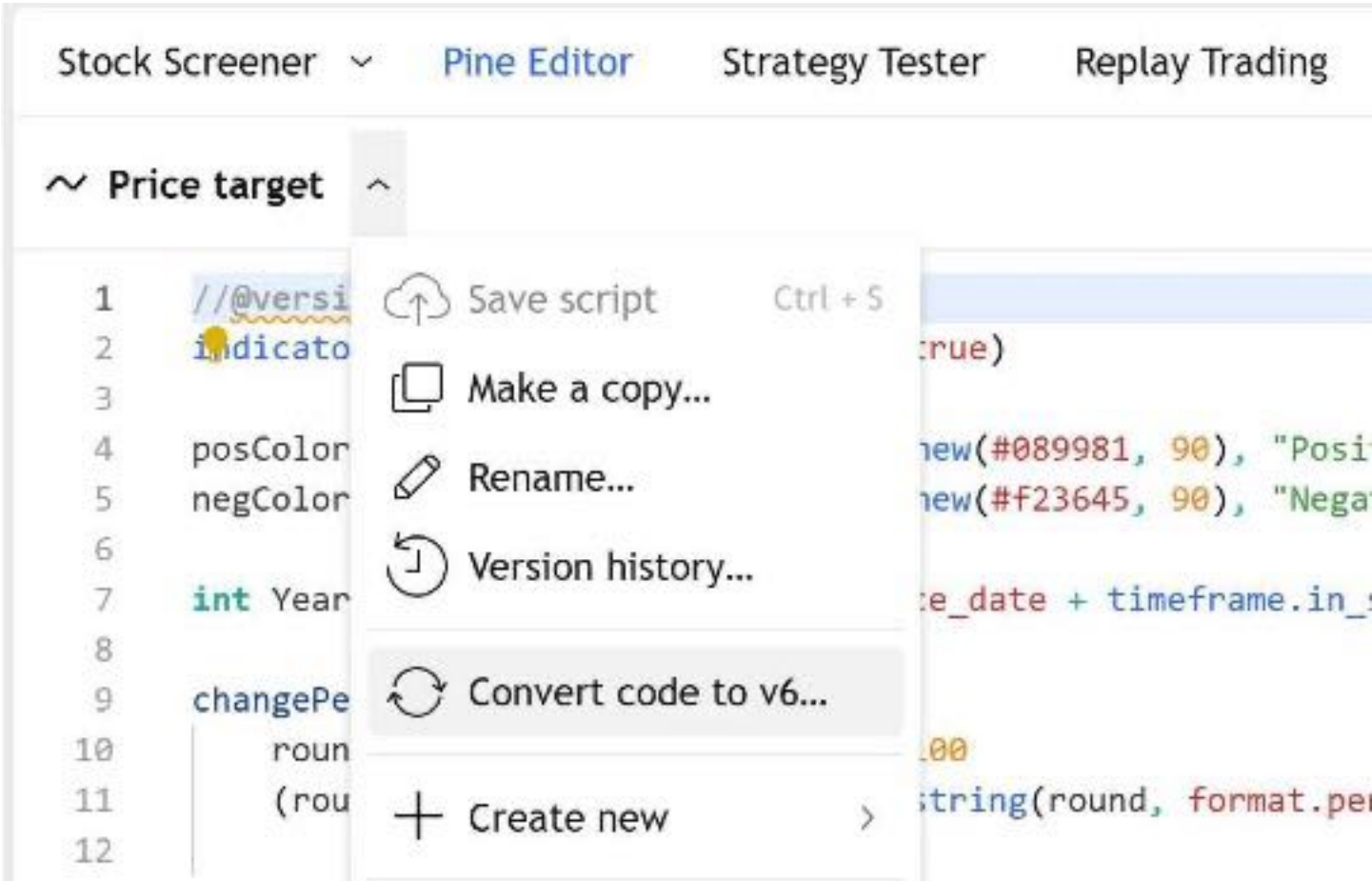


Figure 492: image

A script can be converted only if its code compiles successfully. In rare cases, converting a valid script automatically can result in a script with compilation errors. In that case, resolve the errors using the information in the appropriate article.

[Next

To Pine Script™ version 6

Introduction

Pine Script v6 introduces a number of changes and new features. See the Release Notes for a list of all new features.

Some changes are not compatible with v5 scripts. This guide explains how to update your script from v5 to v6. If you want to convert a script from v4 or earlier to v6, refer to the migration guides for previous versions and update the script one version at a time.

The Pine Editor converter can handle many of these changes *automatically*, while other changes might require *manual* fixes.

Here are the changes that affect v5 scripts:

- Values of the “int” and “float” types are no longer implicitly cast to “bool”.
- Boolean values can no longer be na, and the na(), nz(), and fixnan() functions no longer accept “bool” arguments.
- The and and or operators now evaluate conditions lazily.
- All **request.*()** functions can now execute dynamically.
- Division of two “const int” values can now return a fractional value.
- The **when** parameter is removed from all applicable **strategy.*()** functions.
- The default long and short margin percentage for strategies is now 100.
- Strategies now trim the oldest orders in their results instead of raising an error when they exceed the 9000 trade limit.
- The strategy.exit() command no longer ignores relative parameters defining take-profit and stop-loss prices or trailing stop activation levels when the call also includes arguments for the related absolute parameters.
- The history-referencing operator [] can no longer reference the history of literal values or fields of user-defined types directly.
- Function calls can no longer include more than one argument for the same parameter.
- The **offset** parameter of plot() and other functions no longer accepts “series” values.
- na values are no longer allowed in place of built-in constants of unique types.
- The value of timeframe.period now always includes a multiplier (e.g., “1D” instead of “D”).
- Some **array.*()** functions now accept negative index arguments.
- Some mutable variables are no longer erroneously marked as “const”.
- The **transp** parameter is removed from all applicable functions.
- Some default colors and color constants have updated values.
- The for loop statement now evaluates its end boundary dynamically before every iteration.

Converting v5 to v6 using the Pine Editor

The Pine Editor can automatically convert a v5 script to v6. The Pine Editor highlights the `//@version=5` annotation of a v5 script in yellow.

To convert the script, click the editor’s “Manage script” dropdown menu and select “Convert code to v6”:

A script can be converted only if its v5 code compiles successfully. In rare cases, converting the script automatically can result in a v6 script with compilation errors. In that case, the errors are highlighted in the Editor, and you need to resolve them by using the information in the following sections.

Dynamic requests

In Pine v6, scripts can call all **request.*()** functions *dynamically by default*, allowing any single **request.*()** call instance in the code to request data from different datasets and work within local scopes.

When a script does *not* use dynamic requests, the *context* of a data request (ticker ID and timeframe) must be known on the first script execution and remain *unchanged* across all subsequent executions. Therefore, **symbol**, **timeframe**, and other parameters specifying a non-dynamic **request.*()** call’s context require arguments with the “simple” qualifier. Additionally, non-dynamic requests must execute globally, meaning they are not allowed inside the local scopes of loops and other structures.

In contrast, when a script allows dynamic requests, it can call **request.*()** functions with “*series*” arguments for the parameters that define the context of the data requests. With this qualifier change, scripts can:

- Retrieve data from new datasets on *any* historical bar, even *after* the first available bar, with a single **request.*()** call instance.
- Store symbol and timeframe strings in collections or objects, then use the collected values to define a **request.*()** call’s context.
- Call **request.*()** functions within the local scopes of loops and conditional structures.
- Export library functions containing **request.*()** calls.

Non-dynamic requests are the default in Pine v5. Scripts coded in v5 can execute dynamic requests, but only if programmers specify **dynamic_requests = true** in the indicator(), strategy(), or library() declaration statement. If not specified, the default argument is **false**.

In Pine v6, dynamic requests are *always* available by default. When a script includes **request.*()** calls, the compiler analyzes the script to determine whether dynamic requests are necessary. If unnecessary for the script, the compiler automatically turns the feature off to optimize performance.

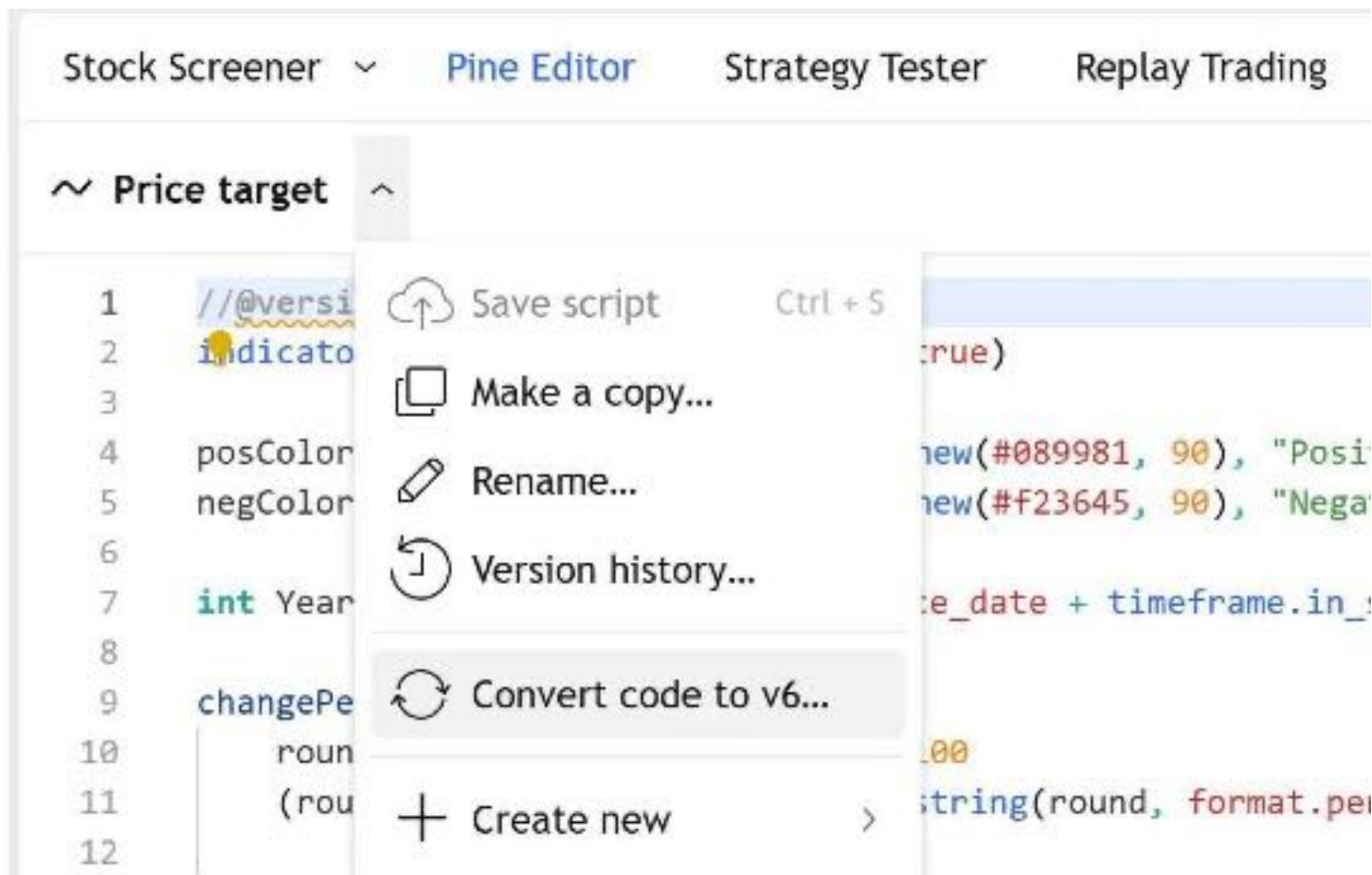


Figure 493: image

The following v6 example uses a single `request.security()` instance in a loop to retrieve data for multiple symbols stored in an array. On each iteration, the script dynamically retrieves the close price for one of the stored symbols from the “1D” timeframe and pushes the retrieved value into an array. After the loop terminates, it calculates that array’s average and plots the result. In Pine v5, this script would cause a *compilation error* unless we included `dynamic_requests = true` in the declaration statement:



Figure 494: image

```
//@version=6
indicator("Dynamic `request` demo")
//For v5: must add `dynamic_requests = true` to `indicator()` for this code to work.

//@variable User-input toggle to display each symbol's `close` price on chart alongside average `close`.
bool showSymbols = input.bool(false, "Plot symbol closes")

//@variable Persistent array of "string" symbol ticker IDs to request for our custom index.
var array<string> symbols = array.from("NASDAQ:MSFT", "NASDAQ:AAPL", "NASDAQ:GOOGL", "NASDAQ:NVDA")

//@variable Array storing the `close` prices for the `symbols` on each bar.
array<float> symCloses = array.new<float>()

// Loop through `symbols` and request daily `close` prices.
for [i, sym] in symbols
    float reqClose = request.security(sym, "1D", close)
    symCloses.push(reqClose)

// Calculate and plot the average `close` for the `symbols` to create our custom index plot.
float avgClose = symCloses.avg()
plot(avgClose, "Avg close", avgClose >= avgClose[1] ? color.green : color.red, 3)

// Plot each symbol's `close` for reference if `showSymbols` is `true`.
plot(showSymbols ? symCloses.get(0) : na, "MSFT", color.blue)
plot(showSymbols ? symCloses.get(1) : na, "AAPL", color.navy)
plot(showSymbols ? symCloses.get(2) : na, "GOOGL", color.aqua)
plot(showSymbols ? symCloses.get(3) : na, "NVDA", color.teal)
```

There are minor differences between dynamic and non-dynamic requests in some obscure cases, such as when using the result of one `request.security()` call in the `expression` argument of another call. As a result, a valid v5 script without dynamic

requests can behave differently after conversion to v6, even if nothing related to the requests was changed in the code.

Fix: In Pine v6, the `dynamic_requests` parameter of the `indicator()`, `strategy()`, and `library()` functions is `true` by default. If you find differences in a `request.*()` call's behavior after converting a script to v6, you can include `dynamic_requests = false` in the declaration statement to force dynamic requests off and replicate most of the previous v5 behavior.

It's important to note that, in Pine v5, it is possible to call user-defined functions or methods containing `request.*()` calls inside the local blocks of loops and conditional structures *without* enabling dynamic requests. However, such calls still require “*simple*” arguments for all parameters defining the requested context, which limits their utility.

In Pine v6, calling a `request.*()` function from the scope of a loop or conditional structure is **not allowed** if `dynamic_requests` is set to `false` in the script's declaration statement, even if the call is within a user-defined function. A v6 script that attempts to use wrapped `request.*()` calls in local scopes without dynamic requests enabled causes a *compilation error*.

Fix: If a v5 script specifies `dynamic_requests` is `false` in its declaration statement and uses functions containing `request.*()` calls inside local blocks, remove the explicit `dynamic_requests` argument when converting the script to v6. The converted script will use dynamic requests automatically, allowing the functions containing `request.*()` calls to work correctly inside local scopes.

Types

The following changes have been made to how Pine handles types.

Explicit “bool” casting

In Pine v6, “int” and “float” values are no longer implicitly cast to “bool”.

In Pine v5, values of “int” and “float” types can be implicitly cast to “bool” when an expression or function requires a boolean value. In such cases, `na`, `0`, or `0.0` are considered `false`, and *any other value* is considered `true`.

For example, take a look at this conditional expression:

```
color expr = bar_index ? color.green : color.red
```

It assigns `color.red` to `expr` on the *first* bar of the chart, because that bar has a `bar_index` of `0`, and then assigns `color.green` on *every* following bar, because any *non-zero* value is `true`. The ternary operator `?:` expects a “bool” expression for its condition, but in v5 it can also accept a numeric value as its conditional expression, which it automatically converts (implicitly casts) to a “bool”.

In v6, scripts must *explicitly* cast a numeric value to “bool” to use it where a “bool” type is required.

Fix: Wrap the numeric value with the `bool()` function to cast it explicitly.

```
color expr = bool(bar_index) ? color.green : color.red
```

Boolean values cannot be na

In v6, “bool” values can no longer be `na`. Consequently, the `na()`, `nz()`, and `fixnan()` functions no longer accept “bool” types.

In v5, “bool” variables have *three* possible values: they can be `true`, `false`, or `na`. The boolean `na` value behaves differently from both `true` and `false`:

- When implicitly cast to “bool”, `na` is evaluated as `false`.
- The boolean `na` value is **not** considered equal to `false` when compared using the `==` operator.
- When the boolean `na` value is passed to the `na()` function, it returns `true`, whereas `na(true)` and `na(false)` both return `false`.

To manage the boolean `na` value, the `na()`, `nz()`, and `fixnan()` functions in v5 have overloads that accept “bool” type arguments. This third boolean state leads to occasional confusion in v5 scripts.

In v6, this is no longer the case: a “bool” must be *either* `true` or `false`, with **no** third state. This means that in v6 scripts:

- A variable declared as “bool” can **no longer** be assigned `na` as its default value.
- In conditional expressions like `if` and `switch`, if the return type of the expression is “bool”, any *unspecified* condition returns `false` instead of `na`.

- Expressions that returned a boolean **na** value in v5 now return **false**. For example, using the history-referencing operator `[]` on the very first bar of the dataset to request a historical value of a “bool” variable returned **na** in v5, because no past bars exist, but in Pine v6 it returns **false**.
- Functions that explicitly check whether a value is **na** – specifically, `na()`, `nz()`, and `fixnan()` – do **not** accept “bool” arguments in v6.

This example v5 script creates a simple strategy that switches between long and short positions when two moving averages cross. An if-statement assigns **true** or **false** to a “bool” variable `isLong` to track the trade’s long or short direction, using the strategy’s positive (> 0) or negative (< 0) position size. However, when the position size is zero, *neither* of these conditions are valid. In v5, the undefined condition ($= 0$) assigns **na** to the variable `isLong`.

Therefore, a boolean **na** value occurs on the first few bars in the dataset before the strategy enters any positions. We can visualize the three “bool” states by setting the background color based on the value of `isLong`:



Figure 495: image

```
//@version=5
strategy("Bool `na` demo v5", overlay=true, margin_long=100, margin_short=100)

// Strategy's long and short trades are based on moving average cross over/under.
longCondition = ta.crossover(ta.sma(close, 14), ta.sma(close, 28))
if (longCondition)
    strategy.entry("My Long Entry Id", strategy.long)

shortCondition = ta.crossunder(ta.sma(close, 14), ta.sma(close, 28))
if (shortCondition)
    strategy.entry("My Short Entry Id", strategy.short)

//@variable Boolean variable that tracks the current direction of the trade.
// Is `true` when `position_size` is greater than 0 (long), and `false` when `position_size` is less than 0 (s
bool isLong = if strategy.position_size > 0
    true
else if strategy.position_size < 0
    false
// When `position_size` is equal to 0, neither condition is met. In v5, an undefined condition sets `isLong` t

//@variable Background color, set depending on the state of `isLong` (`true`/`false`/`na`).
color stateColor = switch
    isLong == true => color.new(color.blue, 90) // Blue color if long position.
    isLong == false => color.new(color.orange, 90) // Orange color if short position.
    na(isLong) => color.new(color.red, 40) // Red color if no position. Note this line is invalid
bgcolor(stateColor)

// On the first bar, display the raw value of `isLong` in a table.
```

```

if barstate.isfirst
    var table t = table.new(position.bottom_right, 2, 4, color.yellow, frame_color = color.black, frame_width
    t.cell(0, 0, "On first bar")
    t.cell(0, 1, "`isLong` raw value:", bgcolor = color.new(color.red, 40))
    t.cell(1, 1, str.tostring(isLong), bgcolor = color.new(color.red, 40))
    // Compare `isLong` value to Boolean `true` and `false` values.
    t.cell(0, 2, "`isLong` == `true`?")
    t.cell(1, 2, str.tostring(isLong == true))
    t.cell(0, 3, "`isLong` == `false`?")
    t.cell(1, 3, str.tostring(isLong == false))

```

Fix: Remove any `na()`, `nz()`, and `fixnan()` functions that run on “bool” values. Ensure that all “bool” values are correctly interpreted as `true` or `false` states **only**. If your code logic requires a third `na` state to execute as intended, rewrite the code using a different type or structure to achieve the previous three-state behavior.

To adapt our code to Pine v6, we must first remove the following line to resolve the initial compilation error:

```
na(isLong)      => color.new(color.red, 40)
```

In v6, the undefined condition (`strategy.position_size == 0`) now returns `false` instead of `na`. Consequently, the script *incorrectly* highlights the bars where there are *no* trade positions the same color as those where there are *short* positions, since `isLong` has the same `false` result for both conditions:



Figure 496: image

We want to distinguish between *three* unique states: long positions, short positions, and no entered positions. Therefore, using a two-state Boolean variable in v6 is no longer suitable. Instead, to maintain our desired behavior, we must *rewrite* the v6 code to replace the “bool” variable with a different type. For example, we can use an “int” variable to represent our three different `position_size` states using -1, 0, and 1:

```

//@version=6
strategy("Bool `na` demo v6", overlay=true, margin_long=100, margin_short=100)

// Strategy's long and short trades are based on moving average cross over/under.
longCondition = ta.crossover(ta.sma(close, 14), ta.sma(close, 28))
if (longCondition)
    strategy.entry("My Long Entry Id", strategy.long)

shortCondition = ta.crossunder(ta.sma(close, 14), ta.sma(close, 28))
if (shortCondition)
    strategy.entry("My Short Entry Id", strategy.short)

//@variable Integer variable that tracks the current direction of the trade.
// Is `-1` when `position_size` is less than 0 (short), `+1` when `position_size` is greater than 0 (long),
// and `0` when `position_size` is equal to 0 (no trades).
int tradeDirection = if strategy.position_size < 0

```




Figure 497: image

```

-1
else if strategy.position_size > 0
    1
else //strategy.position_size == 0
    0

//@variable Background color, set depending on the `tradeDirection`.
color directionColor = switch
    tradeDirection == 1 => color.new(color.blue, 90) // Blue color if long position.
    tradeDirection == -1 => color.new(color.orange, 90) // Orange color if short position.
    tradeDirection == 0 => na // No color if no position.
bgcolor(directionColor)

// On the first bar, display the value of `tradeDirection` in a table for reference.
var table t = table.new(position.bottom_right, 2, 3, color.yellow, frame_color = color.black, frame_width = 1)
if barstate.isfirst
    t.cell(0, 0, "On first bar")
    t.cell(0, 1, "`tradeDirection` value:", bgcolor = color.new(color.green, 60))
    t.cell(1, 1, str.toString(tradeDirection), bgcolor = color.new(color.green, 60))

    //@variable A "string" representation of `tradeDirection` value on current bar.
    string directionString = tradeDirection == 1 ? "Long" : tradeDirection == -1 ? "Short" : "No entered posit
    t.cell(0, 2, "State: ")
    t.cell(1, 2, directionString)

if barstate.islastconfirmedhistory
    //@variable A "string" representation of `tradeDirection` value on current bar.
    string directionString = tradeDirection == 1 ? "Long" : tradeDirection == -1 ? "Short" : "No entered posit
    label.new(bar_index, high, "On last bar \n `tradeDirection` value: " + str.toString(tradeDirection)
        + "\n State: " + directionString)

```

Unique parameters cannot be na

Some Pine Script function parameters expect values of *unique* types. For example, the `style` parameter of the `plot()` function expects a value of the “input plot_style” qualified type, which must be one of the constants in the `plot.style_*` group.

In v5, passing `na` to the `plot()` function’s `style` parameter simply plots a line using the default style `plot.style_line`, without raising an error.

In v6, parameters that expect unique types **no longer** accept `na` values. Additionally, conditional expressions that return these unique types must be used in a form that **cannot** result in an `na` value. For example, a switch-statement must have a

default block, and an if-statement must have an else-block, because these conditional expressions can return na otherwise. The following example script shows two code structures that work in v5 but raise errors in v6.



Figure 498: image

```
//@version=5
indicator("`na` and unique types demo v5")

//@variable User-selected "string" to determine type of plot used for `plot()` function's `style` argument.
string inputStyle = input.string("Area", "Plot style", options = ["Area", "Columns", "Histogram", "Stepline-diamond"])

// Initialize an `input plot_style` type variable based on user's selected `inputStyle`.
selectedPlotStyle = switch inputStyle
    "Area"          => plot.style_area
    "Columns"       => plot.style_columns
    "Histogram"     => plot.style_histogram
    "Stepline-diamond" => plot.style_stepline_diamond
// `switch` statement covers all `inputStyle` options, but does not include `default` block.
// Valid in v5. Invalid in v6 - `switch` statement must include a `default` block, otherwise raises error.

plot(close, "Source plot", color.blue, 2, style = selectedPlotStyle)

//@variable Toggle for the style of the line plotted at price "100".
inputHundredStyle = input.bool(true, " Use crosses style for '100-line'")

hundredLineStyle = if inputHundredStyle
    plot.style_cross
// Since there is no `else` block, setting `inputHundredStyle` to `false` makes this variable `na`.
// In v5, passing `na` to the `style` parameter makes the `plot()` function use its default style `plot.style_area`.
// In v6, this raises a compilation error because `style` cannot be `na`.

// Plot the "100-line" using the `hundredLineStyle` style constant.
plot(100, "100-line", color.orange, 4, style = hundredLineStyle)
```

Fix: Ensure that no na value is passed to parameters that expect unique types, and that all conditional statements return a suitable non-na value.

```
//@version=6
```

```

indicator("`na` and unique types demo v6")

//@variable User-selected "string" to determine type of plot used for `plot()` function's `style` argument.
string inputStyle = input.string("Area", "Plot style", options = ["Area", "Columns", "Histogram", "Stepline-di

// Initialize an `input plot_style` type variable based on user's selected `inputStyle`.
selectedPlotStyle = switch inputStyle
    "Area"            => plot.style_area
    "Columns"         => plot.style_columns
    "Histogram"       => plot.style_histogram
    "Stepline-diamond" => plot.style_stepline_diamond
    // A default block must be included in v6.
    => plot.style_line

plot(close, "Source plot", color.blue, 2, style = selectedPlotStyle)

//@variable Toggle for the style of the line plotted at price "100".
inputHundredStyle = input.bool(true, " Use crosses style for '100-line'")

hundredLineStyle = if inputHundredStyle
    plot.style_cross
else    //`else` block must be included in v6. Sets "line" style if `inputHundredStyle` is `false`.
    plot.style_line

// Plot the "100-line" using the `hundredLineStyle` style constant.
plot(100, "100-line", color.orange, 4, style = hundredLineStyle)

```

Constants

The following changes have been made to how Pine handles constant values.

Fractional division of constants

Dividing two integer “const” values can return a fractional value.

In v5, the result of the division of two “int” values is inconsistent. If *both* values are qualified as “const”, the script performs what is known as *integer division*, and discards any fractional remainder in the result, e.g., $5/2 = 2$. However, if *at least one* of the integers is qualified as “input”, “simple”, or “series”, the script *preserves* the fractional remainder in the division result: $5/2 = 2.5$.



Figure 499: image

```

//@version=5
indicator("`int` division demo")

// `float` division produces fractional remainder in both v5 and v6.

```

```

plot( 5.0 / 2.0, "`float` values", color.blue)

// `const int` division produces rounded-down result in v5. In v6, it produces a fractional remainder.
plot( 5 / 2,          "`const int` values",      color.orange)
plot( int(5) / int(2), "values wrapped `int()`", color.red)

// Wrapped `int()` division produces rounded down result in both v5 and v6.
plot( int(5 / 2), "result wrapped `int()`", color.green)

// Using `input int` type in division preserves the fractional remainder in both v5 and v6.
inputNum = input.int(2, "Division int", minval = 1)
plot( 5 / inputNum, "`input int` value", color.purple)

```

In v6, dividing two “int” values that are not evenly divisible *always* results in a number with a *fractional value*, regardless of the type and qualifier of the two arguments used. Therefore, the v6 division result is $5/2 = 2.5$, even if both values involved are “const int”.



Figure 500: image

Fix: If you need an “int” division result *without* a fractional value, wrap the division with the `int()` function to cast the *result* to “int”, which discards the fractional remainder. Alternatively, use `math.round()`, `math.floor()`, or `math.ceil()` to *round* the division result in a specific direction.

Mutable variables are always “series”

In Pine v5, some mutable variables are qualified as “series” values but are *erroneously* qualified as “const”. This behavior is incorrect and allows a programmer to pass them where “series” variables are usually not accepted.

For example, the `ta.ema()` function expects its `length` argument to be an integer qualified as “simple” or weaker (see the Qualifiers hierarchy). In the example script below the `seriesLen` variable is effectively a “series” type because its value changes between bars. In v5, `seriesLen` can be passed to `ta.ema()`. Although this does not raise an error, it does not work as expected, because only its *first* recorded value 1 is used as the `length` in the script:

```

//@version=5
indicator("`const` mutable variables demo")

// Variable is effectively of `series int` type.
var seriesLen = 0
seriesLen += 1

// `ta.ema()` only uses `length = 1` throughout execution, even as `seriesLen` changes.
plot(ta.ema(close, seriesLen))

```

In v6, `seriesLen` is correctly parsed as a “series int” type, and raises a compilation error if passed in place of the expected “simple int” argument for `length`.

Fix: Pass values of the expected qualified type to built-in functions. In our example, set the `length` argument to a “const int” value.

```
//@version=6
indicator("`const` mutable variables demo")

// Variable is now of `const int` type.
var seriesLen = 1

// `ta.ema()` uses `length = 1` throughout execution.
plot(ta.ema(close, seriesLen))
```

Color changes

The color values behind some of the `color.*` constants have changed in Pine v6 to better reflect the TradingView palette:

Constant name Pine v5 color Pine v6 color
`color.red` #FF5252 #F23645
`color.teal` #00897B #089981
`color.yellow` #FFEB3B #FDD835
Addition
the default text color for `label.new()` is now `color.white` in v6 (previously `color.black` in v5) to ensure that the text is more visible against the default `color.blue` label.



Figure 501: image

```
//@version=6
indicator("Default colors v6")

color defaultColor = switch
    bar_index == last_bar_index    => color.yellow
    bar_index == last_bar_index - 1 => color.green
    bar_index == last_bar_index - 2 => color.red
    => na
bgcolor(defaultColor)

if barstate.islastconfirmedhistory
    label.new(bar_index + 2, 0, "Default text color")
```

Strategies

Removal of when parameter

The `when` parameter for order creation functions was deprecated in v5 and is removed in v6. An order is created only if the `when` condition is `true`, which is its default value. This parameter affects the following functions: `strategy.entry()`, `strategy.order()`, `strategy.exit()`, `strategy.close()`, `strategy.close_all()`, `strategy.cancel()`, and `strategy.cancel_all()`.

The following example strategy shows the use of the `when` parameter, and works in v5 but not v6.

```
//@version=5
strategy("Conditional strategy", overlay=true)

longCondition = ta.crossover(ta.sma(close, 14), ta.sma(close, 28))
strategy.entry("My Long Entry Id", strategy.long, when = longCondition)

shortCondition = ta.crossunder(ta.sma(close, 14), ta.sma(close, 28))
strategy.entry("My Short Entry Id", strategy.short, when = shortCondition)
```

Fix: To trigger the order creation conditionally, use if statements instead.

```
//@version=6
strategy("Conditional strategy", overlay=true)

longCondition = ta.crossover(ta.sma(close, 14), ta.sma(close, 28))
if longCondition
    strategy.entry("My Long Entry Id", strategy.long)

shortCondition = ta.crossunder(ta.sma(close, 14), ta.sma(close, 28))
if shortCondition
    strategy.entry("My Short Entry Id", strategy.short)
```

Default margin percentage

The default margin percentage for strategies is now 100.

In v5, the default value of the `margin_long` and `margin_short` parameters is 0, which means that the strategy **does not check** its available funds before creating or managing orders. It can create orders that require *more* money than is available, and will **not** close short orders even when they lose more money than available to the strategy.

In *Pine v6*, the default margin percentage is 100. The strategy **does not open** entries that require more money than is available, and short orders are *margin called* if too much money is lost.

For example, we can see the difference in strategy behavior by running this simple strategy on the “ARM” symbol’s 4h chart using the v5 and v6 default margin values. When using Pine v5, there are no margin calls:

```
//@version=5
strategy("My strategy", overlay=true, default_qty_type = strategy.percent_of_equity, default_qty_value=100)
// v6 defaults: margin_long=100, margin_short=100
// v5 defaults: margin_long=0, margin_short=0

longCondition = ta.crossover(ta.sma(close, 14), ta.sma(close, 28))
if (longCondition)
    strategy.entry("My Long Entry Id", strategy.long)

shortCondition = ta.crossunder(ta.sma(close, 14), ta.sma(close, 28))
if (shortCondition)
    strategy.entry("My Short Entry Id", strategy.short)
```

However, if we adjust this script to `//@version=6` on the same chart, we see that it triggers 14 margin calls because of the new margin percentages:

Fix: To replicate the previous v5 behavior, set the `strategy()` function’s `margin_short` and `margin_long` arguments to 0.

Excess orders are trimmed

Strategy orders above the 9000 limit are trimmed (removed) in v6.

In v5, outside of Deep Backtesting, when a strategy creates more than 9000 orders, it raises a runtime error and halts any further calculations.

For example, this strategy script places several orders on each bar in the dataset. As a result, it can quickly surpass the 9000 order limit and trigger an error in Pine v5:

```
//@version=5
strategy("Strategy order limit demo", overlay=true, pyramiding=5)
```



Figure 502: image

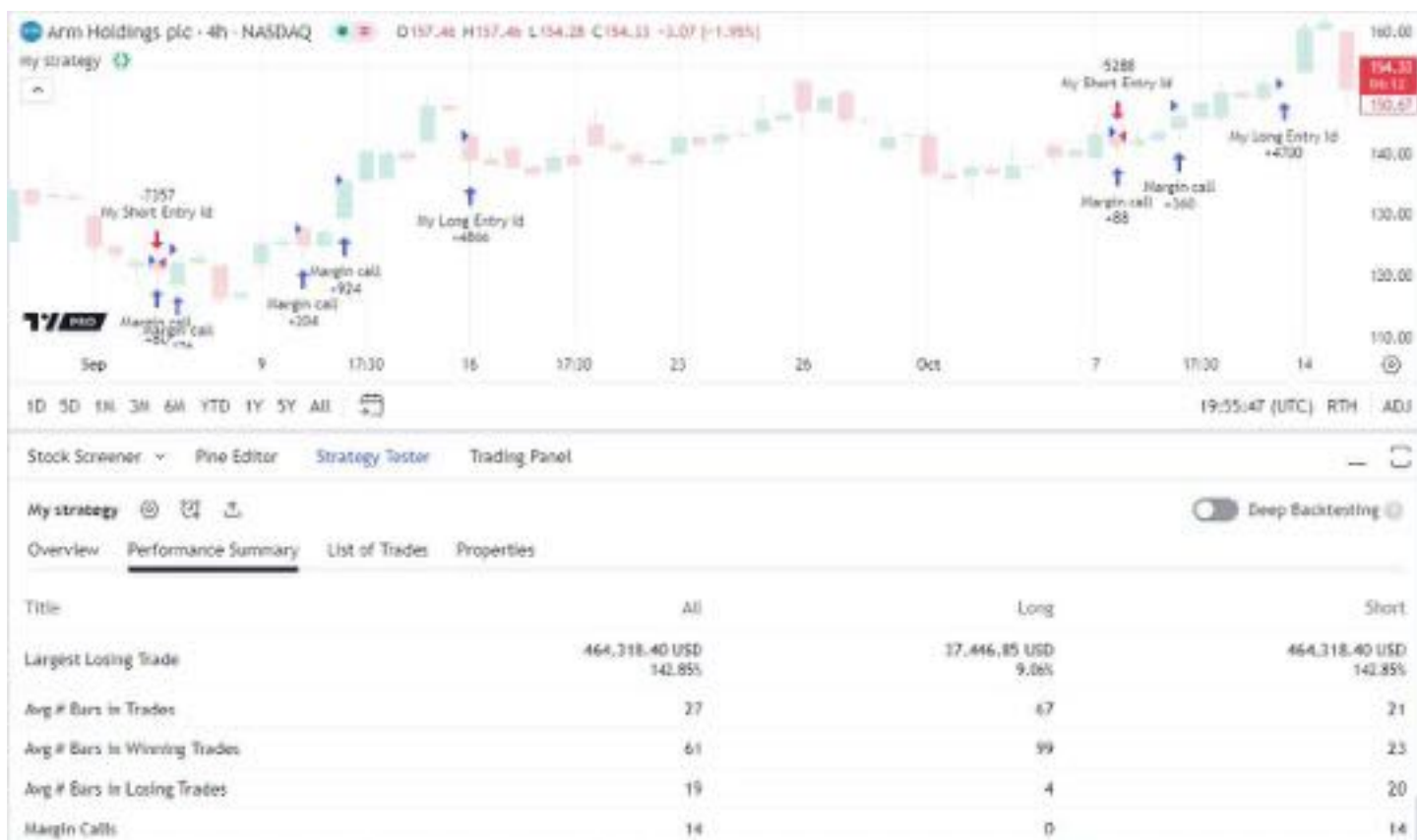


Figure 503: image

```
// Place several long orders on every even bar. This reaches the maximum orders limit in v5 and raises a runtime error
if bar_index % 2 == 0
    for i = 1 to 5
        strategy.entry("Entry " + str.tostring(i), strategy.long, qty = 5)
// Place short orders on every odd bar.
else
    strategy.entry("Short", strategy.short, qty = 25)
```

In v6, when the total number of orders exceeds 9000, the strategy does *not* halt. Instead, the orders are *trimmed* from the beginning until the limit is reached, meaning that the strategy only stores the information for the most recent orders.

Trimmed orders no longer show in the Strategy Tester, and referencing them using the `strategy.closedtrades.*` functions returns na. Use `strategy.closedtrades.first_index` to get the index of the first *non-trimmed* trade:

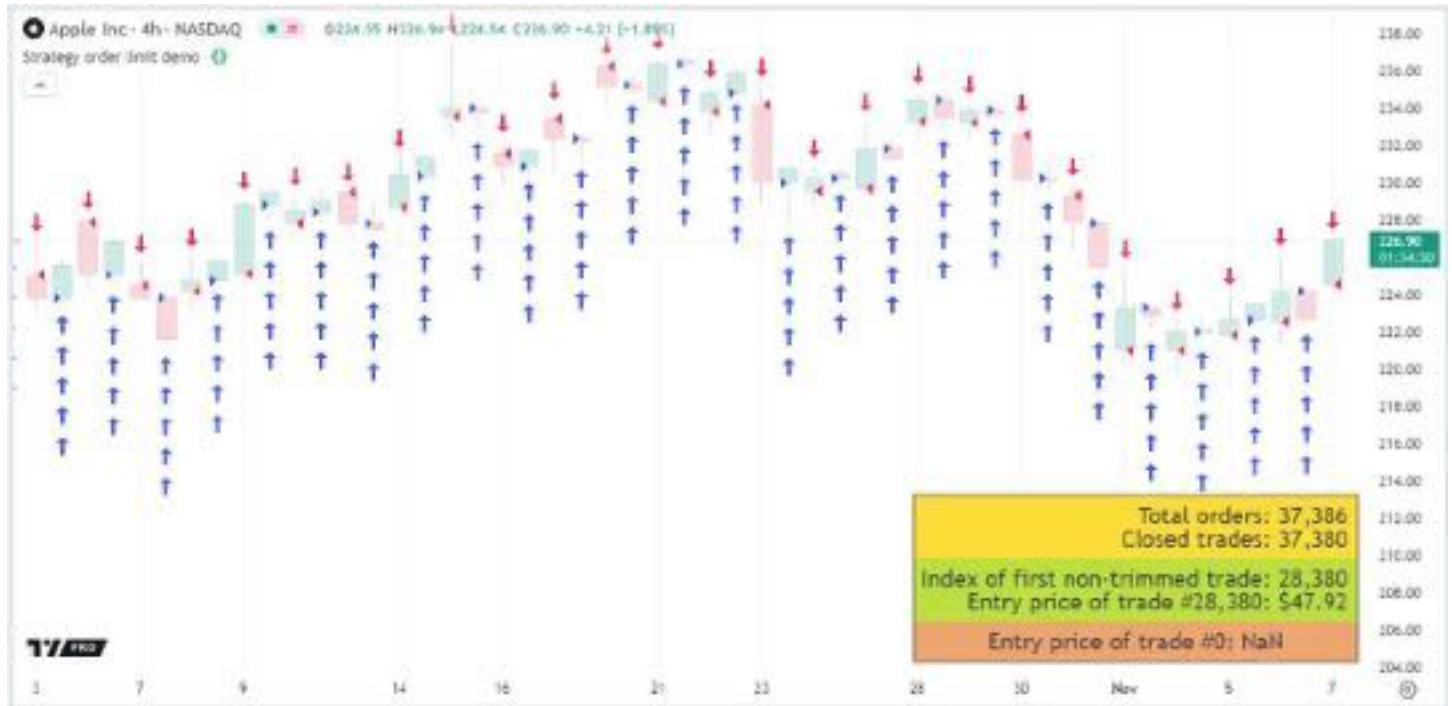


Figure 504: image

```
//@version=6
strategy("Strategy order limit demo", overlay=true, pyramiding=5)

//@variable Count of total orders placed.
var int totalOrders = 0

// Place several long orders on every even bar.
if bar_index % 2 == 0
    for i = 1 to 5
        strategy.entry("Entry " + str.tostring(i), strategy.long, qty = 5)
        totalOrders += 1
// Place short orders on every odd bar.
else
    strategy.entry("Short", strategy.short, qty = 25)
    totalOrders += 1

// Display total orders and index of first non-trimmed trade in a table cell on last bar.
if barstate.islastconfirmedhistory
    var table t = table.new(position.bottom_right, 1, 3, color.yellow, color.black, 1)
    // Display total orders and closed trades counts.
    string ordersText = "Total orders: " + str.tostring(totalOrders, "#,###")
```



```

+ "\n Closed trades: " + str.toString(strategy.closedtrades, "#,###")
t.cell(0, 0, ordersText, text_halign = text.align_right, text_size = size.large)

// Display the first non-trimmed trade index and its entry price.
string firstTradeIndex = str.toString(strategy.closedtrades.first_index, "#,###")
string firstTradePrice = str.toString(strategy.closedtrades.entry_price(strategy.closedtrades.first_index))
string firstTradeText = str.format("Index of first non-trimmed trade: {0}\nEntry price of trade #{0}: {1}")
t.cell(0, 1, firstTradeText, text_halign = text.align_right, text_size = size.large, bgcolor = #61dd5165)

// Trying to reference the trimmed trades (e.g., first closed trade) returns `na`.
if totalOrders > 9000
    string trimmedTradePrice = "Entry price of trade #0: " + str.toString(strategy.closedtrades.entry_price(0))
    t.cell(0, 2, trimmedTradePrice, text_size = size.large, bgcolor = #dd51c665)

```

strategy.exit() evaluates parameter pairs

The strategy.exit() function has three sets of *relative* and *absolute* parameters that define price levels for exit order calculations. The relative parameters **profit**, **loss**, and **trail_points** specify the take-profit and stop-loss levels and trailing stop activation level as *tick distances* from the entry price. In contrast, the absolute parameters **limit**, **stop**, and **trail_price** specify the exit and trail activation *prices* directly.

In Pine v5, a strategy.exit() call containing arguments for both the relative and absolute parameters that define a price level for the same exit order always prioritizes the *absolute* parameter and ignores the relative one. For instance, a call that includes a **limit** and **profit** argument consistently places take-profit orders at the **limit** value. It never places an exit order using the **profit** distance.

In Pine v6, if a strategy.exit() call contains arguments for related absolute and relative parameters, it evaluates *both* specified levels and uses the one that the market price is expected to *trigger first*.

The example below demonstrates how the behavior of this command differs for v5 and v6 scripts. This v5 script creates a long market order and an exit order bracket on each 28th bar. The strategy.exit() call contains arguments for the relative parameters that determine take-profit and stop-loss levels (**profit** and **loss**), and it includes arguments for the absolute parameters (**limit** and **stop**). The **profit** and **loss** arguments are both 0, which would result in consistent exits at the entry price if the command used them. However, the command never uses these values to determine the exit order levels because the **limit** and **stop** parameters *always* take precedence when they have specified values:



Figure 505: image

```

//@version=5
strategy("`strategy.exit()` with parameter pairs demo", overlay = true, margin_long = 100, margin_short = 100)

//@variable The 14-bar Average True Range.
float atr = ta.atr(14)

```



```

if bar_index % 28 == 0
    strategy.entry("Buy", strategy.long)
    strategy.exit("Exit", "Buy", profit = 0, limit = close + 2.0 * atr, loss = 0, stop = close - 2.0 * atr)

```

If we convert the script to Pine v6, its behavior changes. Instead of prioritizing the absolute `limit` and `stop` parameters exclusively, the `strategy.exit()` command always prioritizes the price levels that will trigger exits *first*. In this example, the market price reaches the `limit` or `stop` value *after* the `profit` and `loss` distance of 0 ticks. Consequently, the command ignores the `limit` and `stop` values and places its exit orders at the entry price, which causes the strategy to exit each trade immediately after opening it:



Figure 506: image

```

//@version=6
strategy("`strategy.exit()` with parameter pairs demo", overlay = true, margin_long = 100, margin_short = 100)

//@variable The 14-bar Average True Range.
float atr = ta.atr(14)

if bar_index % 28 == 0
    strategy.entry("Buy", strategy.long)
    strategy.exit("Exit", "Buy", profit = 0, limit = close + 2.0 * atr, loss = 0, stop = close - 2.0 * atr)

```

History-referencing operator

Pine v6 contains several changes to referencing the history of values.

No history for literal values

The history-referencing operator `[]` can no longer be used with literal values or built-in constants.

In v5, the history-referencing operator `[]` can be used with built-in constants, such as `true` and `color.red`, and with *literals*, which are raw values used directly in a script that are not stored as variables, such as `6` or `"myString"`, etc.

However, referencing the history of a literal is usually redundant, because by definition every literal represents a fixed value. The only exception where the returned historic value may vary is if the historical offset points to a *non-existent* bar, in which case referencing the historic literal value returns `na`.

```

//@version=5
indicator("History-referencing on literals demo")

// These lines all use history-referencing on literals, which works in v5, but is not really useful to do here
plot(6[1], "6[1]", linewidth = 3)
bgcolor(true[10] ? color.orange[3] : na)

```

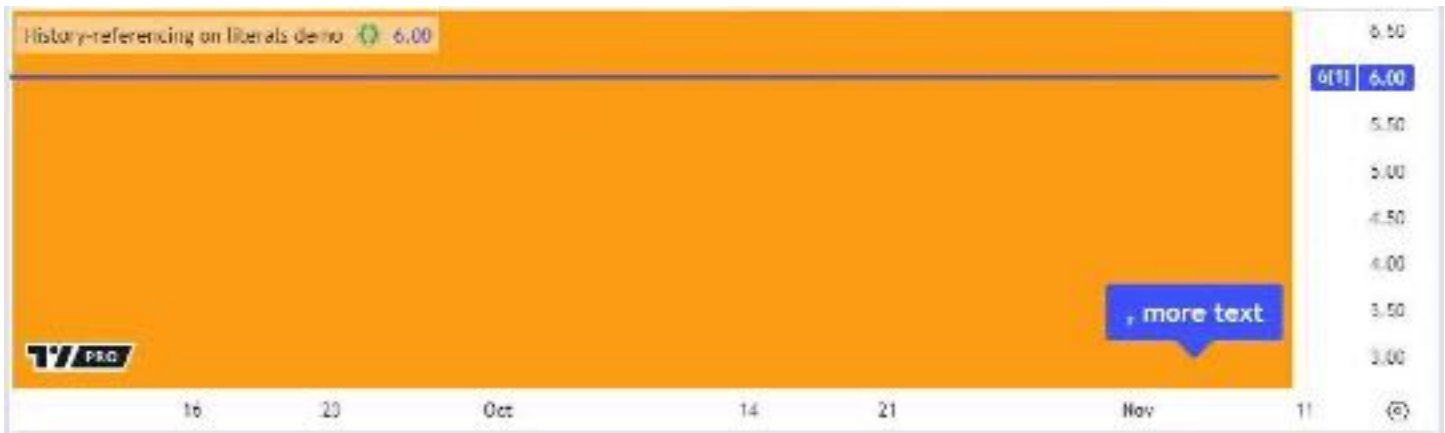


Figure 507: image

```
if barstate.islastconfirmedhistory
    // Since "string literal" is only defined in the last bar scope, history-referencing here returns `na`.
    labelText = "string literal"[20]
    label.new(bar_index - 3, 3, labelText + ", more text", textcolor = color.white, size = size.large)
    // Label output will only show ", more text" in v5, since `labelText` is `na`.
```

// In v6, using any history-referencing on literals or built-in constants causes an error.

In *Pine v6*, you can **no longer** use the history-referencing operator `[]` on literals or built-in constants. Trying to do so triggers a compilation error.

Fix: Remove any `[]` operators used with literals or constants.

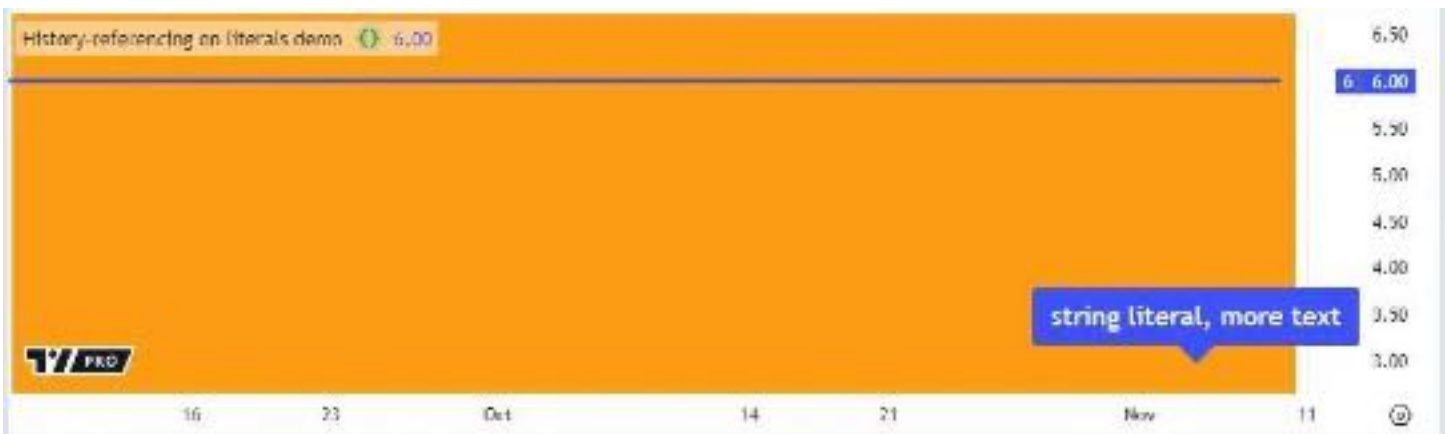


Figure 508: image

```
//@version=6
indicator("History-referencing on literals demo")

// We no longer use history-referencing on literals in v6.
plot(6, "6", linewidth = 3)
bgcolor(true ? color.orange : na)
if barstate.islastconfirmedhistory
    labelText = "string literal"
    label.new(bar_index - 3, 3, labelText + ", more text", textcolor = color.white, size = size.large)
    // Label output shows "string literal, more text" in v6, since `labelText` is defined without history-referencing.
```

History of UDT fields

The history-referencing operator `[]` can no longer be used directly on fields of user-defined types.

In v5, you can use the history-referencing operator `[]` on the *fields* of *user-defined types*. While this does not cause any compilation errors, the behavior itself is erroneous.

For example, the script below draws an arrow label on each bar and displays its percentage increase/decrease. The label style, color, and text are set based on a bar's direction (`close > open`). The script defines a `UDTLblSettings` to initialize an object on each bar that stores these settings. On the last bar, it draws a table cell that displays the arrow direction and percentage difference from 10 bars back. In v5, we could use the history-referencing operator `[]` on the required `LblSettings` fields directly:



Figure 509: image

```
//@version=5
indicator("UDT history-referencing demo", overlay = true)

//@type A custom type to hold bar's `label` settings based on bar's direction.
//Includes bar direction, label style and color, and "string" percentage difference between bar's `open` and `close`
type LblSettings
    bool isUp = false
    string lblStyle
    color lblColor
    string diff

//@variable A `LblSettings` instance declared on every bar.
LblSettings infoObject = LblSettings.new()

// Set the `LblSettings` object fields based on current bar's direction and price information.
infoObject.isUp := close > open
infoObject.lblStyle := infoObject.isUp ? label.style_arrowup : label.style_arrowdown
infoObject.lblColor := infoObject.isUp ? color.green : color.red
infoObject.diff := str.toString((close - open) / open * 100, "#.##") + "%"

// Display a new `label` on each bar using its `infoObject` settings.
label.new(bar_index, high, infoObject.diff, style = infoObject.lblStyle,
    color = infoObject.lblColor, textcolor = infoObject.lblColor)

// Highlight the bar that is 10 bars back from the last bar.
bgcolor(bar_index == last_bar_index - 10 ? color.yellow : na)

// On last bar, output table cell to display `LblSettings` object's `lblStyle` and `diff` fields from 10 bars back
if barstate.islast
    var table t = table.new(position.bottom_right, 1, 1, color.yellow)

    // In v5, you could use history-referencing operator `[]` on UDT fields directly.
    //@variable Text displayed in table cell. Set based on the `lblStyle` and `diff` fields from 10 bars back.
    string txt = "10 bars back: Arrow was "
```

```

    + (infoObject.lblStyle[10] == label.style_arrowdown ? "DOWN" : "UP")
    + " by " + infoObject.diff[10]
t.cell(0, 0, txt, text_size = size.large)

```

In *Pine v6*, you can **no longer** use the history-referencing operator `[]` on the field of a user-defined type directly.

Fix: Use the history-referencing operator on the UDT *object* instead, then retrieve the field of the historic object. To do so, use the syntax `(myObject[10]).field` - ensure the object's historical reference is wrapped in *parentheses*, otherwise it is invalid. Alternatively, assign the UDT *field* to a *variable* first, and then use the history-referencing operator `[]` on the variable to access its historic value.

```

// Reference history of object, then retrieve field of historic object.
[fieldType] historicFieldValue = (myObject[10]).field

```

```

// Alternative: Assign field to variable, then reference history of variable to get historic field value.
[fieldType] newVariable = myObject.field
[fieldType] historicFieldValue = newVariable[10]

```

Therefore, we can adjust the v5 code to access a historic instance of our `infoObject` on the last bar, wrapped in parentheses. Then, we retrieve our desired field values from the historic object (`infoObject[10]`) to display the arrow direction and percentage difference from 10 bars back:

```

//@version=6
indicator("UDT history-referencing demo", overlay = true)

```

```

//@type A custom type to hold bar's `label` settings based on bar's direction.

```

```

//Includes bar direction, label style and color, and "string" percentage difference between bar's `open` and `close`
type LblSettings
    bool isUp = false
    string lblStyle
    color lblColor
    string diff

```

```

//@variable A `LblSettings` instance declared on every bar.
LblSettings infoObject = LblSettings.new()

```

```

// Set the `LblSettings` object fields based on current bar's direction and price information.

```

```

infoObject.isUp := close > open
infoObject.lblStyle := infoObject.isUp ? label.style_arrowup : label.style_arrowdown
infoObject.lblColor := infoObject.isUp ? color.green : color.red
infoObject.diff := str.toString((close - open) / open * 100, "#.##") + "%"

```

```

// Display a new `label` on each bar using its `infoObject` settings.

```

```

label.new(bar_index, high, infoObject.diff, style = infoObject.lblStyle,
    color = infoObject.lblColor, textcolor = infoObject.lblColor)

```

```

// Highlight the bar that is 10 bars back from the last bar.

```

```

bgcolor(bar_index == last_bar_index - 10 ? color.yellow : na)

```

```

// On last bar, output table cell to display `LblSettings` object's `lblStyle` and `diff` fields from 10 bars back
if barstate.islast

```

```

    var table t = table.new(position.bottom_right, 1, 1, color.yellow)

```

```

// In v6, cannot use `[]` on UDT fields (e.g., `infoObject.lblStyle[10]` is invalid).

```

```

// Instead, Use `[]` to reference UDT object's history, wrapped in parentheses, then retrieve its fields.

```

```

//@variable The `lblStyle` field value from 10 bars back. Is either `label.style_arrowdown` or `label.style_arrowup`
string historicArrowStyle = (infoObject[10]).lblStyle

```

```

//@variable The `diff` field value (percentage difference between `close` and `open`) from 10 bars back.
string historicPercentDifference = (infoObject[10]).diff

```

```

//@variable Text displayed in table cell. Set based on the `lblStyle` and `diff` fields from 10 bars back.
string txt = "10 bars back: Arrow was "

```

```

+ (historicArrowStyle == label.style_arrowdown ? "DOWN" : "UP")
+ " by " + historicPercentDifference
t.cell(0, 0, txt, text_size = size.large)

```

Timeframes must include a multiplier

The `timeframe.period` variable holds a “string” that represents the chart’s timeframe, typically consisting of a *quantity* (multiplier) and *unit*.

In v5, the `timeframe.period` variable does *not* include a quantity when the chart timeframe has a multiplier of 1. Instead, the string consists of only the timeframe unit, e.g., "D", "W", "M". This is inconsistent with the timeframe strings for these same units at higher intervals, e.g., "2D", "3M".

To simplify the timeframe format in v6, the `timeframe.period` variable now *always* includes a multiplier with its timeframe unit. So, "D" becomes "1D", "W" becomes "1W", and "M" becomes "1M".

This change might affect the behavior of older scripts that used `==` to compare the value of `timeframe.period` with the “string” representation of a timeframe directly (e.g., `timeframe.period == "D"`).

To show the difference between the v5 and v6 `timeframe.period` variables, we ran the script below on a daily chart (1D) for each Pine version. The script displays the `timeframe.period` string in a table, and compares the variable’s value with the “string” literals "D" and "1D":



Figure 510: image

```

//@version=6
indicator("`timeframe.period` multiplier - v6")

//@function Compares `timeframe.period` to passed `timeframeString` and outputs result in selected table cell.
compareTF(string timeframeString, table t, int row) =>
    bool tfComparison = timeframe.period == timeframeString
    // Format table cell text and determine cell color based on comparison result.
    string displayText = "`timeframe.period` == '" + timeframeString + "'? " + str.toString(tfComparison)
    color cellColor = tfComparison ? color.rgb(76, 175, 79, 40) : color.rgb(255, 82, 82, 40)
    // Display `tfComparison` result.
    t.cell(0, row, displayText, bgcolor = cellColor, text_halign = text.align_left, text_size = size.large)

// Display the chart's timeframe information in a table.

```

```

if barstate.islastconfirmedhistory
    //@variable Table displaying chart timeframe information.
    var table t = table.new(position.middle_center, 1, 3, #FFEB3B99, border_color = color.black, border_width :

    //@variable The text to display in the table, consisting of the chart timeframe and multiplier.
    string tfInfo = "Chart timeframe: " + timeframe.period
        + "\n Chart multiplier: " + str.toString(timeframe.multiplier)

    t.cell(0, 0, tfInfo, text_halign = text.align_left, text_size = size.large)

    // Compare the current chart timeframe (daily chart) to timeframe strings with and without a multiplier.
    compareTF("D", t, 1)
    compareTF("1D", t, 2)

```

Fix: In general, ensure that all timeframe strings include a multiplier. In this example, change the timeframe comparison “string” (`timeframe.period == "D"`) to ensure the “string” literal includes a multiplier (`timeframe.period == "1D"`).

Lazy evaluation of conditions

The `and` and `or` conditions are now evaluated *lazily* rather than *strictly*.

An `and` condition is `true` if *all* of its arguments are `true`, which means that if the *first* argument is `false`, we can deduce that the whole condition is `false`, regardless of the value of the second argument. Conversely, an `or` condition is `true` when *at least one* of the arguments is `true`, so if the *first* argument is already `true`, then the whole condition is `true`, regardless of the second argument’s state.

Pine v5 evaluates all bool expressions except for the `?:` ternary operator *strictly*, meaning the *second* part of a conditional expression is *always* evaluated, regardless of the value of the first argument.

Lazy evaluation can have consequences for script calculation. In the example below, we assign a value of `true` to the `signal` variable *only* when `close > open` and `ta.rsi(close, 14) > 50`. The `ta.rsi()` function must be executed on every bar in order to calculate its result correctly. In v5, the function *is* called on every bar, even when `close > open` is *not* `true`, due to the strict bool evaluation, and therefore the function calculates correctly.

```

//@version=5
indicator("Evaluation test v5")

//@variable A signal flag. Is `true` if two conditions `close > open` and `ta.rsi(close, 14) > 50` are both `true`.
bool signal = false

if close > open and ta.rsi(close, 14) > 50
    signal := true

// Highlight background on bars where `signal` is `true`.
bgcolor(signal ? color.new(color.green, 90) : na)

```

In v6, bool expressions are evaluated *lazily*, which means the expression *stops evaluating* once it determines the overall condition’s result, even if there are other arguments remaining in the expression.

If we convert the script above to v6, we see that the plotted signals *differ* between the two scripts. This variation occurs because of the lazy bool evaluation – since an `and` condition is only `true` if *all* its arguments are `true`, when `close > open` is `false`, the `and` condition is *definitely* `false` regardless of the second argument `ta.rsi(close, 14) > 50`. Consequently, the `ta.rsi()` call is *not* evaluated on every bar, which interferes with the internal history that the RSI function stores for its calculation and results in incorrect values:

Fix: Ensure that the script evaluates all functions that rely on previous values on each bar. For example, extract calls that rely on historical context to the *global scope* and assign them to a variable. Then, reference that *variable* in the `and` and `or` conditions.

Note that you can and should take advantage of the lazy bool evaluation to create smarter, more concise code.

For example, the script below calls `array.first()` on an array that is occasionally empty (on bars where `close > open` is `false`). In *Pine v5*, calling `array.first()` on an empty array results in a runtime error, so you must keep the two if-conditions that check the array size and first element separated in *different scopes* to avoid the error. However, in *Pine v6*, you can have the



Figure 511: image

two conditions in the **same*scope** without error because the `and` condition's lazy evaluation ensures that `array.first()` will only be called if `array.size() != 0` is true first:

```
//@version=5
indicator("Lazy evaluation error showcase")

array<bool> myArray = array.new<bool>()

if close > open
    myArray.push(true)

// Causes a runtime error in v5 when trying to call `array.first()` on an empty array.
// Works in v6 because `array.first()` is only called if the array is not empty.
if myArray.size() != 0 and myArray.first()
    label.new(bar_index, high, "Test")

// A correct approach for v5: `array.first()` is only called when we're sure the array is not empty.
if myArray.size() != 0
    if myArray.first()
        label.new(bar_index, high, "Test")
```

Cannot repeat parameters

In v5, you can specify the same parameter in a function more than once. However, doing so raises a *compiler warning*, and only the *first* value will be used.

```
// In v5, compiles but raises warning. Only uses first value, so plot color will be `blue`.
plot(close, "Close", color = color.blue, linewidth = 2, color = color.red)
```

In v6, you can specify a parameter only *once*, and doing otherwise will result in a *compilation error*.

Fix: Remove the duplicate parameters.

```
// In v6, script will not compile if parameter is specified more than once.
plot(close, "Close", color = color.blue, linewidth = 2)
```

No series offset values

The `offset` parameter can no longer accept “series” values

In Pine v5, the `offset` parameter in `plot()` and similar functions can accept “*series int*” arguments. However, passing a

“series” argument raises a compiler warning, and the behavior is *incorrect*: only the *last* calculated offset is used on the whole chart, regardless of its previous values.

For example, this script uses `bar_index / 2` as a “series” `offset` argument while plotting the high points of each bar’s body. Because the `plot()` function uses only the *lastoffset* value, the plot appears offset by 10 bars here for the *entire* “GOOGL” 12M chart (since the chart’s last `bar_index` is 20 here):

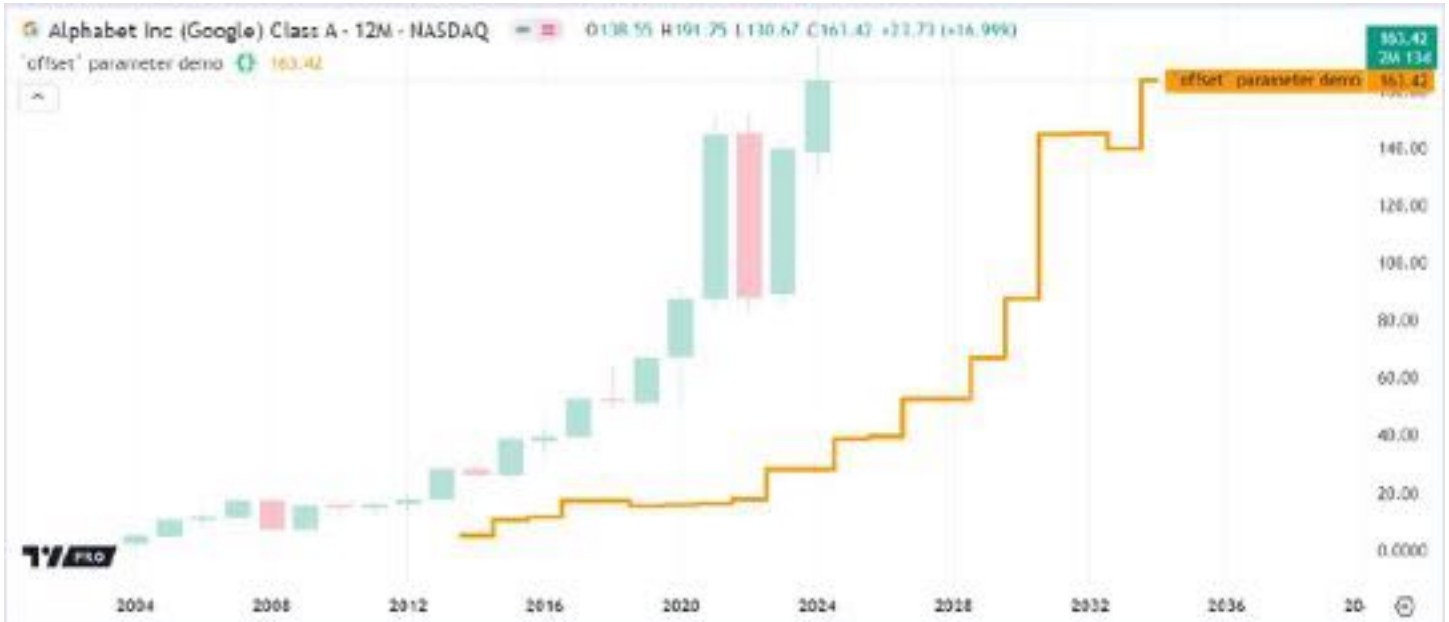


Figure 512: image

```
//@version=5
indicator("`offset` parameter demo", overlay = true)

//@variable `series` int` value. Used as `offset` parameter value in `plot()`.
int seriesOffset = bar_index / 2
// In v5, a `series` type `offset` value is valid, but only the last calculated value is used.
plot(math.max(close, open), "", color.orange, 4, plot.style_stepline, offset = seriesOffset)
```

In v6, the `offset` parameter accepts an argument qualified as “*simple*” or weaker. The value used must be the same on every bar.

Remember that the Pine Script qualifiers hierarchy means that a parameter expecting a “*simple*” value can also accept values qualified as “*input*” or “*const*”. However, passing a “*series*” argument triggers a compilation error.

Fix: Change any “*series*” values passed to `offset` to “*simple*” values.

Minimum linewidth is 1

In v5, the `linewidth` parameter of the `plot()` and `hline()` functions can accept a value smaller than 1, although the width on the chart will still appear as 1 for these drawings:

```
//@version=5
indicator("Linewidth demo")

//@variable User-input width for a line. Default value set to 0, with no minimum limit.
int userWidth = input.int(0, "Linewidth")

// Valid in v5, but line widths on chart all appear as `linewidth=1`. Not valid in v6.
plot(close, "LW 1", linewidth = 1)
plot(close + 5, "LW 0", linewidth = userWidth)
plot(close + 10, "LW -5", linewidth = -5)
hline(240, "hline", color.maroon, linewidth = -3)
```

In v6, the `linewidth` argument **must** be 1 or greater. Passing a smaller value causes a compilation error.

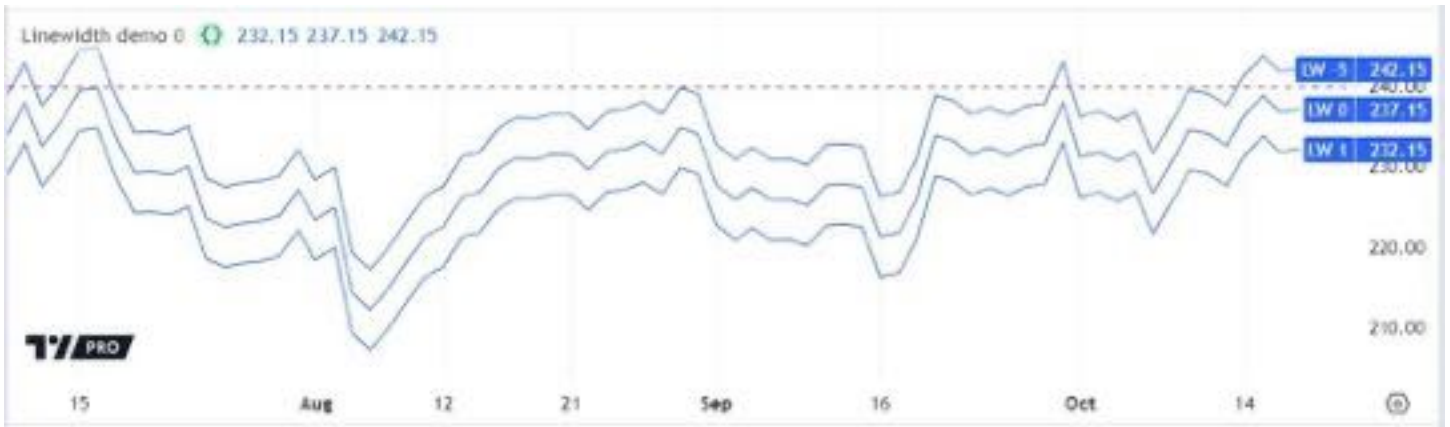


Figure 513: image

Fix: Replace any `linewidth` argument that is smaller than 1 to ensure all width values are *at least* 1.

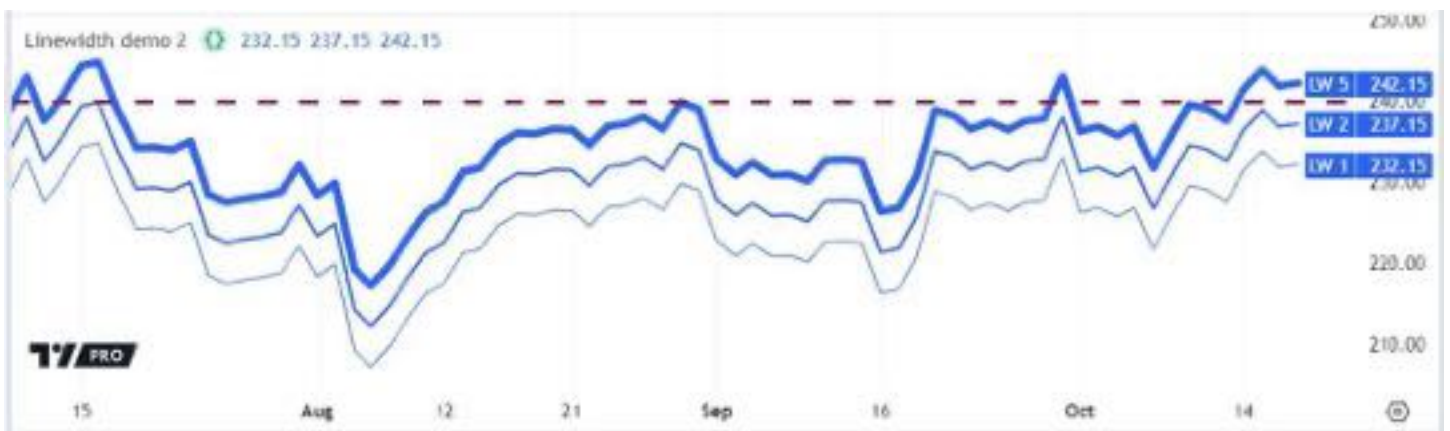


Figure 514: image

```
//@version=6
indicator("Linewidth demo")

//@variable User-input width for a line. Default value set to 2, with minimum value set to 1.
int userWidth = input.int(2, "Linewidth", minval = 1)

// In v6, all line widths must be at least 1 or greater.
plot(close,      "LW 1",   linewidth = 1)
plot(close + 5,  "LW 2",   linewidth = userWidth)
plot(close + 10, "LW 5",   linewidth = 5)
hline(240, "hline", color.maroon, linewidth = 3)
```

Negative indices in arrays

Some array functions now accept negative indices.

In v5, array functions that require an element's *index* always expect a value *greater than or equal to 0*. Therefore, functions like `array.get()`, `array.insert()`, `array.set()`, and `array.remove()` raise a runtime error if a negative index is passed.

In v6, `array.get()`, `array.insert()`, `array.set()`, and `array.remove()` allow you to pass a *negative index* to request items from the *end* of the array. For example, `-1` refers to the last item in the array, `-2` refers to the second to last, and so forth.

```
//@variable Array of "int" numbers from 1 to 5.
array<int> countingArray = array.from(1, 2, 3, 4, 5)
// Array indexing starts from 0 to retrieve first element in both v5 and v6.
int firstValue = countingArray.get(0)      //Returns "1"
```

```
// In v6, can retrieve last array element using negative index. This index is invalid in v5.
int lastValue = countingArray.get(-1) //Returns "5"

// Other `array.*()` functions also accept negative indexing in v6. These lines raise runtime errors in v5.
countingArray.set(-2, 10) // Updated array: [1, 2, 3, 10, 5]
countingArray.remove(-5) // Updated array: [2, 3, 10, 5]
countingArray.insert(-1, 20) // Updated array: [2, 3, 10, 20, 5]
```

As a result, scripts that return a runtime error for using negative indices in v5 can be executed without error in v6.

However, if you create or update a script in v6, you must be aware of this new behavior to ensure that the script does not behave unexpectedly.

Keep in mind that negative indexing is still bound by the size of the array. Therefore, an array of 5 elements only accepts indexing from 0 to 4 (first to last element) or -1 to -5 (last to first element). Any other indices are out of bounds and raise a runtime error:

```
//@variable Array of "int" numbers from 1 to 5.
array<int> countingArray = array.from(1, 2, 3, 4, 5)

// Trying to index negatively beyond the size of the array causes a runtime error.
countingArray.remove(-6)
```

The **transp** parameter is removed

In Pine v4 and earlier, `plot()` and similar functions had a **transp** parameter that specified the transparency of the resulting plot.

Pine v5 deprecated and hid the **transp** parameter, because it is not fully compatible with the color system that Pine currently uses. Using both transparency settings together can result in unexpected behavior, as the **transp** parameter can get overwritten by the transparency of the color passed to the function. In v5, using the `color.new()` function and not the **transp** parameter avoids any such conflicts.

Pine v6 removes the **transp** parameter completely from the following functions: `bgcolor()`, `fill()`, `plot()`, `plotarrow()`, `plotchar()`, and `plotshape()`. Whenever the converter encounters a **transp** argument, it removes the argument from the converted v6 script.

Fix: To set the transparency of a drawn plot, use the `color.new()` function. Pass the color value as the first argument, and the desired transparency value as the second.

For example, this v5 code uses the hidden **transp** parameter to set the color of the plot to 80 transparency:

```
//@version=5
indicator("Transparency demo v5")

color myColor = close > open ? color.green : color.red
plot(close, color = myColor, transp = 80)
```

In Pine v6, the same result can be achieved using `color.new()`:

```
//@version=6
indicator("Transparency demo v6")

color myColor = close > open ? color.green : color.red
plot(close, color = color.new(myColor, 80))
```

If you need to preserve the color inputs in the “Settings/Style” menu, you must ensure that every color that gets passed to every `color.new()` call is qualified as either “const” or “input”. If at least one of these color values is calculated dynamically (like the code above), the color selector does not appear in the settings:

You can learn more about why this happens and how to avoid it [here](#).

Dynamic for loop boundaries

A for loop is a *count-controlled* loop that executes successive iterations of its local block based on a counter variable. The counter starts with an *initial value* (`from_num`) and increases or decreases by a fixed amount after every iteration until it

Color test



Style Visibility



Plot



Figure 515: image

reaches the specified *final value* (`to_num`).

In Pine v5, a for loop statement establishes its final counter value strictly *before* starting its iterations. If the script changes the value of a variable or expression used as the loop's `to_num` argument during the loop's iterations, those changes do **not** affect the counter's boundaries. This behavior differs from while and for...in loops, which have control criteria that can change across iterations.

In Pine v6, all for loops *dynamically* evaluate their stopping criteria before **each iteration**. Variables and expressions used as the `to_num` argument that depend on values or objects modified in the loop's scope can *update* the counter variable's boundaries across iterations. This behavior enables scripts to use a for loop for iterative tasks where the exact iteration boundaries are unknown before the loop starts.

Because for loop boundaries can be *dynamic* in Pine v6, a v5 script using this loop structure with a mutated variable or dynamic expression such as `array.size(id) - 1` as the `to_num` argument can behave differently after conversion to v6.

Fix: If a for loop requires only **one** evaluation of an expression used as the `to_num` argument across iterations, assign the expression to a variable *outside* the loop's scope, then use that variable as the `to_num` argument instead.

The following v5 example uses two user-defined methods to manage the elements in an array. The script calls the `dequeue()` method before a for loop to remove the first element from the `data` array. Then, it calls the `queue()` method inside the loop statement to add the current bar's close into the array and return the array's size for the loop's end boundary. Within the loop's scope, the script increments the `belowCount` variable by one for each element with a value below the current bar's ohlc4 value:

```
//@version=5
indicator("v5 vs v6 `to_num` demo")

//@variable The size of the `data` array.
int sizeInput = input.int(20, "Size", 1)

//@function Appends a new `value` to `this` array and returns the array's size.
method queue(array<float> this, float value) =>
    this.push(value)
    this.size()

//@function Removes the first value from `this` array if the size equals the `sizeInput`.
method dequeue(array<float> this) =>
    if this.size() == sizeInput
        this.shift()

//@variable An array that holds `sizeInput` recent `close` prices.
```



Figure 516: image

```
var array<float> data = array.new<float>()

//@variable The number of elements in the `data` array that are below the current bar's `ohlc4`.
int belowCount = 0

// Remove the oldest element from the `data` array when its size reaches the `sizeInput`.
data.dequeue()

// Push the bar's `close` into `data` and loop from zero to one less than the array's size.
// In v5, the loop evaluates the `data.queue()` call *once*, meaning the final counter value is fixed across i
// In v6, it evaluates the call before *every* iteration, causing the array and loop boundary to expand indefi
for i = 0 to data.queue(close) - 1
    // Add 1 to `belowCount` when the `data` element at index `i` is less than the `ohlc4` value.
    if data.get(i) < ohlc4
        belowCount += 1

// Plot the `belowCount` in a separate pane.
plot(belowCount, "Closes below OHLC4", color.blue, 3)
```

In v5, the above loop statement evaluates `data.queue(close) - 1` only **once**, before it starts the first iteration. It does *not* execute that expression again across iterations. As such, each script execution queues exactly one new value into the `data` array, and the number of times the loop executes its local code *does not change* while the loop runs.

However, the script does not work after conversion to v6, because the for loop evaluates `data.queue(close) - 1` before **every iteration**. Each evaluation of the expression adds a *new element* to the `data` array and *increases* the `to_num` boundary, causing the loop to iterate indefinitely until the script raises a runtime error:

We can fix the script's behavior by assigning the expression's initial result to a variable outside the loop's scope and using that variable in the for loop statement. This change prevents the expression from modifying the array's size or altering the loop's end boundary between iterations:

```
//@version=6
indicator("v5 to v6 fixed `to_num` demo")

//@variable The size of the `data` array.
int sizeInput = input.int(20, "Size", 1)

//@function Appends a new `value` to `this` array and returns the array's size.
method queue(array<float> this, float value) =>
```



Figure 517: image

```

this.push(value)
this.size()

//@function Removes the first value from `this` array if the size equals the `sizeInput`.
method dequeue(array<float> this) =>
    if this.size() == sizeInput
        this.shift()

//@variable An array that holds `sizeInput` recent `close` prices.
var array<float> data = array.new<float>()

//@variable The number of elements in the `data` array that are below the current bar's `ohlc4`.
int belowCount = 0

// Remove the oldest element from the `data` array when its size reaches the `sizeInput`.
data.dequeue()

// Push the `close` into the `data` array and assign one less than the array's size to a variable,
// ensuring only one evaluation per script execution.
int lastCount = data.queue(close) - 1

// Use `lastCount` as the `to_num` in the `for` loop statement.
// This change prevents the array and loop from expanding indefinitely,
// because the value is calculated *outside* the loop's scope.
for i = 0 to lastCount
    // Add 1 to `belowCount` when the `data` element at index `i` is less than the `ohlc4` value.
    if data.get(i) < ohlc4
        belowCount += 1

// Plot the `belowCount` in a separate pane.
plot(belowCount, "Closes below OHLC4", color.blue, 3)

```

[Previous

Overview](#overview)[Next

To Pine Script™ version 5](#to-pine-version-5) User Manual/Migration guides/To Pine Script™ version 5

To Pine Script™ version 5

Introduction

This guide documents the **changes** made to Pine Script from v4 to v5. It will guide you in the adaptation of existing Pine scripts to Pine Script v5. See our Release notes for a list of the **new** features in Pine Script v5.

The most frequent adaptations required to convert older scripts to v5 are:

- Changing `study()` for `indicator()` (the function's signature has not changed).
- Renaming built-in function calls to include their new namespace (e.g., `highest()` in v4 becomes `ta.highest()` in v5).
- Restructuring inputs to use the more specialized `input.*()` functions.
- Eliminating uses of the deprecated `transp` parameter by using `color.new()` to simultaneously define color and transparency for use with the `color` parameter.
- If you used the `resolution` and `resolution_gaps` parameters in v4's `study()`, they will require changing to `timeframe` and `timeframe_gaps` in v5's `indicator()`.

v4 to v5 converter

The Pine Editor can automatically convert v4 indicators and strategies to v5. The Pine converter is described in the Overview page.

Not all scripts can be automatically converted from v4 to v5. If you want to convert the script manually or if your indicator returns a compilation error after conversion, use the following sections to determine how to complete the conversion. A list of some errors you can encounter during the automatic conversion and how to fix them can be found in the Common script conversion errors section of this guide.

Renamed functions and variables

For clarity and consistency, many built-in functions and variables were renamed in v5. The inclusion of v4 function names in a new namespace is the cause of most changes. For example, the `sma()` function in v4 is moved to the `ta.` namespace in v5: `ta.sma()`. Remembering the new namespaces is not necessary; if you type the older name of a function without its namespace in the Editor and press the 'Auto-complete' hotkey (`Ctrl + Space`, or `Cmd` on MacOS), a popup showing matching suggestions appears:

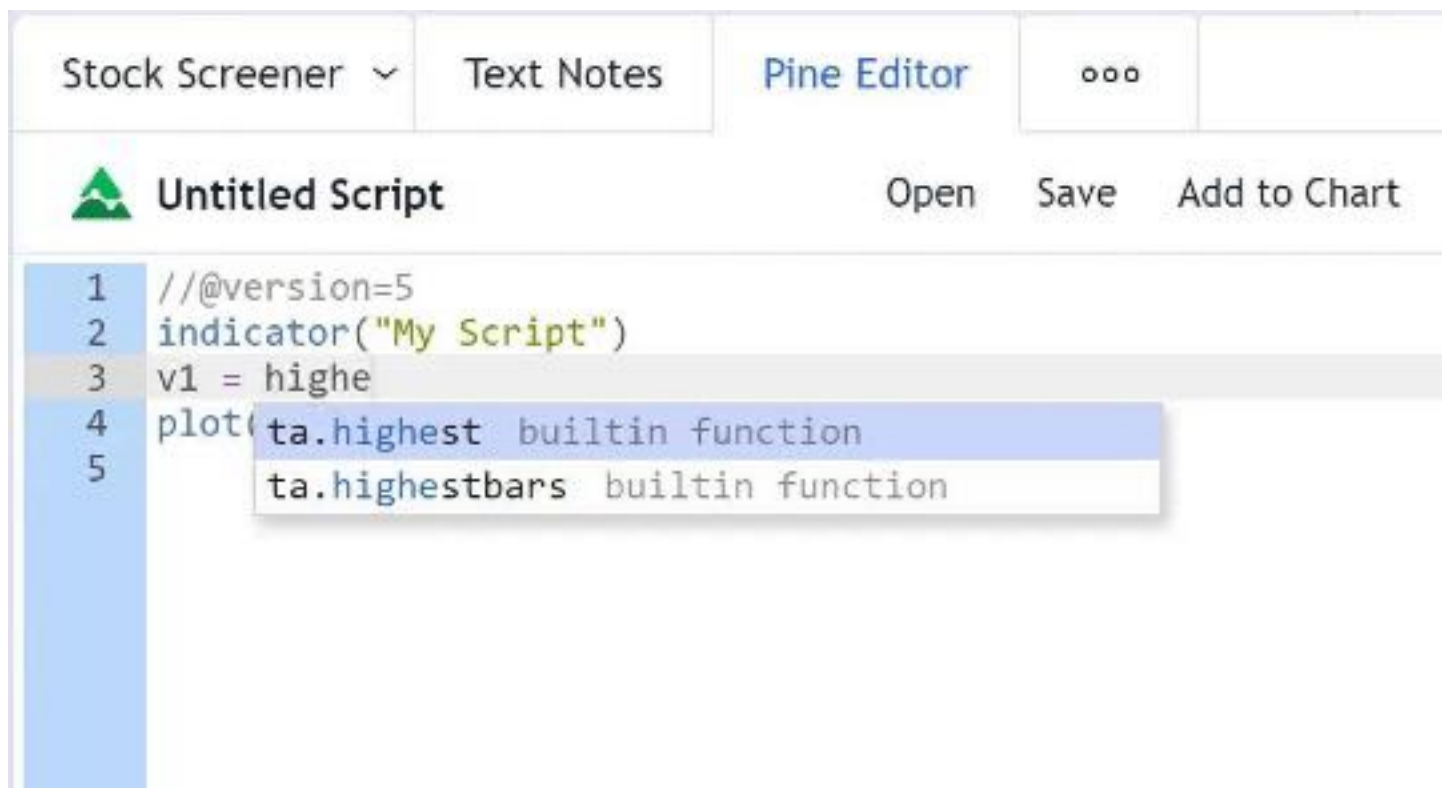


Figure 518: image

Not counting functions moved to new namespaces, only two functions have been renamed:

- `study()` is now `indicator()`.
- `tickerid()` is now `ticker.new()`.

The full list of renamed functions and variables can be found in the All variable, function, and parameter name changes section of this guide.

Renamed function parameters

The parameter names of some built-in functions were changed to improve the nomenclature. This has no bearing on most scripts, but if you used these parameter names when calling functions, they will require adaptation. For example, we have standardized all mentions:

```
// Valid in v4. Not valid in v5.
timev4 = time(resolution = "1D")
// Valid in v5.
timev5 = time(timeframe = "1D")
// Valid in v4 and v5.
timeBoth = time("1D")
```

The full list of renamed function parameters can be found in the All variable, function, and parameter name changes section of this guide.

Removed an `rsi()` overload

In v4, the `rsi()` function had two different overloads:

- `rsi(series float, simple int)` for the normal RSI calculation, and
- `rsi(series float, series float)` for an overload used in the MFI indicator, which did a calculation equivalent to $100.0 - (100.0 / (1.0 + \text{arg1} / \text{arg2}))$.

This caused a single built-in function to behave in two very different ways, and it was difficult to distinguish which one applied because it depended on the type of the second argument. As a result, a number of indicators misused the function and were displaying incorrect results. To avoid this, the second overload was removed in v5.

The `ta.rsi()` function in v5 only accepts a “simple int” argument for its `length` parameter. If your v4 code used the now deprecated overload of the function with a `float` second argument, you can replace the whole `rsi()` call with the following formula, which is equivalent:

```
100.0 - (100.0 / (1.0 + arg1 / arg2))
```

Note that when your v4 code used a “series int” value as the second argument to `rsi()`, it was automatically cast to “series float” and the second overload of the function was used. While this was syntactically correct, it most probably did **not** yield the result you expected. In v5, `ta.rsi()` requires a “simple int” for the argument to `length`, which precludes dynamic (or “series”) lengths. The reason for this is that RSI calculations use the `ta.rma()` moving average, which is similar to `ta.ema()` in that it relies on a length-dependent recursive process using the values of previous bars. This makes it impossible to achieve correct results with a “series” length that could vary bar to bar.

If your v4 code used a length that was “const int”, “input int” or “simple int”, no changes are required.

Reserved keywords

A number of words are reserved and cannot be used for variable or function names. They are: `catch`, `class`, `do`, `ellipse`, `in`, `is`, `polygon`, `range`, `return`, `struct`, `text`, `throw`, `try`. If your v4 indicator uses any of these, rename your variable or function for the script to work in v5.

Removed `iff()` and `offset()`

The `iff()` and `offset()` functions have been removed. Code using the `iff()` function can be rewritten using the ternary operator:

```
// iff(<condition>, <return_when_true>, <return_when_false>)
// Valid in v4, not valid in v5
barColorIff = iff(close >= open, color.green, color.red)
// <condition> ? <return_when_true> : <return_when_false>
// Valid in v4 and v5
```

```
barColorTernary = close >= open ? color.green : color.red
```

Note that the ternary operator is evaluated “lazily”; only the required value is calculated (depending on the condition’s evaluation to `true` or `false`). This is different from `iff()`, which always evaluated both values but returned only the relevant one.

Some functions require evaluation on every bar to correctly calculate, so you will need to make special provisions for these by pre-evaluating them before the ternary:

```
// `iff()` in v4: `highest()` and `lowest()` are calculated on every bar
v1 = iff(close > open, highest(10), lowest(10))
plot(v1)
// In v5: forced evaluation on every bar prior to the ternary statement.
h1 = ta.highest(10)
l1 = ta.lowest(10)
v1 = close > open ? h1 : l1
plot(v1)
```

The `offset()` function was deprecated because the more readable `[]` operator is equivalent:

```
// Valid in v4. Not valid in v5.
prevClosev4 = offset(close, 1)
// Valid in v4 and v5.
prevClosev5 = close[1]
```

Split of `input()` into several functions

The `v4 input()` function was becoming crowded with a plethora of overloads and parameters. We split its functionality into different functions to clear that space and provide a more robust structure to accommodate the additions planned for inputs. Each new function uses the name of the `input.*` type of the `v4 input()` call it replaces. E.g., there is now a specialized `input.float()` function replacing the `v4 input(1.0, type = input.float)` call. Note that you can still use `input(1.0)` in `v5`, but because only `input.float()` allows for parameters such as `minval`, `maxval`, etc., it is more powerful. Also note that `input.int()` is the only specialized input function that does not use its equivalent `v4 input.integer` name. The `input.*` constants have been removed because they were used as arguments for the `type` parameter, which was deprecated.

To convert, for example, a `v4` script using an input of type `input.symbol`, the `input.symbol()` function must be used in `v5`:

```
// Valid in v4. Not valid in v5.
aaplTicker = input("AAPL", type = input.symbol)
// Valid in v5
aaplTicker = input.symbol("AAPL")
```

The `input()` function persists in `v5`, but in a simpler form, with less parameters. It has the advantage of automatically detecting input types “`bool/color/int/float/string/source`” from the argument used for `defval`:

```
// Valid in v4 and v5.
// While "AAPL" is a valid symbol, it is only a string here because `input.symbol()` is not used.
tickerString = input("AAPL", title = "Ticker string")
```

Some function parameters now require built-in arguments

In `v4`, built-in constants such as `plot.style_area` used as arguments when calling Pine Script functions corresponded to pre-defined values of a specific type. For example, the value of `barmerge.lookahead_on` was `true`, so you could use `true` instead of the named constant when supplying an argument to the `lookahead` parameter in a `security()` function call. We found this to be a common source of confusion, which caused unsuspecting programmers to produce code yielding unintended results.

In `v5`, the use of correct built-in named constants as arguments to function parameters requiring them is mandatory:

```
// Not valid in v5: `true` is used as an argument for `lookahead`.
request.security(syminfo.tickerid, "1D", close, lookahead = true)
// Valid in v5: uses a named constant instead of `true`.
request.security(syminfo.tickerid, "1D", close, lookahead = barmerge.lookahead_on)

// Would compile in v4 because `plot.style_columns` was equal to 5.
// Won't compile in v5.
```



```
a = 2 * plot.style_columns
plot(a)
```

To convert your script from v4 to v5, make sure you use the correct named built-in constants as function arguments.

Deprecated the **transp** parameter

The **transp=** parameter used in the signature of many v4 plotting functions was deprecated because it interfered with RGB functionality. Transparency must now be specified along with the color as an argument to parameters such as **color**, **textcolor**, etc. The **color.new()** or **color.rgb()** functions will be needed in those cases to join a color and its transparency.

Note that in v4, the **bgcolor()** and **fill()** functions had an optional **transp** parameter that used a default value of 90. This meant that the code below could display Bollinger Bands with a semi-transparent fill between two bands and a semi-transparent background color where bands cross price, even though no argument is used for the **transp** parameter in its **bgcolor()** and **fill()** calls:

```
//@version=4
study("Bollinger Bands", overlay = true)
[middle, upper, lower] = bb(close, 5, 4)
plot(middle, color=color.blue)
p1PlotID = plot(upper, color=color.green)
p2PlotID = plot(lower, color=color.green)
crossUp = crossover(high, upper)
crossDn = crossunder(low, lower)
// Both `fill()` and `bgcolor()` have a default `transp` of 90
fill(p1PlotID, p2PlotID, color = color.green)
bgcolor(crossUp ? color.green : crossDn ? color.red : na)
```

In v5 we need to explicitly mention the 90 transparency with the color, yielding:

```
//@version=5
indicator("Bollinger Bands", overlay = true)
[middle, upper, lower] = ta.bb(close, 5, 4)
plot(middle, color=color.blue)
p1PlotID = plot(upper, color=color.green)
p2PlotID = plot(lower, color=color.green)
crossUp = ta.crossover(high, upper)
crossDn = ta.crossunder(low, lower)
var TRANSP = 90
// We use `color.new()` to explicitly pass transparency to both functions
fill(p1PlotID, p2PlotID, color = color.new(color.green, TRANSP))
bgcolor(crossUp ? color.new(color.green, TRANSP) : crossDn ? color.new(color.red, TRANSP) : na)
```

Changed the default session days for **time()** and **time_close()**

The default set of days for **session** strings used in the **time()** and **time_close()** functions, and returned by **input.session()**, has changed from "23456" (Monday to Friday) to "1234567" (Sunday to Saturday):

```
// On symbols that are traded during weekends, this will behave differently in v4 and v5.
t0 = time("1D", "1000-1200")
// v5 equivalent of the behavior of `t0` in v4.
t1 = time("1D", "1000-1200:23456")
// v5 equivalent of the behavior of `t0` in v5.
t2 = time("1D", "1000-1200:1234567")
```

This change in behavior should not have much impact on scripts running on conventional markets that are closed during weekends. If it is important for you to ensure your session definitions preserve their v4 behavior in v5 code, add ":23456" to your session strings. See this manual's page on Sessions for more information.

strategy.exit() now must do something

Gone are the days when the **strategy.exit()** function was allowed to loiter. Now it must actually have an effect on the strategy by using at least one of the following parameters: **profit**, **limit**, **loss**, **stop**, or one of the following pairs: **trail_offset**

combined with either `trail_price` or `trail_points`. When uses of `strategy.exit()` not meeting these criteria trigger an error while converting a strategy to v5, you can safely eliminate these lines, as they didn't do anything in your code anyway.

Common script conversion errors

Invalid argument 'style'/'linestyle' in 'plot'/'hline' call

To make this work, you need to change the "int" arguments used for the `style` and `linestyle` arguments in `plot()` and `hline()` for built-in constants:

```
// Will cause an error during conversion
plotStyle = input(1)
hlineStyle = input(1)
plot(close, style = plotStyle)
hline(100, linestyle = hlineStyle)

// Will work in v5
//@version=5
indicator("")

plotStyleInput = input.string("Line", options = ["Line", "Stepline", "Histogram", "Cross", "Area", "Columns",
hlineStyleInput = input.string("Solid", options = ["Solid", "Dashed", "Dotted"])

plotStyle = plotStyleInput == "Line" ? plot.style_line :
    plotStyleInput == "Stepline" ? plot.style_stepline :
    plotStyleInput == "Histogram" ? plot.style_histogram :
    plotStyleInput == "Cross" ? plot.style_cross :
    plotStyleInput == "Area" ? plot.style_area :
    plotStyleInput == "Columns" ? plot.style_columns :
    plot.style_circles

hlineStyle = hlineStyleInput == "Solid" ? hline.style_solid :
    hlineStyleInput == "Dashed" ? hline.style_dashed :
    hline.style_dotted

plot(close, style = plotStyle)
hline(100, linestyle = hlineStyle)
```

See the [Some function parameters now require built-in arguments](#) section of this guide for more information.

Undeclared identifier 'input.%input__name%'

To fix this issue, remove the `input.*` constants from your code:

```
// Will cause an error during conversion
_integer = input.integer
_bool = input.bool
i1 = input(1, "Integer", _integer)
i2 = input(true, "Boolean", _bool)

// Will work in v5
i1 = input.int(1, "Integer")
i2 = input.bool(true, "Boolean")
```

See the [User Manual's page on Inputs](#), and the [Some function parameters now require built-in arguments](#) section of this guide for more information.

Invalid argument 'when' in 'strategy.close' call

This is caused by a confusion between `strategy.entry()` and `strategy.close()`.

The second parameter of `strategy.close()` is `when`, which expects a "bool" argument. In v4, it was allowed to use `strategy.long` an argument because it was a "bool". With v5, however, named built-in constants must be used as arguments, so `strategy.long` is no longer allowed as an argument to the `when` parameter.

The `strategy.close("Short", strategy.long)` call in this code is equivalent to `strategy.close("Short")`, which is what must be used in v5:

```
// Will cause an error during conversion
if (longCondition)
    strategy.close("Short", strategy.long)
    strategy.entry("Long", strategy.long)

// Will work in v5:
if (longCondition)
    strategy.close("Short")
    strategy.entry("Long", strategy.long)
```

See the [Some function parameters now require built-in arguments](#) section of this guide for more information.

Cannot call ‘input.int’ with argument ‘minval’=‘%value%’. An argument of ‘literal float’ type was used but a ‘const int’ is expected

In v4, it was possible to pass a “float” argument to `minval` when an “int” value was being input. This is no longer possible in v5; “int” values are required for “int” inputs:

```
// Works in v4, will break on conversion because minval is a 'float' value
int_input = input(1, "Integer", input.integer, minval = 1.0)

// Works in v5
int_input = input.int(1, "Integer", minval = 1)
```

See the [User Manual’s page on Inputs](#), and the [Some function parameters now require built-in arguments](#) section of this guide for more information.

All variable, function, and parameter name changes

Removed functions and variables

v4v5 inputReplaced by `input.bool()` inputReplaced by `input.color()` inputReplaced by `input.float()` inputReplaced by `input.int()` inputReplaced by `input.timeframe()` inputReplaced by `input.session()` inputReplaced by `input.source()` inputReplaced by `input.string()` inputReplaced by `input.symbol()` inputReplaced by `input.time()``iff()` Use the `?:` operator instead`offset()` Use the `[]` operator instead### Renamed functions and parameters

No namespace change v4v5study(<...>, resolution, resolution_gaps, <...>)`indicator(<...>, timeframe, timeframe_gaps, <...>)``strategy.entry(long)``strategy.entry(direction)``strategy.order(long)``strategy.order(y)``nz(source, replacement)`##### “ta” namespace for technical analysis functions and variables

Indicator functions and variables v4v5accdist`ta.accdist``alma()``ta.alma()``atr()``ta.atr()``bb()``ta.bb()``bb(y)``ta.rsi(source, length)``sar()``ta.sar()``sma()``ta.sma()``stoch()``ta.stoch()``supertrend()``ta.supertrend`Supporting functions

v4v5barsince()`ta.barsince()``change()``ta.change()``correlation(source_a, source_b, length)``ta.correlation(source2, length)``cross(x, y)``ta.cross(source1, source2)``crossover(x, y)``ta.crossover(source1, source2)``crossunder(x, y)``ta.crossunder(source1, source2)``cum(x)``ta.cum(source)``dev()``ta.dev()``falling(“math”` namespace for math-related functions and variables

v4v5abs(x)`math.abs(number)``acos(x)``math.acos(number)``asin(x)``math.asin(number)``atan(x)``math.atan(number, precision)``math.round(number, precision)``round_to_mintick(x)``math.round_to_mintick(number)``sign(x)``math.s“request”` namespace for functions that request external data

v4v5financial()`request.financial()``quandl()``request.quandl()``security(<...>, resolution, <...>)``request.timeframe, <...>)``splits()``request.splits()``dividends()``request.dividends()``earnings()``request.earnings(“ticker”` namespace for functions that help create tickers

v4v5heikinashi()`ticker.heikinashi()``kagi()``ticker.kagi()``linebreak()``ticker.linebreak()``pointfigure()`“str” namespace for functions that manipulate strings

```
v4v5toString(x, y)`str.toString(value, format)`tonumber(x)`str.tonumber(string)
```

[\[Previous\]](#)

[To Pine Script™ version 6\]\(#to-pine-version-6\)\[Next](#)

[To Pine Script™ version 4\]\(#to-pine-version-4\)](#) [User Manual/Migration guides/To Pine Script™ version 4](#)

To Pine Script™ version 4

This is a guide to converting Pine Script code from @version=3 to @version=4.

Converter

The Pine Editor can automatically convert v3 indicators and strategies to v4. The Pine converter is described in the Overview page.

Not all scripts can be automatically converted from v3 to v4. If you want to convert the script manually or if your indicator returns a compilation error after conversion, consult the guide below for more information.

Renaming of built-in constants, variables, and functions

In Pine Script v4 the following built-in constants, variables, and functions were renamed:

- Color constants (e.g. `red`) are moved to the `color.*` namespace (e.g. `color.red`).
- The `color` function has been renamed to `color.new`.
- Constants for `input()` types (e.g. `integer`) are moved to the `input.*` namespace (e.g. `input.integer`).
- The plot style constants (e.g. `histogram` style) are moved to the `plot.style_*` namespace (e.g. `plot.style_histogram`).
- Style constants for the `hline` function (e.g. the `dotted` style) are moved to the `hline.style_*` namespace (e.g. `hline.style_dotted`).
- Constants of days of the week (e.g. `sunday`) are moved to the `dayofweek.*` namespace (e.g. `dayofweek.sunday`).
- The variables of the current chart timeframe (e.g. `period`, `isintraday`) are moved to the `timeframe.*` namespace (e.g. `timeframe.period`, `timeframe.isintraday`).
- The `interval` variable was renamed to `timeframe.multiplier`.
- The `ticker` and `tickerid` variables are renamed to `syminfo.ticker` and `syminfo.tickerid` respectively.
- The `n` variable that contains the bar index value has been renamed to `bar_index`.

The reason behind renaming all of the above was to structure the standard language tools and make working with code easier. New names are grouped according to assignments under common prefixes. For example, you will see a list with all available color constants if you type 'color' in the editor and press Ctrl + Space.

Explicit variable type declaration

In Pine Script v4 it's no longer possible to create variables with an unknown data type at the time of their declaration. This was done to avoid a number of issues that arise when the variable type changes after its initialization with the `na` value. From now on, you need to explicitly specify their type using keywords or type functions (for example, `float`) when declaring variables with the `na` value:

```
//@version=4
study("Green Candle Close")
// We expect `src` to hold float values, so we declare in with the `float` keyword
float src = na
if close > open
    src := close
plot(src)
```

[\[Previous\]](#)

[To Pine Script™ version 5\]\(#to-pine-version-5\)\[Next](#)

[To Pine Script™ version 3\]\(#to-pine-version-3\)](#) [User Manual/Migration guides/To Pine Script™ version 3](#)

To Pine Script™ version 3

This document helps to migrate Pine Script code from @version=2 to @version=3.

Default behaviour of security function has changed

Let's look at the simple `security` function use case. Add this indicator on an intraday chart:

```
// Add this indicator on an intraday (e.g., 30 minutes) chart
//@version=2
study("My Script", overlay=true)
s = security(tickerid, 'D', high, false)
plot(s)
```

This indicator is calculated based on historical data and looks somewhat *into the future*. At the first bar of every session an indicator plots the high price of the entire day. This could be useful in some cases for analysis, but doesn't work for backtesting strategies.

We worked on this and made changes in Pine Script version 3. If this indicator is compiled with `//@version=3` directive, we get



a completely different picture:

The old behaviour is still available though. We added a parameter to the `security` function (the fifth one) called `lookahead`.

It can take on the form of two different values: `barmerge.lookahead_off` (and this is the default for Pine Script version 3) or `barmerge.lookahead_on` (which is the default for Pine Script version 2).

Self-referenced variables are removed

Pine Script version 2 pieces of code, containing a self-referencing variable:

```
//@version=2
//...
s = nz(s[1]) + close
```

Compiling this piece of code with Pine Script version 3 will give you an `Undeclared identifier 's'` error. It should be rewritten as:

```
//@version=3
//...
```

```
s = 0.0
s := nz(s[1]) + close
```

`s` is now a *mutable variable* that is initialized at line 3. At line 3 the initial value gives the Pine Script compiler the information about the variable type. It's a float in this example.

In some cases you may initialize that mutable variable (like `s`) with a `na` value. But in complex cases that won't work.

Forward-referenced variables are removed

```
//@version=2
//...
d = nz(f[1])
e = d + 1
f = e + close
```

In this example `f` is a forward-referencing variable, because it's referenced at line 3 before it was declared and initialized. In Pine Script version 3 this will give you an error **Undeclared identifier 'f'**. This example should be rewritten in Pine Script version 3 as follows:

```
//@version=3
//...
f = 0.0
d = nz(f[1])
e = d + 1
f := e + close
```

Resolving a problem with a mutable variable in a security expression

When you migrate script to version 3 it's possible that after removing self-referencing and forward-referencing variables the Pine Script compiler will give you an error:

```
//@version=3
//...
s = 0.0
s := nz(s[1]) + close
t = security(tickerid, period, s)
```

Cannot use mutable variable as an argument for security function!

This limitation exists since mutable variables were introduced in Pine Script, i.e., in version 2. It can be resolved as before: wrap the code with a mutable variable in a function:

```
//@version=3
//...
calcS() =>
    s = 0.0
    s := nz(s[1]) + close
t = security(tickerid, period, calcS())
```

Math operations with booleans are forbidden

In Pine Script v2 there were rules of implicit conversion of booleans into numeric types. In v3 this is forbidden. There is a conversion of numeric types into booleans instead (0 and `na` values are **false**, all the other numbers are **true**). Example (In v2 this code compiles fine):

```
//@version=2
study("My Script")
s = close >= open
s1 = close[1] >= open[1]
s2 = close[2] >= open[2]
sum = s + s1 + s2
col = sum == 1 ? white : sum == 2 ? blue : sum == 3 ? red : na
bgcolor(col)
```

Variables `s`, `s1` and `s2` are of *bool* type. But at line 6 we add three of them and store the result in a variable `sum`. `sum` is a number, since we cannot add booleans. Booleans were implicitly converted to numbers (`true` values to 1.0 and `false` to 0.0) and then they were added.

This approach leads to unintentional errors in more complicated scripts. That's why we no longer allow implicit conversion of booleans to numbers.

If you try to compile this example as a Pine Script v3 code, you'll get an error: **Cannot call operator + with arguments (series__bool, series__bool); <...>** It means that you cannot use the addition operator with boolean values. To make this example work in Pine Script v3 you can do the following:

```
//@version=3
study("My Script")
bton(b) =>
    b ? 1 : 0
s = close >= open
s1 = close[1] >= open[1]
s2 = close[2] >= open[2]
sum = bton(s) + bton(s1) + bton(s2)
col = sum == 1 ? white : sum == 2 ? blue : sum == 3 ? red : na
bgcolor(col)
```

Function `bton` (abbreviation of boolean-to-number) explicitly converts any boolean value to a number if you really need this.

[Previous

To Pine Script™ version 4](#to-pine-version-4)[Next

To Pine Script™ version 2](#to-pine-version-2) User Manual/Migration guides/To Pine Script™ version 2

To Pine Script™ version 2

Pine Script version 2 is fully backwards compatible with version 1. As a result, all v1 scripts can be converted to v2 by adding the `//@version=2` annotation to them.

An example v1 script:

```
study("Simple Moving Average", shorttitle="SMA")
src = close
length = input(10)
plot(sma(src, length))
```

The converted v2 script:

```
//@version=2
study("Simple Moving Average", shorttitle="SMA")
src = close
length = input(10)
plot(sma(src, length))
```

[Previous

To Pine Script™ version 3](#to-pine-version-3) User Manual/Where can I get more information?

Where can I get more information?

- A description of all the Pine Script™ operators, variables, and functions can be found in the Reference Manual.
- Use the code from one of TradingView's built-in scripts to start from. Open a new chart and select the "Pine Editor" button in the toolbar. Once the editor window opens, click the dropdown menu and select "Create new/Built in..." to open a dialog box containing a list of TradingView's built-in scripts.
- There is a TradingView public chat dedicated to Pine Script™ Q&A where active developers of our community help each other out.

- Information about major releases and modifications to Pine Script™ (as well as other features) is regularly published on TradingView's blog.
- TradingView's Community scripts contain all user-published scripts. Access them from TradingView's homepage by selecting "Community/Indicators and strategies", or from the charts by using the "Indicators, metrics, and strategies" button and selecting the "Community" tab of the script searching dialog box.
- The Knowledge Base provides many articles about all aspects of the TradingView platform, including indicators, alerts, and Pine Script.

External resources

- You can ask questions about programming in Pine Script™ in the `[pine-script]` tag on StackOverflow.
- The `/r/TradingView` subreddit is the place for all TradingView-related feature requests, including suggestions about Pine Script functionality.

[Previous

To Pine Script™ version 3](#to-pine-version-3)